

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

И. Ф. Астахова, Е. В. Трофименко, Ю. В. Хицкова

**ПРОЕКТИРОВАНИЕ
ИНФОРМАЦИОННЫХ СИСТЕМ.
ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ
ПРОЕКТИРОВАНИЕ**

Учебное пособие

Воронеж
Издательский дом ВГУ
2025

УДК 004.652.5
ББК 32.972.1
А91

Р е ц е н з е н т –
доктор технических наук, профессор кафедры математических
методов исследования операций
Воронежского государственного университета *Ю. В. Бондаренко*

Астахова И. Ф.

А91 Проектирование информационных систем. Объектно-ориентированное проектирование : учебное пособие / И. Ф. Астахова, Е. В. Трофименко, Ю. В. Хицкова; Воронежский государственный университет. – Воронеж : Издательский дом ВГУ, 2025. – 135 с.

Учебное пособие содержит описание языка для проектирования информационных систем и подборку примеров, задач и упражнений различной степени сложности для обеспечения лабораторных занятий по изучению этих возможностей в рамках учебного курса «Проектирование ИС», посвященного информационным системам.

Для обучающихся по направлению подготовки 02.04.02 «Фундаментальная информатика и информационные технологии» и направлению «Математическое обеспечение и администрирование информационных систем» по дисциплине Б1.В.06 «Проектирование информационных систем».

УДК 004.652.5
ББК 32.972.1

© Астахова И. Ф., Трофименко Е. В.,
Хицкова Ю. В., 2025
© Воронежский государственный
университет, 2025
© Оформление. Издательский дом ВГУ,
2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ОСНОВНЫЕ ПОНЯТИЯ, ОПРЕДЕЛЕНИЯ	5
1.1. Сложность программного обеспечения	5
1.2. Объектно-ориентированная технология проектирования	8
2. ДИАГРАММЫ ВИЗУАЛЬНОГО МОДЕЛИРОВАНИЯ ЯЗЫКА UML 2.0	13
2.1. Use case diagram (диаграмма сценариев)	13
2.2. Диаграмма классов (class diagram)	33
2.3. <i>Sequence diagram (диаграммы последовательностей действий)</i>	53
2.4. Package diagram (диаграмма пакетов)	62
2.5. Диаграмма компонентов	69
2.6. Диаграмма деятельности (activity diagram)	71
2.7. Диаграмма конечного автомата (state machine diagram)	82
2.8. Диаграмма развертывания (deployment diagram)	88
3. ПРИМЕРЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНЫХ РАБОТ	93
СПИСОК ЛИТЕРАТУРЫ	113
ПРИЛОЖЕНИЕ - ЗАДАЧИ ДЛЯ ВЫПОЛНЕНИЯ ЛАБОРАТОРНЫХ РАБОТ	114

ВВЕДЕНИЕ

Под проектированием информационных систем в широком смысле понимается процесс преобразования входной информации об объекте проектирования, о методах проектирования и об опыте проектирования объектов аналогичного назначения в соответствии с ГОСТом в проект ИС (действующую ИС и проектно-технологическую документацию). В узком смысле – это вторая стадия ЖЦ ИС. Технология проектирования ИС – это совокупность методологии и средств проектирования ИС, а также методов и средств организации проектирования (управления процессом создания и модернизации проекта ИС). Рассматривается технология структурного проектирования и объектно-ориентированного. Структурное проектирование рассматривается с помощью DEF, DEF3 и DFD.

Объектно ориентированный подход к проектированию больших программных систем в последнее десятилетие стал безусловным стандартом, поскольку способствует лучшей управляемости проектом на всех стадиях разработки, тестирования и внедрения. Проектирование IDEF0, IDEF3, DFD рассматриваются с помощью ERWIN или BPWIN. Объектно-ориентированное проектирование рассматривается с помощью Star UML.

1. ОСНОВНЫЕ ПОНЯТИЯ, ОПРЕДЕЛЕНИЯ

1.1. Сложность программного обеспечения

Информационные системы имеют программное обеспечение, так называемое software.

В середине прошлого века программы разрабатывались непосредственно на машинном языке. Эти программы представляли собой очень длинные последовательности нулей и единиц, которые человек вводил при помощи переключателей или перфолент, перфокарт и т.п. Позже были созданы языки программирования, сначала машинно-ориентированный (Assembler), а затем высокого уровня (Алгол, Фортран, Basic, Pascal и др.), которые позволили существенно облегчить работу программистов. Сейчас с появлением систем визуального программирования (Visual Basic, Delphi и др.) создание программ стало доступно даже для начинающих пользователей.

Вместе с тем при решении определенной практической задачи в реальной жизни приходится сталкиваться с необходимостью проектирования информационных систем, построенных на базе компьютерной техники, предназначенных для хранения, поиска, обработки и передачи больших объемов информации. Таких задач очень много, например, создание информационно-поисковой системы традиционной книжной библиотеки или проектирование системы автоматического управления производством. Понятно, что сколь бы и был гениален один человек, он не в состоянии выполнить такую задачу, именно поэтому на современном этапе на первый план выходит объектно-ориентированное проектирование.

Промышленные программные продукты обладают некоторым уровнем сложности. Сложность программного продукта объясняется рядом объективных причин. Этими причинами являются: сложность окружающе-

го мира, трудность управления процессом разработки, гибкость программного обеспечения, так как разработчик может обеспечить себя всеми необходимыми элементами, проблемы, связанные с описанием поведения больших дискретных систем, которыми являются прикладные программы [1]. Рассмотрим подробнее объективные причины возникновения сложности программного продукта.

Сложность окружающего мира. Сложность окружающего мира объясняется «нестыковкой» между пользователем системы и ее разработчиками, пользователи, как правило, с трудом могут объяснить понятно разработчику, что необходимо сделать. У пользователя и разработчика разные взгляды на сущность проблемы способы ее решения. Дополнительные сложности возникают в результате изменений требований к программной системе в процессе разработки. Большая программная система – это крупное капиталовложение, и поэтому необходимо сохранение на каждом шаге, большая часть затрат идет на сохранение.

Трудность управления процессом разработки. Основная задача разработчика состоит в создании иллюзии простоты, в защите пользователей от сложности описываемого предмета или процесса. В настоящее время программные системы имеют размер, который оценивается десятками тысяч или даже миллионами строк, поэтому такой объем потребует привлечения команды разработчиков. Возникают трудности управления процессом разработки и с организацией коллективной разработки. Главная задача сохранить единство и целостность разработки.

Гибкость программного обеспечения, так как разработчик может обеспечить себя всеми необходимыми элементами в отличие от других отраслей, например, строительства, где есть блоки, из которых строится здание.

Проблемы, связанные с описанием поведения больших дискретных систем, которыми являются прикладные программы. Внутри большой прикладной программы могут существовать сотни и тысячи переменных. Исполнение программы происходит на цифровом компьютере, то имеется в каждый момент времени система с дискретными состояниями. Изменение входных параметров меняет дискретное состояние.

Каждая сложная система имеет пять признаков [1]:

1. Сложные системы часто являются иерархическими и состоят из взаимозависимых подсистем, которые в свою очередь могут быть разделены на подсистемы, и т. д., вплоть до самого низкого уровня.
2. Выбор, какие компоненты являются в данной системе элементарными, относительно произволен и оставляется на усмотрение исследователя.
3. Внутри компонентные связи обычно сильнее, чем связь между компонентами.
4. Иерархические системы обычно состоят из немногих типов подсистем, по-разному скомбинированных и организованных.
5. Любая работающая сложная система является результатом развития работающей более простой системы.

На первом этапе занимаются проектированием сложной системы.

Сложность программных систем возрастает, но человеческие способности ограничены, поэтому необходима технология проектирования сложных систем, позволяющая выйти из этого затруднительного положения. При этом под проектированием будет пониматься сложный итерационный процесс, при помощи которого требования к программной системе (ПС) транслируются в инженерное представление ПС.

Одним из способов управления сложными системами является метод декомпозиции. Известны две схемы декомпозиции: алгоритмическая и объектно-ориентированная. В основе алгоритмической декомпозиции лежит

разбиение по действиям-алгоритмам, объектно-ориентированная декомпозиция обеспечивает разбиение по автономным лицам – объектам реального (или виртуального) мира. Опыт показывает, что начинать необходимо с объектной декомпозиции, это поможет придать организованность сложной ПС. Объектно-ориентированное представление ПС основывается на принципах абстрагирования, инкапсуляции, модульности и иерархической организации. Совместное использование этих принципов рассчитано на проведение объектно ориентированной декомпозиции. Объектно ориентированная декомпозиция снижает сложность при разработке ПС, так как она развивается из меньших систем, в работе которых уже есть уверенность. Есть два метода проектирования структурное и объектно-ориентированное, основанный на разной декомпозиции.

1.2 Объектно-ориентированная технология проектирования

Что же такое объектно-ориентированная технология программирования и проектирования?

Объектно-ориентированная технология программирования – это методология, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром какого-то класса, а классы образуют иерархию наследования. Программа будет объектно-ориентированной только в том случае, если выполнены эти требования: 1) базовые элементы – это объекты; 2) каждый объект – экземпляр класса; 3) классы образуют иерархию.

Объектно-ориентированное проектирование – это методология, соединяющая в себе процесс объектной декомпозиции и приемы представления логической и физической, статической и динамической моделей системы. Данное определение состоит из двух частей: 1) объектно-

ориентированное проектирование основывается на объектной декомпозиции; 2) представляются модели: логическая (классы и объекты) и физическая (модули и процессы), а также их динамический и статический аспекты.

Многообразие объектно-ориентированных моделей можно изобразить на следующем рисунке [1]:

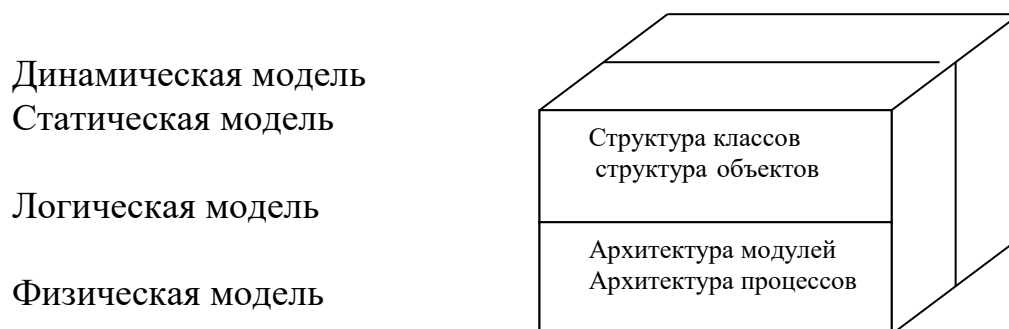


Рис.1. Объектно ориентированные модели

Модель – это представление чего-либо в некоторой среде, в модели отражаются только важные с определенной точки черты моделируемой сущности, а остальные передаются в упрощенном виде. Модель программного продукта создается с помощью языка моделирования, например, UML. Модель нужна для точного описания требований к системе и знаний о предметной области, чтобы все вовлеченные в проект стороны могли их понять и прийти к общему соглашению. Модель нужна для разработки плана системы, для отображения проектных решений различного рода, для создания промежуточных продуктов, для получения, фильтрации, организации, исследования и редактирования информации о больших системах, для экономной обработки нескольких возможных решений, и, наконец, модель нужна, чтобы справиться с разработкой сложной системы. На разных уровнях абстракции в зависимости от целей модели могут принимать различные формы. В самом начале проекта разрабатываются модели высокого уровня, в них определяются возможности системы. Далее эти модели заменяются более детальными и подробными. Модель имеет два основных аспекта: семантику и визуальное представление (нотация). Семантика опре-

деляет содержание модели. Нотация определяет визуальное представление модели.

Объектно ориентированный анализ – это методология, при которой требования к системе воспринимаются с точки зрения классов и объектов из данной предметной области.

Каждый стиль программирования имеет свою концептуальную базу, для объектно ориентированного стиля – это объектная модель. Объектная модель имеет четыре главных свойства, это [1]:

- абстрагирование – выделение существенных характеристик объектов, отличающих его от других видов объектов;
- инкапсуляция – скрывание внутренней реализации объекта;
- модульность – способность системы быть разложенной на внутренние сильно или слабо связанные между собой модули;
- иерархия – упорядочивание абстракций и расположение их по уровням.

Эти свойства главные для объектной модели, при отсутствии любого из них модель не будет объектной. Существуют три дополнительных свойства, это:

- типизация – создание объектов на основе шаблонов определенного типа;
- параллелизм – способность системы обрабатывать несколько задач параллельно;
- сохраняемость – способность хранить данные и объекты в промежутке между отдельными запусками системы.

Для проектирования объектно-ориентированных систем используют языки визуального моделирования. В качестве стандартного языка третьего поколения был принят Unified Modeling Language (UML). UML – стандарт-

ный язык для анализа, проектирования и реализации объектно-ориентированных систем. UML является составной частью программного пакета Rational Rose. Rational Rose – это визуальный редактор, позволяющий моделировать программные системы любой сложности на основе графических диаграмм языка UML. Rational Rose поддерживает не только UML, но и другие нотации создания диаграмм, однако UML – самый распространенный из всех.

Для разработки **статических моделей** используются диаграммы [1–4]:

- 1) **class diagram** (диаграмма классов);
- 2) **use case diagram** (диаграмма сценариев);
- 3) **packet и component diagram** (диаграммы пакетов и компонент);
- 4) **deployment diagram** (диаграмма развертывания).

Для разработки **динамических моделей** используются следующие диаграммы [3]:

- 1) **statechart diagram** (диаграмма состояний);
- 2) **interaction diagram** (диаграмма взаимодействия);
- 3) **sequence diagram** (диаграмма последовательностей действий);
- 4) **Activity diagram** (диаграмма активности).

Диаграммы прецедентов (диаграммы вариантов использования, use case diagrams) – это обобщенная модель функционирования системы в окружающей среде.

Диаграммы видов деятельности (activity diagrams) – модель бизнес-процесса или поведения системы в рамках прецедента.

Диаграммы взаимодействия (interaction diagrams) – модель процесса обмена сообщениями между объектами. Представляется в виде диаграммы последовательностей (sequence diagrams).

Диаграммы состояний (statechart diagrams) – модель динамического поведения системы и ее компонентов при переходе из одного состояния в другое.

Диаграммы классов (class diagrams) – логическая модель базовой структуры системы, отражает статическую структуру системы и связи между ее элементами.

Диаграммы пакетов и компонентов (component diagrams) – модель иерархии подсистем, отражает физическое размещение пакетов и компонент, приложений и интерфейсов ИС.

Диаграммы развертывания (диаграммы размещения, deployment diagrams) – модель физической архитектуры системы, отображает аппаратную конфигурацию ИС.

2. ДИАГРАММЫ ВИЗУАЛЬНОГО МОДЕЛИРОВАНИЯ ЯЗЫКА UML 2.0

2.1. Use case diagram (диаграмма сценариев)

Диаграмма вариантов использования предназначена для достижения следующих целей: определить общие границы функциональности проектируемой системы в контексте моделируемой предметной области; специфицировать требования к функциональному поведению проектируемой системы в форме вариантов использования; разработать исходную концептуальную модель системы для ее последующей детализации в форме логических и физических моделей; подготовить исходную документацию для взаимодействия разработчиков системы с ее заказчиками и пользователями. Этот тип диаграмм иначе называют Use case diagram или диаграммой функций, они позволяют создать список операций, которые выполняет система. Основными графическими элементами диаграммы вариантов использования являются варианты использования (**Use Case**) и актеры (**Actors**). В дополнение к ним на диаграмме вариантов использования могут изображаться комментарии.

Самый простой способ создания элементов этой диаграммы – это при помощи блока инструментов. После активации диаграммы Use Case появится блок инструментов (рис.2):

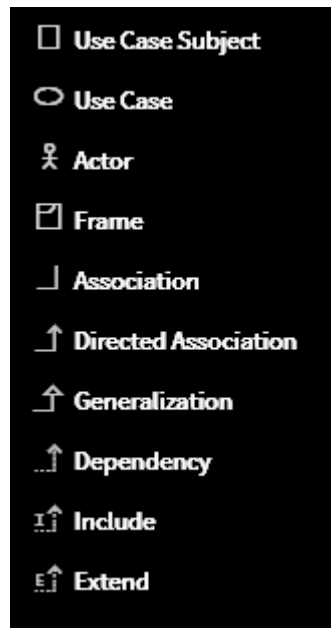


Рис. 2. Блок инструментов UML

2.1.1. Основные графические элементы

Actor (актер) – это роль объекта вне системы, который прямо взаимодействует с ее частью – конкретным элементом (элементом Use Case). Различают актеров и пользователей. Пользователь – это физический объект, который использует систему, он может моделироваться несколькими актерами, потому что может играть несколько ролей. Этот значок обозначает объект, который:

- взаимодействует с системой или использует систему;
- передает или принимает информацию в/из системы;
- является внешним по отношению к системе.

Актер (actor) представляет собой любую внешнюю по отношению к проектируемой системе сущность, которая взаимодействует с системой и использует ее функциональные возможности для достижения определенных целей или решения частных задач. Формально в контексте языка UML 2.0 каждый актер специфицирует некоторую роль, которую играет пользователь или любая другая система, взаимодействующая с субъектом. Термин "роль" здесь используется неформально. В модели вариантов использова-

ния пользователи и любые другие заинтересованные лица, которые могут взаимодействовать с субъектом, представляются как актеры. Актеры всегда моделируют сущности.

Общепринятым графическим обозначением актера на диаграммах в нотации языка UML 2.0 является фигурка "проволочного человечка", под которой записывается обязательное имя актера (рис. 3а). Поскольку актер является специализацией классификатора, он может изображаться также символом прямоугольника с ключевым словом `<<actor>>`, записанным в форме стереотипа выше его имени (рис. 3б). Дополнительно в языке UML 2.0 допускается изображать актера в форме предложенной разработчиком пиктограммы, внешний вид которой, по мнению самого разработчика, более наглядно иллюстрирует характер выполняемой им роли.

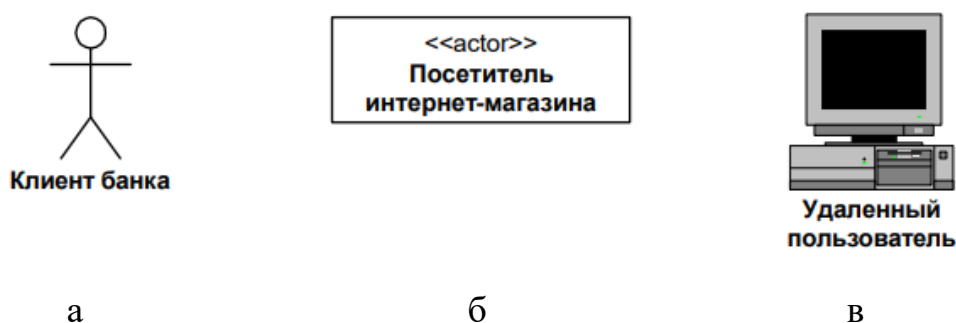


Рис. 3. Варианты символов графического обозначения актера: а — в виде "проволочного человечка"; б — в виде прямоугольника; в — в форме пиктограммы

Use Case (сценарии поведения, вариант использования) позволяет создавать простые формы сценариев поведения объектов системы. Это представление работы системы объектов, выполняющих в системе определенные функции. Один актер может использовать несколько элементов Use Case, и, наоборот, один элемент Use Case может иметь несколько актеров, использующих его. Набор всех Use Case определяет полные функциональные воз-

возможности системы. Вариант использования (use case) представляет собой общую спецификацию совокупности выполняемых системой действий с целью предоставления некоторого наблюдаемого результата, который имеет значение для одного или нескольких актеров. Другими словами, вариант использования представляет собой законченный фрагмент поведения системы с точки зрения тех или иных заинтересованных лиц. Хотя каждый вариант использования предполагает реализацию системой некоторой последовательности действий, которые должны быть выполнены с целью предоставления соответствующего результата или сервиса, сами эти действия на диаграмме вариантов использования никак не изображаются.

Общепринятым графическим изображением варианта использования в нотации языка UML 2.0 является эллипс, внутри которого или ниже его записывается имя (name) в форме строки текста. Имя используется для идентификации отдельного варианта использования, как элемента модели. Каждый вариант использования обязательно должен иметь некоторое имя, которое рекомендуется формулировать в форме законченного предложения, начинающегося, как правило, с существительного или глагола (рис.4а). Сам текст имени варианта использования должен начинаться с заглавной буквы. Также для изображения вариант использования в языке UML 2.0 на диаграмме может быть также использован символ стандартной нотации классификатора в форме прямоугольника с необязательными секциями для его характеристик (рис.4б).

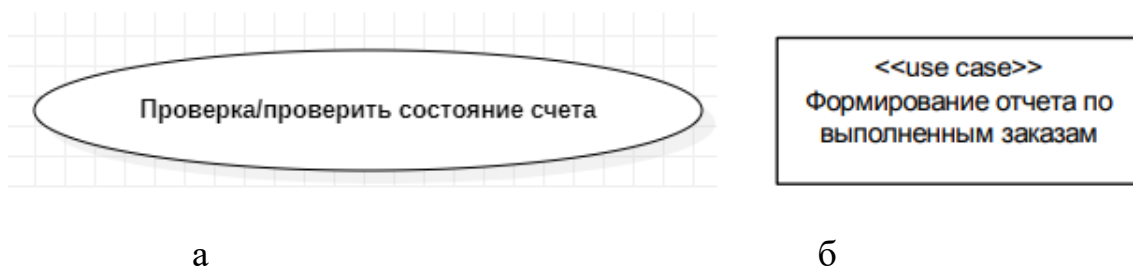


Рис. 4. Графическое обозначение варианта использования: а – стандартное; б – в форме символа классификатора с ключевым словом <<use case>>

2.1.2. Отношения на диаграмме вариантов использования

Отношение ассоциация (association) является одним из фундаментальных понятий в языке UML и может использоваться на различных канонических диаграммах при построении визуальных моделей. Применительно к диаграммам вариантов использования отношение ассоциации может служить только для обозначения взаимодействия актера с вариантом использования. Другими словами, на диаграмме вариантов использования ассоциация всегда является бинарной и специфицирует семантические особенности отдельного взаимодействия актера и варианта использования.

На диаграмме вариантов использования, так же, как и на других диаграммах, отношение ассоциации обозначается сплошной линией, соединяющей актера и вариант использования. При этом следует различать направленные и ненаправленные ассоциации. Если направление взаимодействия актера и вариантом использования для разработчика не имеет принципиального значения, то такое отношение может быть представлено в форме ненаправленной ассоциации. Если же, по мнению разработчика, направление взаимодействия между актером и вариантом использования имеет значение, то такое отношение может быть представлено в форме направленной ассоциации. Также на диаграмме могут размещаться комментарий (рис.5).

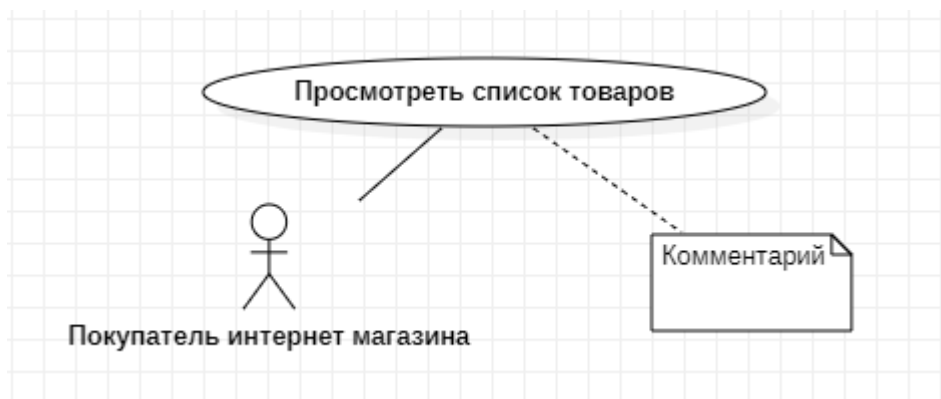


Рис. 5. Пример графического представления отношения ненаправленной ассоциации между актером и вариантом использования

Undirectional Association позволяет обозначить связь между элементами актера и Use Case. Между ними возможен один вид отношения – ассоциация, она может быть помечена именем, ролями и мощностью.

Отношение обобщения (generalization) предназначено для спецификации того факта, что один элемент модели является специальным или частным случаем другого элемента модели. Главной особенностью отношения обобщения является то, что оно может связывать между собой **только элементы одного типа**. Графически отношение обобщения обозначается сплошной линией со стрелкой в форме не закрашенного треугольника, которая указывает на общий элемент модели (рис.6).

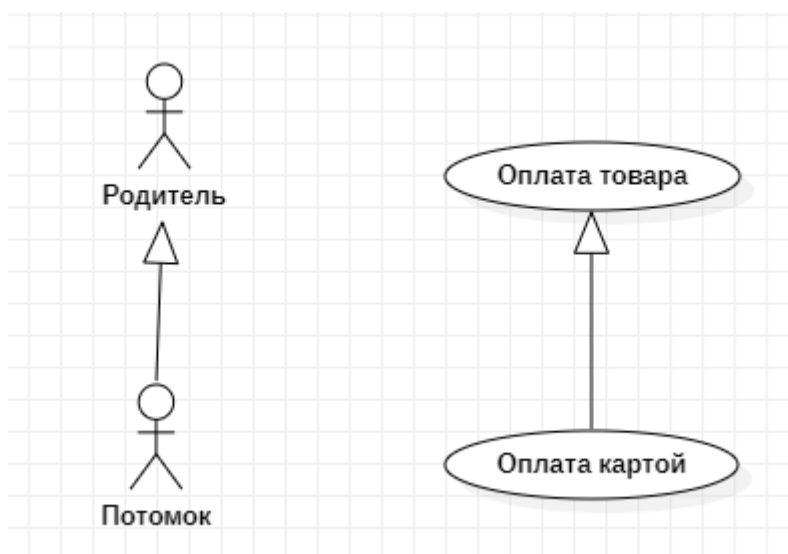


Рис. 6. Пример графического изображения отношения обобщения

В языке UML 2.0 один вариант использования может иметь несколько родительских вариантов использования. В этом случае реализуется так называемое множественное наследование свойств и поведения всех родительских вариантов использования. С другой стороны, один вариант использования может быть предком для нескольких дочерних вариантов использования, что соответствует таксономическому характеру отношения обобщения.

Отношение включения между двумя вариантами использования в языке UML 2.0 является частным случаем общего отношения зависимости.

Отношение зависимости (dependency) определяется как форма взаимосвязи между двумя элементами модели, предназначенная для спецификации того обстоятельства, что изменение одного элемента модели приводит к изменению некоторого другого элемента. В общем случае зависимость является направленным бинарным отношением, которое связывает между собой только два элемента модели — независимый и зависимый. На диаграмме вариантов использования существуют две разновидности отношения зависимости — включения и расширения.

Отношение включения (include) специфицирует тот факт, что некоторый вариант использования содержит поведение, определенное в другом варианте использования.

Семантически данное отношение означает, что поведение или выполнение одного варианта использования включается в качестве составного фрагмента в поведение или выполнение другого варианта использования. Отношение включения является направленным бинарным отношением в том смысле, что два варианта использования всегда упорядочены в отношении включения. Отношение включения может связывать в модели только **два варианта использования**.

Графически данное отношение обозначается как отношение зависимости в форме пунктирной линии с "V"-образной стрелкой, направленной от зависимого варианта использования к независимому варианту использования. Зависимый вариант использования часто называют также *базовым* или *включающим*, а независимый — *включаемым* вариантом использования. При этом линия со стрелкой обязательно помечается ключевым словом <<include>>, записанным в форме стереотипа (рис. 7).



Рис. 7. Отношение включения между элементами Use Case

Отношение включения между элементами Use Case означает, что базовый элемент Use Case явно включает поведение другого элемента, включающий элемент никогда не используется самостоятельно — его конкретизация может быть только частью другого.

Отношение зависимости на рис. 7 означает, что каждое выполнение варианта «Оформление заказа в ИМ» всегда будет включать в себя функциональное поведение или выполнение варианта использования «Регистрация покупателя».

На практике отношение включения применяется для моделирования ситуаций, когда существуют общие части поведения двух или более вариантов использования. Тогда эта общая часть может быть выделена в отдельный вариант использования, чтобы затем включить его во все базисные варианты использования, имеющие эту часть в своем поведении. При этом главное назначение отношения включения связано с повторным использо-

ванием этой общей части, которая не указывается в базисных вариантах использования.

В общем случае один базовый вариант использования может быть связан отношением включения с несколькими включаемыми вариантами использования, а также самому быть включаемым в другие варианты использования.

Важно! Разработчикам следует помнить, что на одной диаграмме вариантов использования не может быть замкнутого пути по отношению включения. В противном случае такая диаграмма будет являться семантически противоречивой или не согласованной

Отношение расширения (extend) между Use Case означает, что базовый элемент Use Case неявно включает поведение другого элемента Use Case. Базовый элемент может быть автономен, но при определенных условиях его поведение может расширяться поведением из другого элемента Use Case. Отношение расширения определяет взаимосвязь одного варианта использования с некоторым другим вариантом использования, функциональность или поведение которого задействуется первым не всегда, а только при выполнении некоторых дополнительных условий.



Рис.8. Отношение расширения между элементами Use Case

Важно! Разработчикам следует помнить, что на одной диаграмме вариантов использования не может быть замкнутого пути по отношению расширения. В противном случае такая диаграмма будет являться семантически противоречивой или несогласованной

2.1.3. Пример диаграммы вариантов использования для системы продажи товаров в интернет-магазине Анализ сценариев использования системы

Проанализировав функциональные диаграммы и определив, какие события в системе необходимо автоматизировать, можно приступить к построению диаграмм прецедентов.

Диаграмма прецедентов описывает типичное взаимодействие между пользователем и системой. На диаграмме прецедентов человеческие фигурки (актеры) обозначают действующих лиц, овалы – прецеденты, а линии и стрелки – различные связи между действующими лицами и прецедентами.

Диаграмма прецедентов, как правило, отражает требования к системе с точки зрения пользователя. Таким образом, прецеденты – это функции, выполняемые системой, а действующие лица – это заинтересованные лица по отношению к создаваемой системе. Такие диаграммы показывают, какие действующие лица инициируют прецеденты.

Сценарии прецедента выделяют в отдельные прецеденты, связанные отношением включает (include), при выполнении следующих условий:

- эти сценарии дублируются в других прецедентах.
- прецедент является очень сложным и длинным, поэтому выделение сценариев в отдельный прецедент позволит значительно его упростить.

Взаимосвязь расширяет (extend) предоставляет дополнительную возможность в том случае, когда основной прецедент нельзя модифицировать. Основная идея заключается в создании расширяющего или дополнительного прецедента, в котором описывается, где и при каких условиях он расширяет некоторый основной прецедент.

Рассмотрим диаграмму прецедентов “Интернет-магазин”, которая приведена на рис. 9.

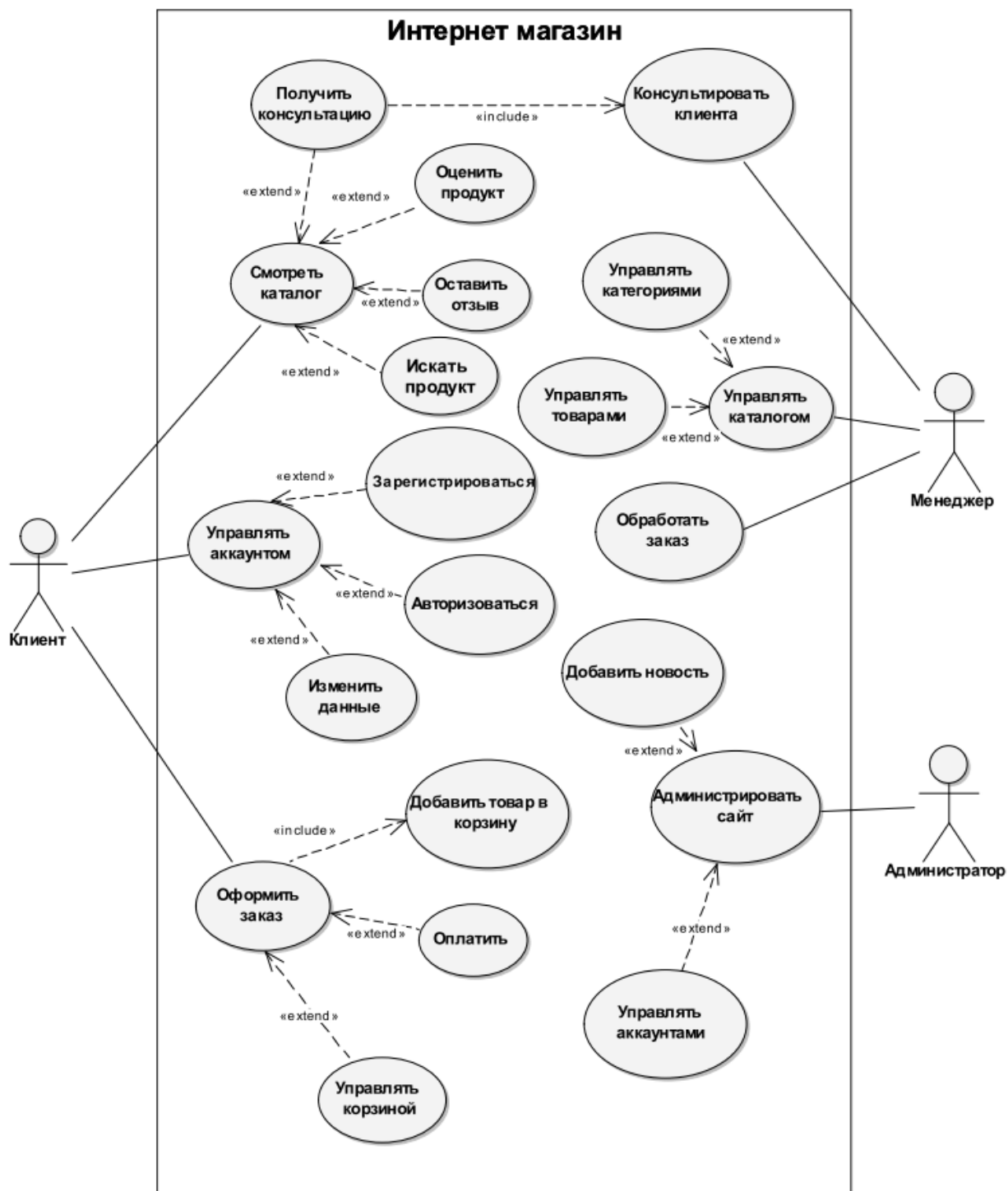


Рис. 9. Диаграмма прецедентов «Интернет-магазин»

Она содержит действующие лица: клиент, менеджер и администратор. Все перечисленные актеры будут взаимодействовать непосредственно с информационной системой. Рамкой показаны границы системы.

Основные прецеденты включают в себя:

1. Управление аккаунтом. Он расширяется прецедентами «Авторизация» – процесс прохождения авторизации на сайте, «Регистрация» – регистрация клиента на сайте, «Управление аккаунтом» – редактирование профиля пользователя, это может быть изменение личных данных, пароля доступа, адреса доставки и другое.
2. Просмотр каталога – просмотр клиентом магазина товаров доступных в продаже. Прецедент связан отношением расширения с прецедентами “Получить консультацию” – связаться с менеджером и узнать дополнительную информацию о продукте, “Оценить продукт” – возможность поставить оценку просматриваемому продукту, “Оставить отзыв” – возможность оставить комментарий к продукту и “Искать продукт” – возможность поиска продукта по ассортименту магазина.
3. Оформление заказа. Этот прецедент включает в себя прецеденты “Добавление товара в корзину” (тип связи “include”), “Оплатить” – непосредственно проведение транзакции через платежную систему и “Управлять корзиной” – возможность просмотра товаров, находящихся в корзине и управление ими (изменение количества, удаление конкретного товара из корзины).
4. Консультировать клиента. Клиент может получать консультации как на веб-сайте, так и с помощью систем обратной связи, например, форма обратной связи, консультации через системы мгновенного обмена сообщениями и т.д. Консультацию осуществляет менеджер.
5. Управление каталогом. Менеджер может управлять каталогом, добавлять новые категории товаров, новые товары, управлять информацией о товарах, редактировать их описание и другое.
6. Обработать заказ. Менеджер управляет статусом заказа и указывает номер почтового отправления. В случае если клиент хочет что-либо

изменить в заказе, менеджер может это сделать в интерфейсе управления заказами.

7. Администрирование сайта. Веб-сайт нуждается в постоянном администрировании, так как малейший простой в случае сбоя может негативно сказаться на рейтинге магазина. Прецедент включает в себя добавление новостей на сайт, управление аккаунтами пользователей и сотрудников магазина.

Идеальные прецеденты — это развернутые прецеденты, выражающие общую сущность процесса без детализации их реализации. Проектные решения, особенно связанные с интерфейсом пользователя, при этом опускаются. Идеальный прецедент описывает процесс в терминах наиболее существенных видов деятельности и обоснований. Степень абстракции идеальных прецедентов может варьироваться, т.е. прецедент может быть идеальным в большей или меньшей степени (таблица 1).

Таблица 1. Табличное описание use case

Заголовок	Название варианта использования
Участники варианта использования	Основное действующее лицо (actor) и все дополнительные участники
Предусловие	В каком состоянии должен находиться каждый участник use case, чтобы ситуация произошла Например: заказать еду в приложении, доп. условие пользователь должен быть авторизован
Триггер	Что провоцирует выполнение сценария, например пользователь решил пообедать
Результат или гарантии	Что может быть гарантией успеха
Основной сценарий	Пользователь начал и кончил, потому что он может передумать обедать или курьеры все заняты
Альтернативный сценарий	Строится из основного, пользователь и все члены семьи ушли из дома
Расширения	После заказа такси в радиусе 1 км нет машин

Таблица 2. Шаблон для написания сценария отдельного варианта использования (более короткое)

Главный раздел	Раздел "Типичный ход событий"	Раздел "Исключения"	Раздел "Примечания"
Имя варианта использования	Типичный ход событий, приводящий к успешному выполнению данного варианта использования	Исключение № 1	Примечание № 1
Актеры		Исключение № 2	Примечание № 3
Цель		...	...
Краткое описание Тип		Исключение № n	Примечание №n
Ссылки на другие варианты использования			

Реальный прецедент описывает конкретное проектное решение по реализации идеального прецедента в терминах выбранной технологии. Описание реальных прецедентов аналогично описанию идеальных прецедентов (табл. 3-5).

Таблица 3. Описание прецедента регистрации пользователя

Название прецедента	Зарегистрироваться
Исполнитель	Клиент
Цель	Пройти процедуру регистрации на сайте
Основной успешный сценарий	Клиент переходит на веб-сайт магазина. Кликает кнопку зарегистрироваться. Вводит данные в регистрационную форму и нажимает кнопку отправить данные. На указанный адрес электронной почты поступает письмо содержащее информацию о регистрационных данных
Тип	Идеальный
Ссылки	Добавление нового пользователя, Авторизация, Отправка письма

Таблица 4. Описание прецедента регистрации пользователя

Название прецедента	Управление аккаунтом
Исполнитель	Клиент
Цель	Изменить данные профиля
Основной успешный сценарий	Клиент переходит на веб-сайт магазина,

	регистрируется. Осуществляет процедуру авторизации на сайте. Управляет данными в своем профиле. Система, взаимодействуя с БД, осуществляет изменение данных пользователя
Тип	Идеальный
Ссылки	Отправить на e-mail уведомление о успешном изменении пароля, изменение данных пользователя в БД

Таблица 5. Описание прецедента «Оформление заказа»

Название прецедента	Оформление заказа
Исполнитель	Клиент
Цель	Оформить заказ
Основной успешный сценарий	Выбрать товары, которые нужно приобрести, нажать напротив товаров кнопку “добавить в корзину”. Повторять действия до тех пор, пока не будет укомплектован заказ. Затем нажать кнопку “Оформить заказ”. Пройти процедуру оплаты.
Тип	Идеальный
Ссылки	Обновить корзину, Авторизация платежа, Отправка данных о оплаченном заказе на e-mail клиента и в почтовый отдел

Типичный и альтернативный ход событий

Типичный ход событий – обеспечивает наглядное представление подробного диалога между исполнителями и системой. В этом разделе описывается наиболее стандартная, или типичная, последовательность событий. Альтернативные ситуации в типичную последовательность не включаются.

Как правило, типичный ход событий описывают с использованием таблицы, где в первой колонке приводятся действия внешних исполнителей, а во второй колонке – отклик системы на действия исполнителей.

По мере уточнения требований к системе потоки событий прецедента будет удобнее записать в графической нотации, используя диаграммы взаимодействий. Обычно для главного потока (типичного хода событий) используют диаграмму последовательностей, а для дополнительных (альтернатив) ее варианты. Поскольку прецедент описывает не одну, а множество последовательностей, то выразить все детали исследуемого прецедента с помощью одной последовательности просто невозможно. Один прецедент, обычно, описывает несколько последовательностей, или сценариев, каждый из которых представляет одну из возможных вариаций данного потока событий.

Альтернативный ход событий должен содержать важные альтернативы или исключения, которые могут возникать в ходе типичной последовательности. Если эти исключения слишком сложны, они могут быть развернуты в собственные прецеденты.

В табл. 6 приведено описание типичного хода событий для прецедента «Оформление заказа».

Таблица 6. Описание типичного хода событий прецедента «Оформление заказа»

Действия исполнителя	Отклик системы
1. Клиент чтобы авторизоваться на веб-сайте вводит свой логин и пароль	2. Система авторизует пользователя
3. Клиент открывает каталог со списком товаров	4. Система возвращает список товаров для открытого каталога
5. Клиент переходит на страницу конкретного товара	6. Система возвращает клиенту страницу с запрашиваемым продуктом

7. Клиент кликает на странице товара, который хочет купить, кнопку “Добавить в корзину”	8. Система добавляет товар в виртуальную корзину и отображает ее содержимое пользователю
9. После того как клиент сформировал корзину, он нажимает кнопку “Оформить заказ”	10. Система перенаправляет клиента на страницу оформления заказа
11. Клиент вводит данные о заказе (адрес доставки, на чье имя и т.д.)	12. Система возвращает пользователю страницу с предварительным просмотром заказа
13. Пользователь нажимает кнопку “Заплатить”	14. Система перенаправляет пользователя на сайт платежной системы
15. Клиент совершает платежную транзакцию на сайте платежной системы	16. Платежная система после завершения оплаты перенаправляет пользователя обратно на сайт магазина, где отображается информация о том, что заказ принят к исполнению

В таблице 7 приведено описание альтернативного хода событий для прецедента “Оформление заказа”.

Таблица 7. Описание альтернативного хода событий прецедента «Оформление заказа»

Альтернатива	Отклик системы
Клиент открывает страницу товара, которого нет в наличии	Система деактивирует кнопку “Добавить в корзину” и выводит альтернативные продукты ниже
Клиент откладывает оплату товара либо платежная транзакция завершается неудачно	Система сохраняет данные заказа на 1 неделю, чтобы пользователь мог оплатить его позже
Клиент в предварительном просмотре ввел не корректные данные и нажимает в браузере кнопку назад	Система возвращает клиенту страницу оформления заказа с ранее заполненными данными

Клиент ввел в корзине не корректное количество товара	Система отображает оповещение о недопустимом значении
---	---

Диаграмма use case - это взаимодействие системы с пользователем, это ответ на вопрос что делает система, а что пользователь, а не как.

Диаграмма позволяет

- составить общее представление;
- показать работодателям, что система должна сделать;
- сэкономить время для тестирования;
- показать порядок действий.

Для удобства записи и чтения сценария часто используют модификацию данного шаблона, располагая разделы последовательно сверху вниз. При этом написание сценариев модели начинают с базовых или основных вариантов использования, после чего рассматриваются сценарии второстепенных или включаемых вариантов использования. Сценарии расширяющих вариантов использования обычно помещаются в разделе исключений базового варианта использования. Желательно, чтобы каждому исключению соответствовал отдельный вариант использования, который соединяется с базовым отношением расширения. В противном случае будет нарушено логическое соответствие между текстами сценариев и диаграммой вариантов использования. Если поведение некоторого варианта использования представляется разработчику тривиальным, то и написание для него сценария не является необходимым. В этом случае можно ограничиться простым примечанием к соответствующему исключению. При написании сценариев вариантов использования важно понимать, что сценарий должен дополнять или уточнять диаграмму вариантов использования, но не заменять ее полностью.

В качестве примера выберем достаточно простой вариант использования "Просмотр списка товаров"(табл.8), для которого соответствующий сценарий удобно представить в виде двух отдельных таблиц, каждая из которых описывает отдельный раздел шаблона. В главном разделе сценария указывается имя рассматриваемого варианта использования, имена взаимосвязанных с ним актеров, цель выполнения, тип и ссылки на другие варианты использования.

Таблица 8. Главный раздел сценария выполнения варианта использования «Просмотр списка товаров»

Вариант использования	Просмотр списка товаров
Актеры	Посетитель интернет-магазина
Краткое описание □	Получение требуемой информации о товарах, представленных в интернет-магазине
Цель	Посетитель интернет-магазина просматривает информацию о товарах. Система обеспечивает доступ к любому товару и удобную навигацию по различным категориям товаров
Тип	Базовый
Ссылки на другие варианты использования	Отсутствуют

Поскольку для варианта использования "Просмотр списка товаров" отсутствуют включаемые варианты использования, ссылки на другие варианты использования также будут отсутствовать. В следующем разделе сценария описывается последовательность действий, приводящая к успешному выполнению рассматриваемого варианта использования. В качестве инициатора действий выступает актер "Посетитель интернет-магазина"(табл.9). Для удобства каждое действие помечается порядковым номером в последовательности.

Таблица 9. Раздел типичного хода событий сценария выполнения варианта использования «Просмотр списка товаров»

Действия актеров	Отклик системы
1. Посетитель загружает исходную страницу интернет-магазина в браузер	2. Система отображает исходную страницу интернет-магазина
3. Посетитель интернет-магазина выбирает категорию интересующих товаров	4. Система отображает информацию о выбранной категории товаров

5. Посетитель интернет-магазина выбирает интересующий товар	6. Система отображает общую информацию о выбранном товаре
7. Посетитель интернет-магазина выбирает просмотр детальной информации об интересующем товаре	8. Система отображает детальную информацию о выбранном товаре
9. Посетитель интернет-магазина может пожелать вернуться на исходную страницу интернет-магазина	10. Система отображает исходную страницу интернет-магазина

Поскольку для рассматриваемого варианта использования отсутствуют расширяющие варианты использования, секции исключений останутся пустыми. Заинтересованным читателям в качестве упражнения для самостоятельной работы предлагается написать аналогичные текстовые сценарии для остальных вариантов использования, представленных на ранее построенной диаграмме.

2.1.4. Основные рекомендации при описании use case

Частые ошибки:

1. В каждом пункте клиент, нет взаимодействия с системой.
2. Отсутствует действующее лицо
3. Много окон интерфейса, не надо описывать конечный сценарий системы, есть технические диаграммы более подробные.
4. Низкие уровни цели, слишком подробная детализация.

Ошибки не такие важные

1. Описание очень подробное.
2. Не указывается альтернативный сценарий (что приводит к успеху другим путем).
3. Расширение- это то, что может пойти не так.
4. Предусловия и триггеры не указывают.
5. Относятся к системе как к «черному ящику», есть технический сценарий более подробный.
6. Диаграмма без шагов, поэтому нужна расшифровка.

2.2. Диаграмма классов (class diagram)

Диаграмма классов (class diagram) — диаграмма, предназначенная для представления модели статической структуры программной системы в терминологии классов объектно-ориентированного программирования. Разработка диаграммы классов преследует следующие цели: определить сущности предметной области и представить их в форме классов с соответствующими атрибутами и операциями; определить взаимосвязи между сущностями предметной области и представить их в форме типовых отношений между классами; разработать исходную логическую модель программной системы для ее последующей реализации в форме физических моделей; подготовить документацию для последующей разработки программного кода. В общем случае диаграмма классов представляет собой граф, вершинами или узлами которого являются элементы типа "классификатор", связанные различными типами структурных отношений. Классификатор (classifier) — специальное понятие, предназначенное для классификации экземпляров, которые имеют общие характеристики.

Диаграмма классов позволяет создать логическую модель системы, на основе которой создается исходный код описанных классов. Диаграмма классов воссоздает внутреннюю структуру системы, описывает наследование и взаимное положение классов относительно друг друга. Значки диаграммы позволяют отображать сложную иерархию систем, взаимосвязи классов и интерфейсов.

2.2.1 Графический элемент класса

Класс (class) — элемент модели, который описывает множество объектов, имеющих одинаковые спецификации характеристик, ограничений и семантики.

Класс в Star UML – это установки структуры и шаблона поведения для некоторого множества реальных объектов, которые в дальнейшем будут определены в программе на основе данного шаблона. Понятие класса в UML аналогично понятию класса в C++. Класс – это некоторая абстракция реального мира. Когда эта абстракция принимает конкретное воплощение, она становится объектом.

Дополнительно класс может быть специфицирован как активный или пассивный. Активный класс (active class) — класс, каждый экземпляр которого имеет свою собственную нить управления. Пассивный класс (passive class) — класс, каждый экземпляр которого выполняется в контексте некоторого другого объекта.

Класс в UML изображается в виде прямоугольника, в верхней части которого стоит имя класса, в середине – атрибуты (список полей), в нижней части – операции (список методов класса) (рис.10).



Рис.10. Варианты графического изображения класса на диаграмме классов: а) в виде прямоугольника с пустыми секциями; б) в виде прямоугольника с заполненными секциями; в) абстрактный класс.



Рис.11. Пример графического представления класса с заполненными свойствами и атрибутами

Имя класса

Имя класса (class name) — строка текста, предназначенная для идентификации класса на диаграмме классов. Оно должно быть уникальным в пределах пакета, который может содержать одну или несколько диаграмм классов. Имя класса записывается в одну строку по центру полужирным шрифтом. Примерами имен классов могут быть существительные предметной области, такие как: **Компания, Сотрудник, Проект, Заказ**, имеющие непосредственное отношение к моделируемой предметной области. Примерами имен классов также могут быть имена классов используемых библиотек, например, **CObject, CRecordView, TObject, TMenu**, соответствующих функциональному назначению отдельных элементов проектируемой системы.

Класс может не иметь экземпляров или объектов. В этом случае он называется *абстрактным (abstract)* классом, а для обозначения его имени используется наклонный шрифт (курсив).

Атрибуты класса

Атрибут (attribute) класса служит для представления отдельной структурной характеристики или свойства, которое является общим для всех объектов данного класса.

В общем случае атрибут представляет собой структурную характеристику или свойство, относящееся к классификатору или концу ассоциации. Атрибут устанавливает отношение экземпляра класса со значением или коллекций значений типа атрибута. Атрибуты класса записываются во второй сверху секции прямоугольника класса, которую называют секцией атрибутов. При этом все спецификации атрибутов выравниваются слева.

В языке *UML* принята определенная стандартизация записи *атрибутов класса*, которая подчиняется некоторым синтаксическим правилам. Каждому *атрибуту класса* соответствует отдельная строка текста, которая в нотации БНФ имеет следующий формат:

$$\begin{aligned} \langle \text{атрибут} \rangle ::= & [\langle \text{видимость} \rangle] ['/'] \langle \text{имя} \rangle [':' \langle \text{тип атрибута} \rangle] \\ & [[' \langle \text{кратность} \rangle ']] ['=' \langle \text{значение по умолчанию} \rangle] \\ & [[' \langle \text{модификатор атрибута} \rangle [',' \langle \text{модификатор атрибута} \rangle]^* ']], \end{aligned}$$

где:

$\langle \text{видимость} \rangle ::= '+' \mid '-' \mid \# \mid \sim$. (visibility) может принимать одно из 4-х возможных значений и отображаться либо посредством специального символа, либо соответствующего ключевого слова. Является типом перечисления следующих значений литералов:

- ✓ + – public (общедоступный), доступен или виден из любого другого класса пакета, в котором определена диаграмма;
- ✓ # – protected (защищенный), недоступен или не виден для всех классов, за исключением подклассов данного класса;
- ✓ – – private (закрытый), недоступен или не виден для всех классов без исключения;

- ✓ ~ – package (пакетный), недоступен или не виден для всех классов за пределами пакета, в котором определен класс -владелец данного атрибута.

'/' – означает, что атрибут является производным (derive). Значение производного атрибута может быть вычислено на основе значений других атрибутов этого или других классов. Поэтому данный атрибут называют иногда вычислимым.

<имя> (name) – представляет собой строку текста, которая используется в качестве идентификатора соответствующего атрибута и поэтому должна быть уникальной в пределах данного класса. Имя атрибута является единственным обязательным элементом в обозначении атрибута, должно начинаться со строчной (малой) буквы и, как правило, не должно содержать пробелов.

<тип атрибута> (attribute type) – есть имя классификатора, который является типом данного атрибута. Типы атрибутов классов часто соответствуют основным типам данных того языка программирования, который предполагается использовать для реализации данной модели. Типу атрибута должно предшествовать двоеточие.

<кратность> (multiplicity) – атрибута характеризует общее количество конкретных значений для атрибута, которые могут быть заданы для объектов данного класса. Спецификация кратности в нотации БНФ имеет следующий формат:

<кратность> ::= <диапазон-кратности> ['{' <указатель-упорядоченности>

[';' <указатель-уникальности>] '}']

<диапазон-кратности> ::= [<нижняя-граница> '..'] <верхняя-граница>

<нижняя-граница> ::= <целое число> | <спецификация значения>

<верхняя-граница> ::= '*' | <спецификация значения>

<указатель-упорядоченности> ::= 'ordered' | 'unordered'

<указатель-уникальности> ::= 'unique' | 'nonunique'.

<значение по умолчанию> (default) — некоторое выражение, которое служит для задания начального значения или значений данного атрибута в момент создания отдельного экземпляра соответствующего класса.

<модификатор атрибута> (attribute modifier) представляет собой текстовое выражение, которое придает дополнительную семантику данному атрибуту. Может быть представлен в следующем виде (БНФ):

<модификатор атрибута> ::= 'readOnly' | 'union' | 'subsets' <имя атрибута> | 'redefines' <имя атрибута> | 'ordered' | 'unique' |

<ограничение атрибута>

Примеры записи атрибутов для класса Сотрудник:

- + имяСотрудника : String {readOnly} — здесь имяСотрудника является именем общедоступного атрибута, который служит для представления информации только об имени конкретного сотрудника. Тип атрибута String (Строка) указывает на тот факт, что отдельное значение имени сотрудника представляет собой строку текста из одного слова (например, 'Кирилл' или 'Мария'). Модификатор readOnly указывает на тот факт, что после задания этого имени при инстанцировании объекта соответствующего класса оно не может быть изменено;
- ~ дата Рождения: Data {readOnly} — атрибут с пакетной видимостью и типом Data (Дата), который должен быть дополнительно определен в модели. Модификатор readOnly указывает на тот факт, что после задания значения этого атрибута оно не может быть изменено;
- # /возраст Сотрудника: Integer — производный атрибут с защищенной видимостью и простейшим типом Integer (Целое). Значение этого атрибута может быть вычислено как разность между текущей датой и датой рождения соответствующего сотрудника;

- + номер Телефона: Integer [*] {unique} — общедоступный атрибут, используемый в форме коллекции значений, когда у сотрудников рассматриваемой организации может быть несколько телефонных номеров, а может и не быть ни одного. На это указывает кратность атрибута в форме звездочки. В то же время среди номеров телефонов любого сотрудника не должно быть повторяющихся значений. Именно на этот факт указывает модификатор unique;
- – заработная Плата: Currency = \$500 — закрытый атрибут с типом Currency (Денежный), который должен быть дополнительно определен в модели. При создании новых экземпляров класса Сотрудник, чему соответствует, например, прием сотрудника на работу, для него по умолчанию устанавливается заработная плата в \$500. В последующем это значение может быть изменено.

Параметры класса

Параметр (parameter) является спецификацией аргумента, который используется при выполнении операции или при вызове характеристики поведения.

Каждый параметр должен иметь следующий формат записи (БНФ):
<параметр> ::= [<направление>*] *<имя параметра>* ':' *<выражение типа>* [*'<кратность>'*] [*'=' <значение по умолчанию>*] [*'{' <свойство параметра> [',' <свойство параметра>]* '}'*]*

Здесь:

<направление> ::= 'in' | 'out' | 'inout' | 'return'. Другими словами, направление (direction) является одним из ключевых слов: in, out, inout или return. Если оно не указано, то по умолчанию принимается значение in.

<имя параметра> (parameter name) представляет собой идентификатор формального параметра, при записи которого необходимо следовать правилам задания имен атрибутов;

<выражение типа> (type expression) является спецификацией типа данных для возможных значений соответствующего формального параметра;

<кратность> (multiplicity) характеризует общее количество конкретных параметров с данным именем, которые могут принадлежать тому или иному объекту данного класса;

<значение по умолчанию> (default) представляет собой некоторое выражение, которое специфицирует конкретное значение по умолчанию для данного формального параметра;

<свойство параметра> (parameter property) указывает дополнительные свойства значений данного формального параметра. В качестве значений свойств параметра могут быть использованы модификаторы атрибутов.

Примеры записи операций для класса Сотрудник:

- +добавить(in номерТелефона : Integer [*] {unique}) — общедоступная операция, которая использует многозначный целочисленный параметр номерТелефона в качестве передаваемых операций значений. Среди них не должно быть повторяющихся значений, на что указывает модификатор свойство параметра unique;
- –изменить(in заработнаяПлата : Currency) — закрытая операция с единственным передаваемым параметром заработнаяПлата типа Currency;
- +создать() : Boolean — общедоступная операция по созданию отдельного объекта класса Сотрудник, которая не содержит параметров. Эта операция может быть вызвана объектами любого класса в пространстве имен модели. После вызова данной операции она должна вернуть логическое значение.

Еще примеры графического представления классов (рис. 11а).

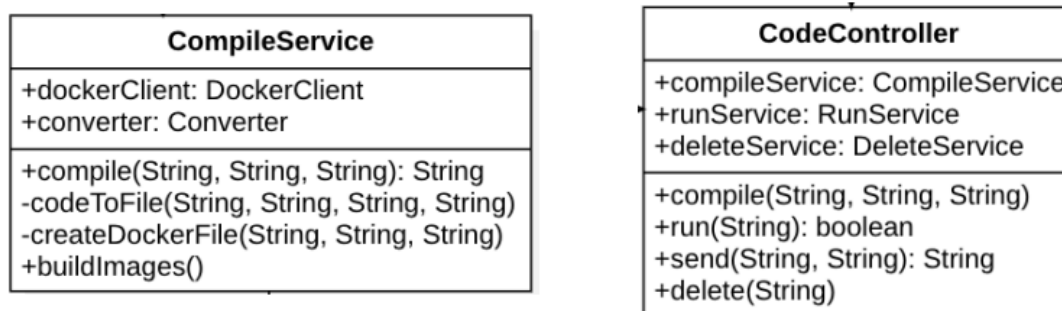


Рис.11 а. Пример графического представление класса с заполненными полями атрибутов и операций

2.2.2. Отношения на диаграмме классов

Кроме внутреннего устройства классов важную роль при разработке моделей программных систем имеют различные отношения между классами, которые также могут быть изображены на диаграмме классов. Каждое из этих отношений имеет собственное графическое представление и семантику, которая отражает характер и особенности соответствующей взаимосвязи между классами. Всего существует шесть [2–5] основных видов отношений между классами: ассоциации и обобщения, агрегации и композиции, зависимости и реализации.

Для покрытия этих основных отношений большинство объектно ориентированных языков поддерживают следующие отношения:

- ассоциации;
- наследования;
- агрегации;
- композиции;
- зависимости;
- реализации.

Ассоциация (association) — произвольное отношение или взаимосвязь между классами обозначает семантическое соединение классов, но не

указывает направления. Она задает роли участников семантической связи, мощность (количество элементов) участников связи, ассоциация бывает «один-к-одному», «один-ко-многим», «многие-ко-многим». В виде направленной ассоциации, когда один класс (*первичный*), использует другой класс (*вторичный*), в качестве своего поля или метода. Отношение ассоциации используют, чтобы показать, что между классами (например, между двумя классами) существует некоторая связь. Обычно с помощью него на диаграмме классов показывают, что один класс пользуется функционалом другого класса (рис.12).

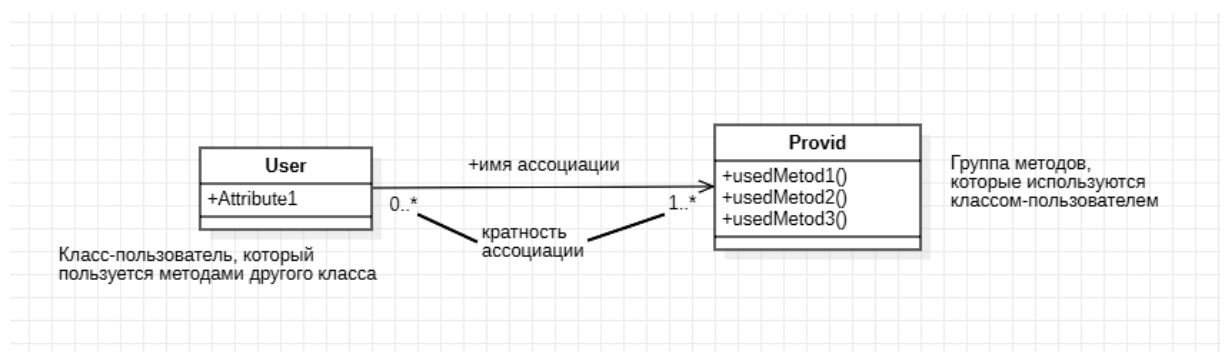


Рис. 12. Основные элементы ассоциации.

Частным случаем отношения ассоциации является так называемая исключающая ассоциация (Xor-association). Семантика данной ассоциации указывает на тот факт, что из нескольких потенциально возможных вариантов данной ассоциации в каждый момент времени может использоваться только один ее экземпляр. На диаграмме классов исключающая ассоциация изображается с помощью дополнительной пунктирной линии, соединяющей две и более ассоциации, рядом с которой записывается ограничение в форме специального ключевого слова {xor}.

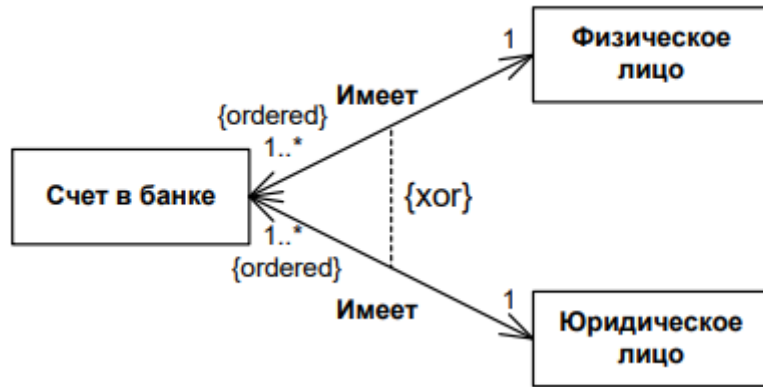


Рис.13. Графическое изображение исключяющей ассоциации между тремя классами.

Более общий случай представляет собой n-арная ассоциация (n-ary association, читается — "эн арная ассоциация"), которая связывает некоторым отношением ассоциации более двух классов. В качестве значения N может выступать произвольное натуральное число больше 1. Каждый экземпляр реализации n-арной ассоциации представляет собой n-арный кортеж, состоящий в точности из n объектов соответствующих классов.

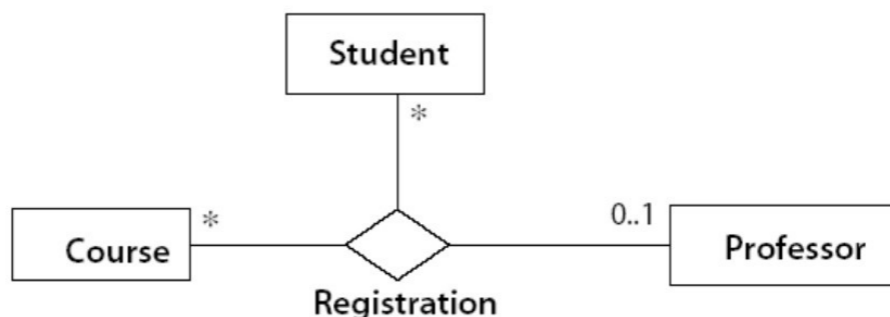


Рис. 14. Пример тернарной ассоциации.

Ассоциация является наиболее общей формой отношения в языке UML 2.0. Практически все другие типы отношений в языке UML 2.0 являются специальными случаями данного отношения.

Обобщение (generalization) — таксономическое отношение между более общим классификатором (родителем или предком) и более специальным классификатором (дочерним или потомком). В этом случае говорят,

что специальный классификатор наследует характеристики более общего классификатора.

Согласно одному из главных принципов методологии ООАП — наследованию, класс-потомок обладает всеми свойствами и поведением класса-предка, а также может иметь дополнительные свойства и поведение, которые отсутствуют у класса-предка. Применительно к диаграмме классов обобщение обеспечивает наследование атрибутов и операций классов. На рис. 15 представлен пример отношения обобщения на диаграмме классов.

Отношение обобщения допускает, чтобы от одного класса-предка одновременно наследовали несколько классов-потомков.

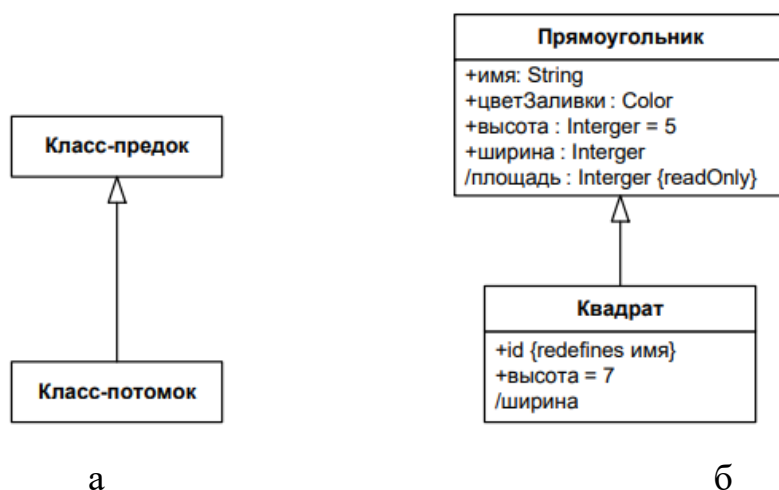


Рис.15. Графическое изображение отношения обобщения в языке UML 2.0:
а — свернутое представление классов; б — развернутое представление классов

Агрегация (aggregation) — направленное отношение между двумя классами, предназначенное для представления ситуации, когда один из классов представляет собой некоторую сущность, которая включает в себя в качестве составных частей другие сущности.

Агрегация имеет фундаментальное значение для описания структуры сложных систем, поскольку применяется для представления системных взаимосвязей типа "часть-целое". Раскрывая внутреннюю структуру систе-

мы, агрегация показывает, из каких элементов состоит система, и как они связаны между собой. С точки зрения модели отдельные части системы могут выступать как в виде элементов, так и в виде подсистем, которые, в свою очередь, тоже могут состоять из подсистем или элементов (рис.16).

Агрегация является частным случаем отношения ассоциации, от которой она наследует такие свойства, как имена концов ассоциации, кратность и ограничения. Однако агрегациями могут быть только бинарные ассоциации.



Рис. 16. Графическое изображение отношения агрегации на диаграмме классов в языке UML 2.0.

Композиция (composition) или композитная агрегация предназначена для спецификации более сильной формы отношения "часть-целое", при которой с уничтожением объекта класса-контейнера уничтожаются и все объекты, являющиеся его составными частями (рис.17).

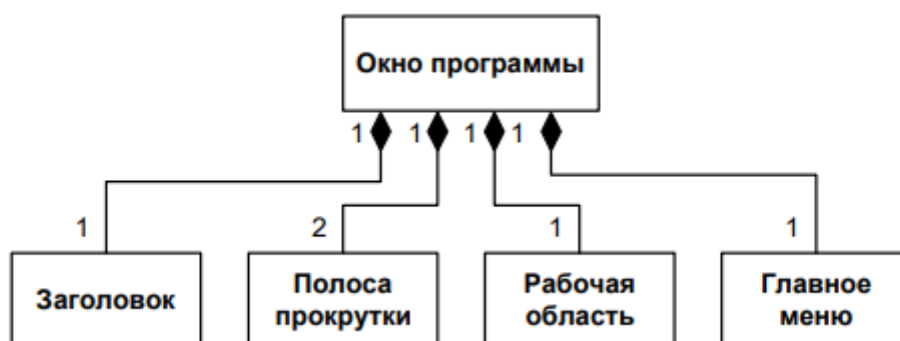


Рис. 17. Графическое изображение отношения композиции на диаграмме классов в языке UML 2.0.

Зависимость (dependency) — отношение, предназначенное для описания ситуации, когда отдельному элементу или множеству элементов модели требуются другие элементы модели для своей спецификации или реализации.

Зависимость – это отношение, которое показывает, что изменение в одном классе (независимом) может повлиять на другой класс (зависимый), который использует его. Графически зависимость изображается пунктирной стрелкой, направленной на класс, от которого зависят. С помощью зависимости определяют клиента и поставщика определенной услуги (рис.18).

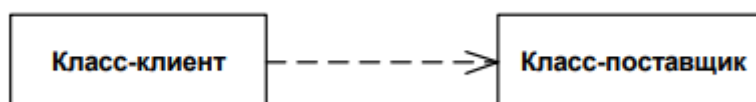


Рис.18. Графическое изображение отношения зависимости на диаграмме классов

Реализация (realization) — специализированное отношение зависимости между двумя элементами модели, один из которых представляет некоторую спецификацию (поставщик), а другой представляет его реализацию (клиент).

Реализация – данный тип связи позволяет показать, что один класс является реализацией другого, то есть создан на основе шаблона другого класса (рис.19).

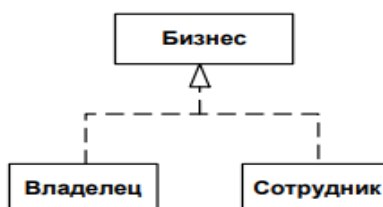


Рис.19. Графическое изображение отношения реализации на диаграмме классов.

Интерфейс (interface) — вид класса, который представляет собой объявление множества общедоступных характеристик и обязанностей.

Интерфейс в языке UML 2.0 является специальным случаем класса, у которого, как правило, имеются операции и отсутствуют атрибуты. Интерфейсы предназначены для спецификации таких операций класса, объявления которых видимы извне, однако особенности их реализации скрыты от клиентов.

Множество интерфейсов, реализуемых классификатором, являются его предоставляемыми интерфейсами (provided interface), описывающие обязанности или сервисы, которые экземпляры этого классификатора имеют перед своими клиентами (рис.20 а). Соответствующее отношение между классом и предоставляемым им интерфейсом представляется отношением реализации интерфейса от класса к интерфейсу (рис.20 б).

Интерфейсы могут быть также использованы в качестве требуемых интерфейсов (required interface), которые специфицируют сервисы, необходимые классу для того, чтобы выполнять собственные обязанности для своих клиентов (рис. 20 б). Соответствующее отношение между классом и требуемым интерфейсом представляется отношением зависимости использования от класса к интерфейсу (рис.20 б).

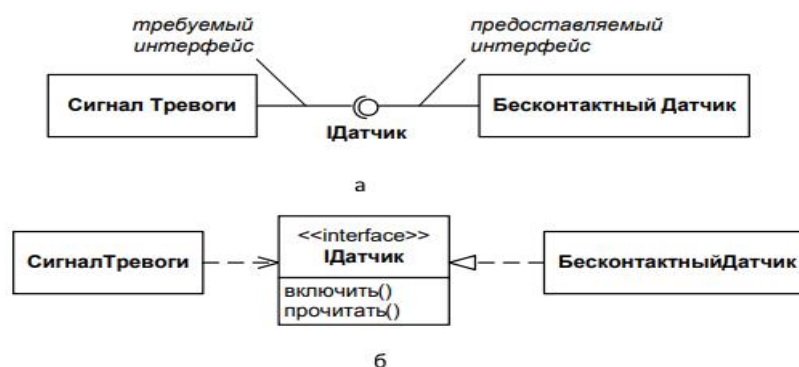


Рис. 20. а – Пример графического изображения, требуемого (слева) и предоставляемого (справа) интерфейса; б – пример альтернативной нотации для графического изображения интерфейса.

Шаблон (template) — классификатор, который в своем описании имеет несколько формальных параметров.

Шаблон является параметризуемым элементом, который может быть использован для получения других элементов модели с помощью отношения связывания шаблона (рис. 21).

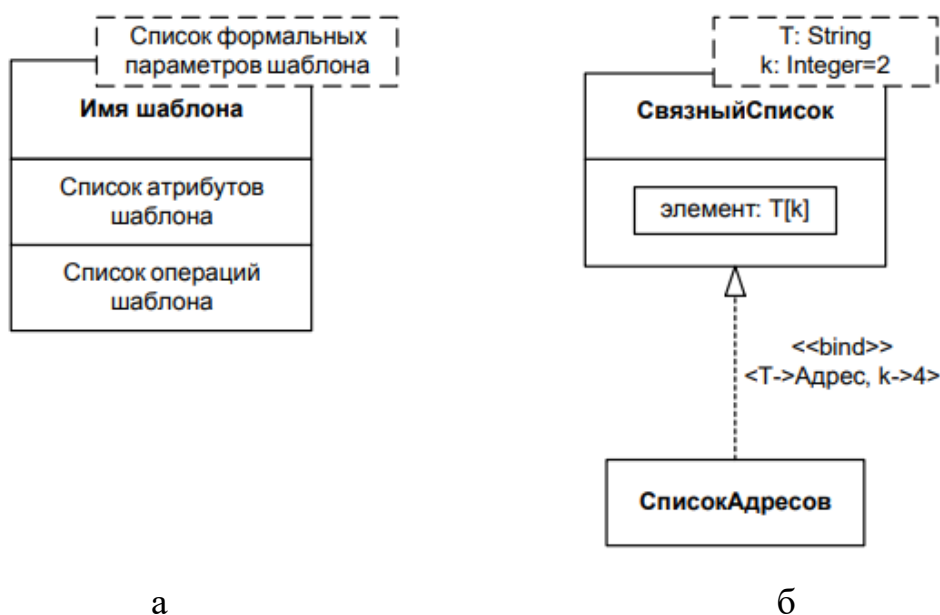


Рис. 21. Графическое изображение шаблона на диаграмме классов: а — общее представление шаблона; б — связывание шаблона

Основные обозначения на диаграмме классов приведены на рис. 22.

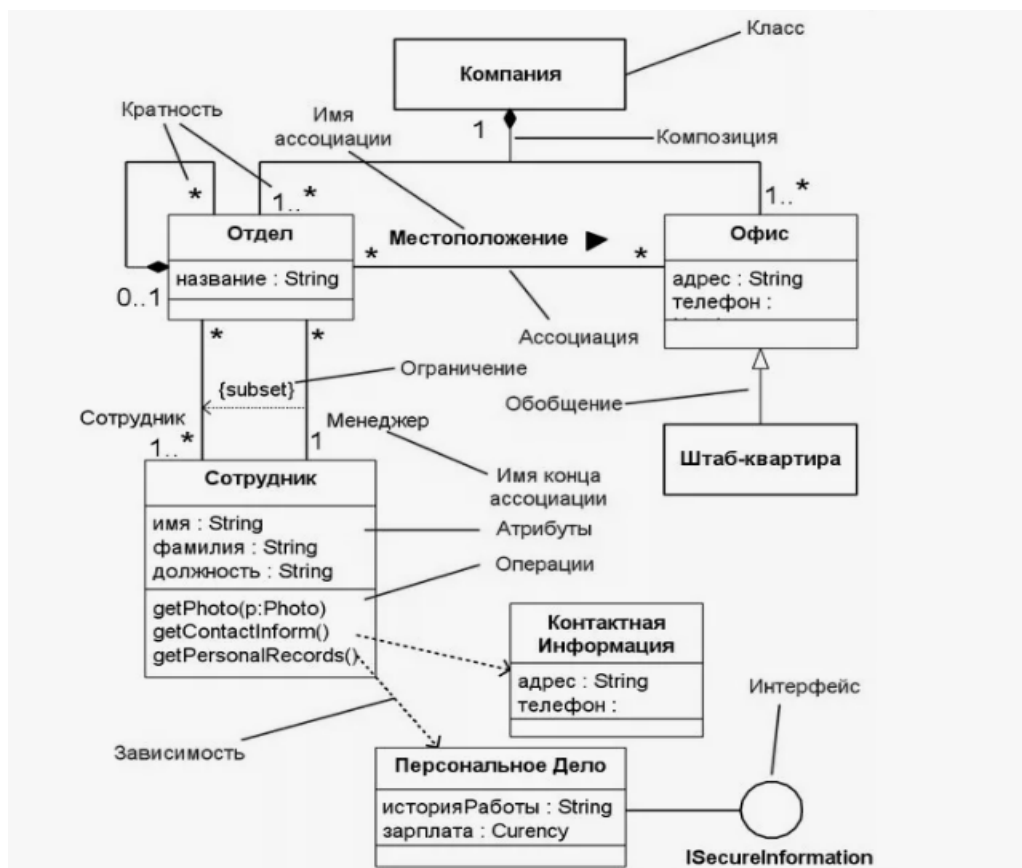


Рис. 22. Основные обозначения на диаграмме классов.

Также ниже приведены примеры диаграммы классов задачи оформление платежа (рис.23) и задачи обучения в вузе (рис.24)

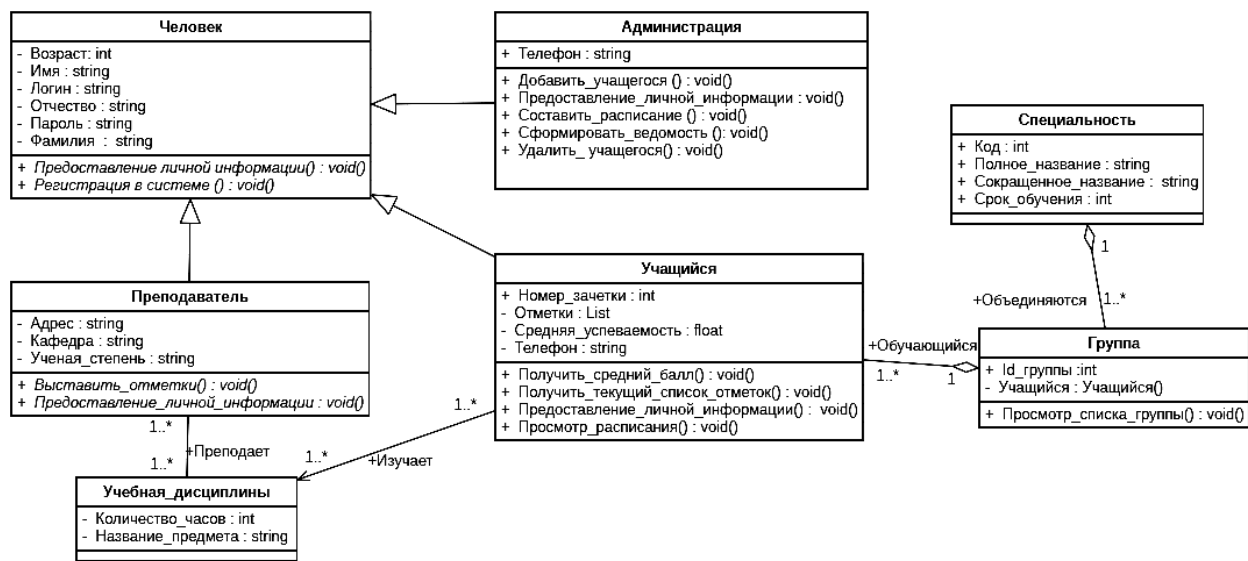


Рис. 24. Диаграмма классов «Составление списков предметов и групп учащихся» для ИС «Обучение в ВУЗе».

2.2.3 Пример

Задача. Создать диаграмму классов информационной системы «Обслуживание склада»

Объектно ориентированная система должны содержать следующие данные:

- информацию об «единицах хранения» – номер ордера, дата, код поставщика, балансный счет, код сопроводительного документа по справочнику документов, номер сопроводительного документа, код материала по справочнику материалов, счет материала, код единицы измерения, количество пришедшего материала, цена единицы измерения);
- информацию о хранящихся на складе материалах (справочник материалов – код класса материала, код группы материала, наименование материала);
- информацию о единицах измерения конкретных видов материалов – код материала, единица измерения (метры, килограммы, литры и т. д.);
- информацию о поставщиках материалов – код поставщика, его наименование, ИНН, юридический адрес (индекс, город, улица, дом), адрес банка (индекс, город, улица, дом), номер банковского счета.

Class Diagram (Диаграмма классов)

В данной задаче можно выделить 4 класса (рис.25):

1. Единица измерения.
2. Материал.
3. Поставщик.
4. Единица хранения (заказ).

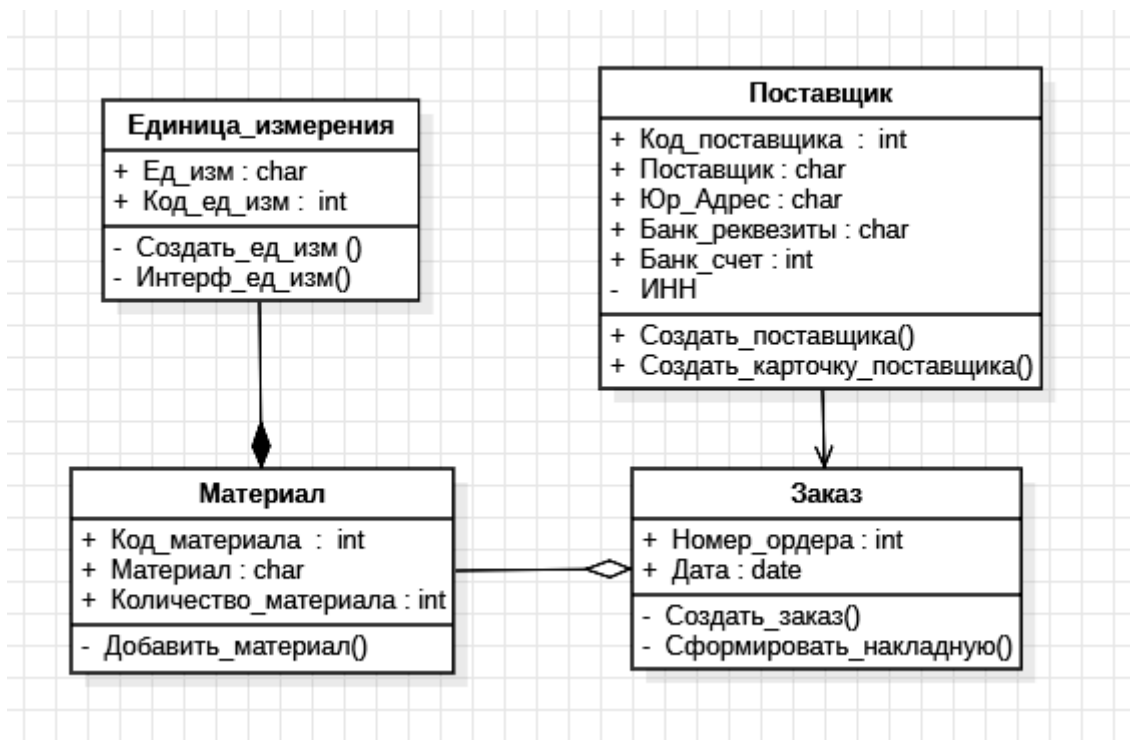


Рис.25. Диаграмма классов для задачи обслуживания склада

2.3. Sequence diagram (диаграммы последовательностей действий)

Диаграмма последовательности (sequence diagram) — диаграмма, предназначенная для представления взаимодействия между элементами модели программной системы в терминологии линий жизни и сообщений между ними.

Графически диаграмма последовательности имеет два измерения. Одно — слева направо в виде вертикальных линий, каждая из которых соответствует линии жизни отдельного участника взаимодействия. Второе — вертикальная временная ось, направленная сверху вниз. Начальному моменту времени соответствует самая верхняя часть диаграммы. Реализация взаимодействия моделируется посредством сообщений, которые передаются между различными линиями жизни. Сообщения изображаются в виде стрелок различной формы и образуют некоторый порядок относительно времени своей передачи. При этом сообщения, расположенные на диа-

грамме последовательности выше, передаются раньше тех, которые расположены ниже.

Диаграмма последовательности (sequence diagram) — диаграмма, предназначенная для представления взаимодействия между элементами модели программной системы в терминологии линий жизни и сообщений между ними.

Взаимодействие объектов в системе происходит посредством приема и передачи сообщений объектами – клиентами и обработки этих сообщений объектами – серверами. Эта диаграмма позволяет отразить последовательность передачи сообщений между объектами.

Линия жизни (lifeline) представляет одного индивидуального участника взаимодействия или отдельную взаимодействующую сущность. Отдельные линии жизни, выполнив свою роль в системе, могут быть уничтожены или разрушены, чтобы освободить занимаемые ими ресурсы. Для обозначения момента уничтожения линии жизни в языке UML используется специальный символ в форме латинской буквы "X" (рис.26).

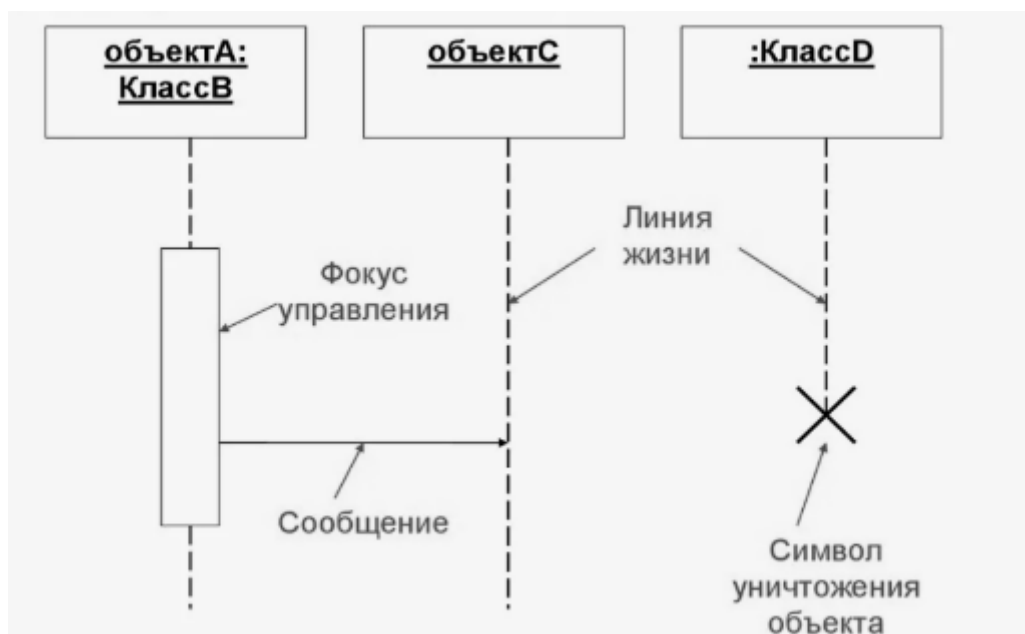


Рис.26. Графическое изображение линий жизни на диаграмме последовательности

Сорт сообщения является перечислением следующих значений:

- 'synchCall' — синхронное сообщение, которое соответствует синхронному вызову операции. Синхронные сообщения обычно представляют вызовы методов и изображаются сплошной линией с закрашенной стрелкой;
- 'asynchCall' — асинхронное сообщение, которое соответствует асинхронному вызову операции. Асинхронные сообщения изображаются сплошной линией с открытой стрелкой в форме буквы "V" ;
- 'asynchSignal' — асинхронный сигнал, который соответствует некоторому асинхронному действию. Асинхронные сигналы также изображаются сплошной линией с открытой стрелкой в форме буквы "V".

Сообщение также может быть ответным (reply) от вызова метода. Такое сообщение изображается пунктирной линией с открытой стрелкой в форме буквы "V" . Сообщение создания объекта (object creation) также изображается пунктирной линией с открытой стрелкой в форме буквы "V".

Сообщение (message) — элемент модели, предназначенный для представления отдельной коммуникации между линиями жизни некоторого взаимодействия.

Вид сообщения является перечислением следующих значений: 'complete' — полное сообщение, для которого существует событие передачи и событие приема.

- 'lost' — потерянное сообщение, для которого существует событие передачи и отсутствует событие приема;
- 'found' — найденное сообщение, для которого существует событие приема и отсутствует событие передачи;

- 'unknown' — неизвестное сообщение, для которого отсутствуют событие передачи и событие приема.

Обычно сообщения на диаграмме последовательности изображаются горизонтальными линиями между двумя линиями жизни. При этом неявно предполагается, что время передачи сообщения достаточно мало по сравнению с процессами выполнения действий линиями жизни. Если же это предположение не выполняется, то стрелка сообщения изображается под некоторым наклоном, так чтобы конец стрелки располагался ниже ее начала.

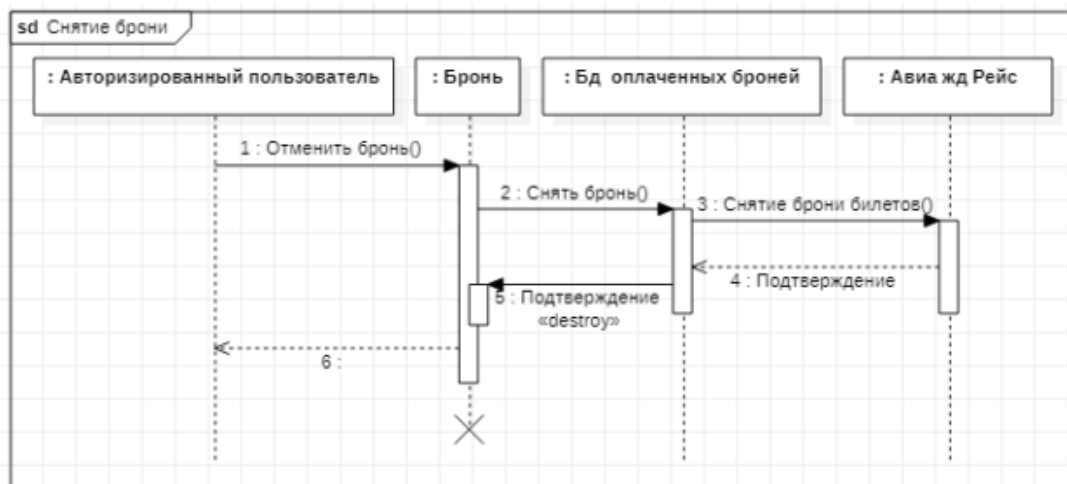


Рис.27. Пример диаграммы последовательности «Снятие брони».

Комбинированный фрагмент — это фрагмент взаимодействия, который определяет комбинацию (выражение) фрагментов взаимодействия. Комбинированный фрагмент определяется оператором взаимодействия и соответствующими операндами взаимодействия. С помощью комбинированных фрагментов пользователь сможет компактно и лаконично описать несколько трасс.

Комбинированный фрагмент может иметь ограничения взаимодействия, также называемые защитными механизмами в UML 2.4.

Оператор взаимодействия может быть одним из:

- alt - альтернативные варианты;

- opt – опция;
- loop – итерация;
- break – перерыв;
- par – параллельный;
- strict - строгая последовательность;
- seq - слабое секвенирование;
- critical – критический регион;
- ignor – игнорировать;
- consider - рассмотреть
- assert – утверждение;
- neg – отрицательный;

Примеры использования комбинированных фрагментов представлены на рис. 28-30.

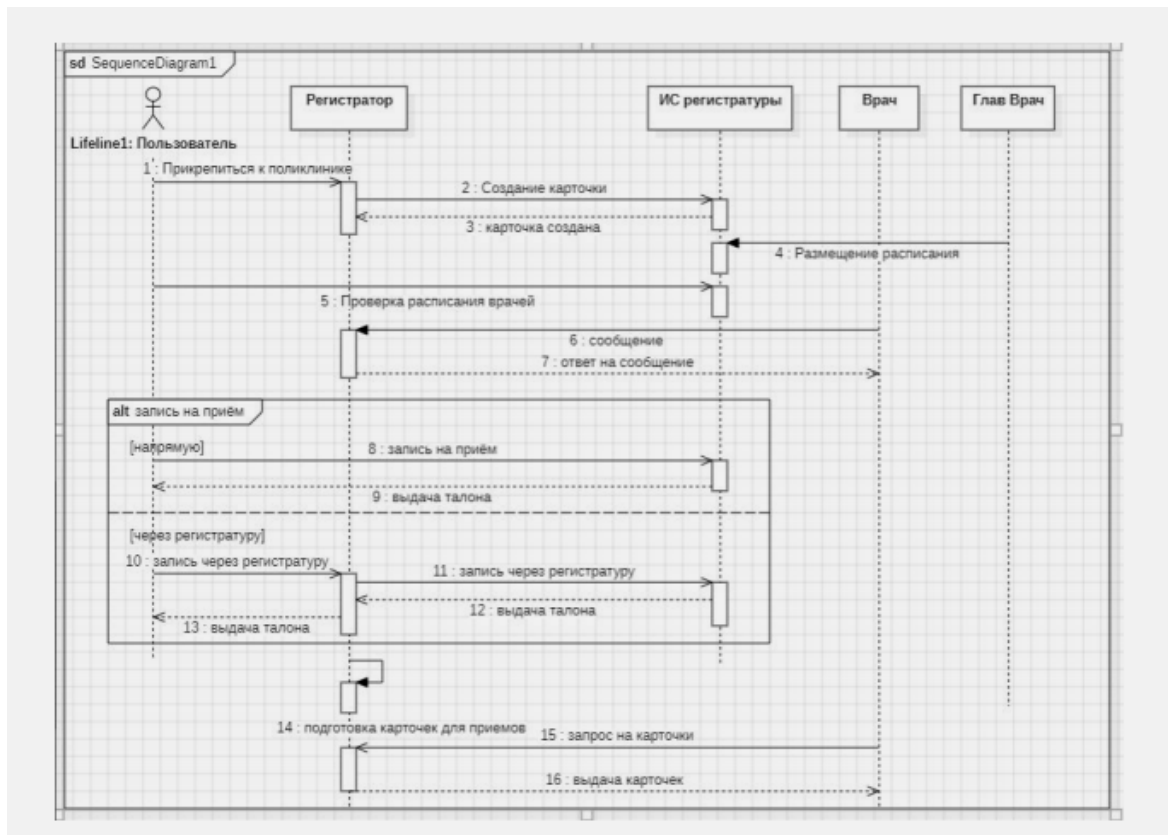


Рис. 28. Диаграмма последовательности ИС «Регистратура поликлиники», пример комбинированного фрагмента Альтернатива

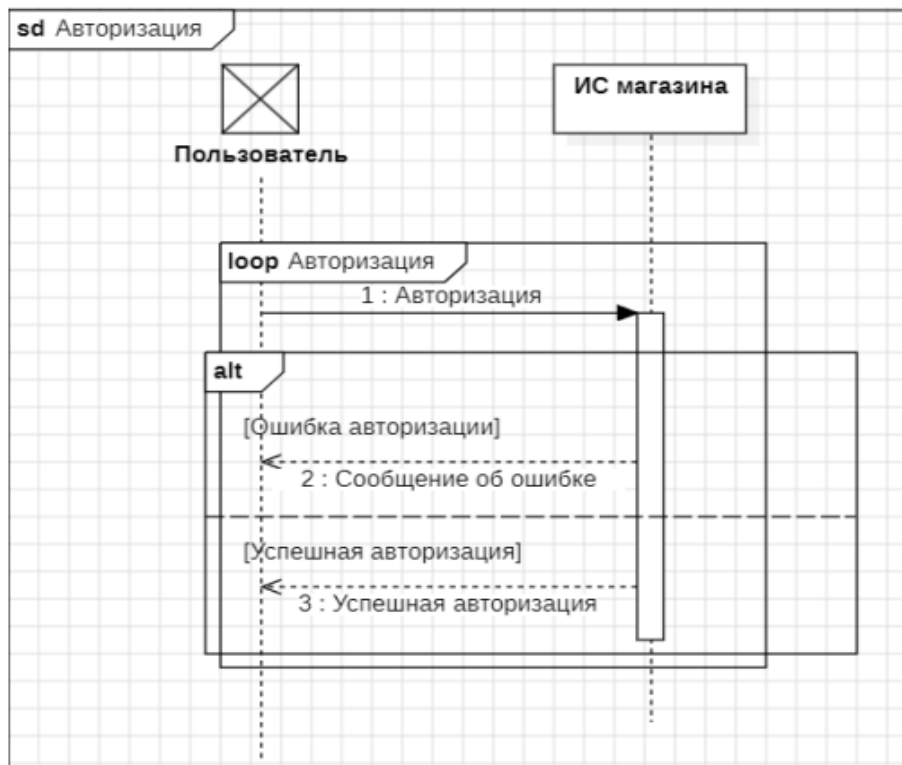


Рис. 29. Диаграмма последовательности «Авторизация». Пример использования комбинированного фрагмента Альтернатива и Цикл

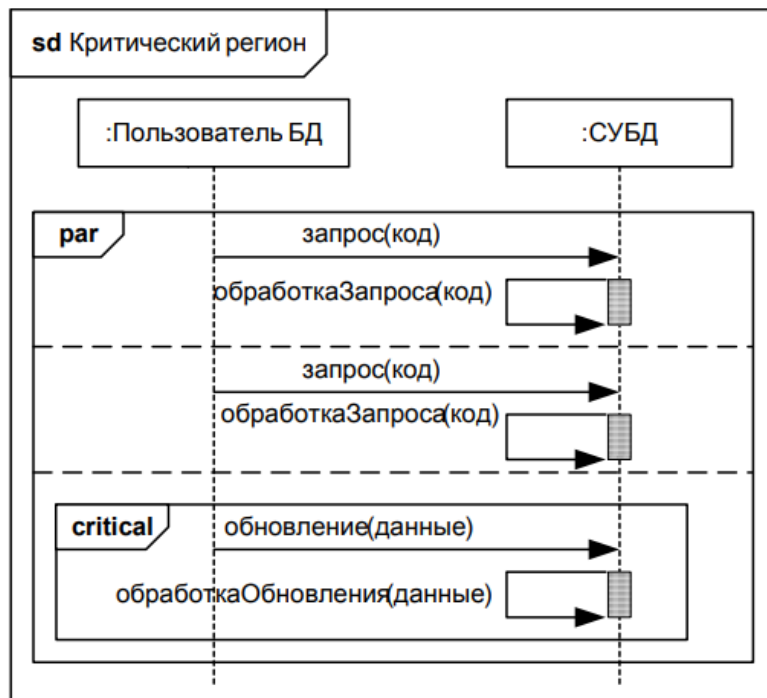


Рис. 30. Пример комбинированного фрагмента Критический регион

2.3.1. Пример

Задача. Создать диаграмму последовательности информационной системы «Описание бизнес-процесса»

Постановка задачи:

Предположим, пользователь – посетитель сайта хочет купить курс по бизнес-анализу и оплатить его онлайн банковской картой. При этом он может получить скидку, применив промокод. Промокод действует только на определенный курс и ограничен по времени. Данные о промокоде хранятся в СУБД Учебного Центра (УЦ): сам промокод, курс, на который он распространяется, размер скидки, срок действия (дата валидности) и состояние (новый, выдан, использован, истек). Сам процесс оплаты, т.е. непосредственного списания денег с банковской карты выполняется на стороне банковского платежного шлюза. Выполняемую при этом последовательность шагов можно упрощенно представить следующим образом:

1. Пользователь выбирает курс по бизнес-анализу и нажимает кнопку «Купить».
2. Открывается страница оплаты курса на сайте учебного центра с формой договора покупки курса и чек-боксом о наличии промокода.
3. Пользователь вводит свои данные в форму договора покупки курса.
4. Пользователь отмечает наличие промокода в чек-боксе.
5. Открывается поле ввода промокода и кнопка проверки его валидности.
6. Пользователь вводит промкод.
7. Пользователь проверяет его валидность, кликнув на кнопку «Проверить».
8. Выполняется проверка валидности промокода с учетом данных по нему, которые имеются в СУБД УЦ.

9. Если промокод корректен (т.е. привязан к выбранному курсу и дата действия его еще не истекла) и находится в состоянии «выдан», цена курса меняется с учетом скидки по промокоду. Иначе цена курса остается прежней и пользователю показывается сообщение о невозможности применить этот код по одной из следующих причин: уже использован, не применим к выбранному курсу или закончился срок его действия.
10. Соглашаясь с ценой (пересчитанной или прежней), пользователь нажимает кнопку «Оплатить». При этом в сторону платежного шлюза посылается запрос со всеми параметрами заказа, где сумма списания равна итоговой цене, рассчитанной с учетом скидки по промокоду, если ее удалось применить.
11. Открывается веб-страница платежного шлюза с формой ввода данных банковской карты пользователя и суммой списания;
12. Пользователь вводит реквизиты своей банковской карты и нажимает кнопку «Оплатить». При этом на сервер банка в систему «Антифрод» отправляются детали заказа и данные карты, чтобы проверить заказ на мошенничество.
- в платёжный шлюз возвращаются результаты проверки заказа на мошенничество. Если заказ признан мошенническим, оплата отклоняется и платеж считается неуспешным. Иначе платёжный шлюз списывает деньги со счёта клиента;
 - платёжный шлюз возвращает сайту УЦ статус платежа (успешный или неуспешный);
 - на странице оплаты сайта УЦ отображается статус платежа.
 - При успешном платеже в СУБД УЦ меняется состояние промокода на «Использован».

Хотя при проектировании систем не рекомендуется включать в описание бизнес-процессов детали интерфейса, для простоты и наглядности рассматриваемого примера, мы нарушим это правило. Теперь рассмотрим, как данное текстовое описание последовательности шагов показать в виде UML-диаграммы последовательности.

Итак, в UML-диаграмме описанной последовательности будут участвовать объекты (рис.30):

- Пользователь;
- страница оплаты сайта учебного центра (Сайт УЦ);
- СУБД УЦ;
- Платежный шлюз;
- Антифрод-система.

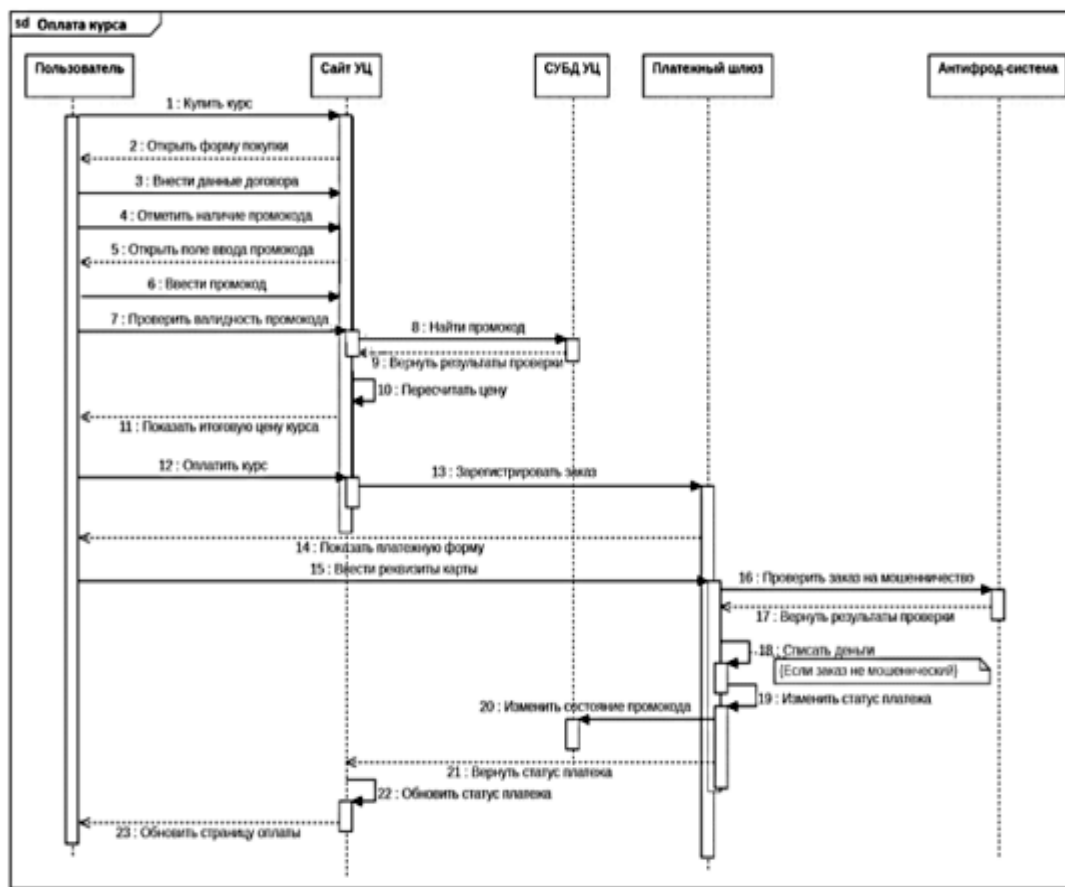


Рис. 31. Диаграмма последовательности «Оплата курса»

Взаимодействие между ними на диаграмме будет отображено в виде прямых сигналов (сплошные стрелки) и ответов (пунктирные) в рамках линии жизни объекта, показанной как длинный вертикальный прямоугольник.

2.4. Package diagram (диаграмма пакетов)

Пакет (package) — элемент модели, используемый для группировки других элементов модели. Элементы модели, которые входят в состав некоторого пакета, называются членами этого пакета. Пакет владеет всеми своими членами. В свою очередь, про члены пакета говорят, что они находятся в собственности пакета или принадлежат ему. Если некоторый пакет удаляется из модели, то из модели также удаляются все члены, которые находятся в собственности этого пакета. Элементы модели, которые входят в пространство имен некоторого пакета, называются элементами этого пакета. Среди них могут находиться как члены этого пакета, так и члены других импортируемых пакетов. На диаграммах пакетов одни пакеты могут быть вложены в другие пакеты. В этом случае вложенный пакет называется подпакетом, а все элементы подпакета будут также принадлежать любому пакету, для которого рассматриваемый подпакет является вложенным. Тем самым для элементов модели может быть задано отношение вложенности пакетов, которое представляет собой строгую иерархию. Как уже было описано ранее, пакет изображается в форме большого прямоугольника, к верхней левой стороне которого присоединен маленький прямоугольник. Напомним, что содержимое пакетов может быть изображено на диаграммах двумя способами. Первый способ связан с изображением членов пакета в большом прямоугольнике. В этом случае имя пакета размещается в верхней вкладке. Альтернативно члены пакета могут быть изображены снаружи пакета и соединены с пакетом отрезком линии со знаком плюс (+) в круге на конце, присоединенном к пакету. В этом случае

имя пакета размещается в большом прямоугольнике. В качестве примера рассмотрим несколько вариантов представления пакет

Перед именем пакета может помещаться строка текста, содержащая ключевое слово в форме стереотипа, например: <<façade>>, <<framework>, <<model>>, <<stub>> и <<topLevel>>.

Импорт пакета (package import) — направленное отношение между пакетами, при котором члены одного пакета могут быть добавлены в пространство имен другого пакета.

Импорт пакета (package import) — направленное отношение между пакетами, при котором члены одного пакета могут быть добавлены в пространство имен другого пакета.

Диаграмма пакетов (package diagram) предназначена для представления размещения элементов модели в пакетах и спецификации зависимостей между пакетами и их элементами.

Текстовый синтаксис имеет следующий вид:

```
'{import' '}' | '{access' '}'
```

Дополнительно может быть показано имя псевдоним:

```
'{import' 'as' '}' | '{access' 'as' '}'
```

Импорт пакета изображается с использованием пунктирной линии с открытой стрелкой от импортирующего пространства имен к импортируемому пакету. Рядом с пунктирной линией обязательно изображается одно из двух ключевых слов для указания вида импорта пакета: <<import>> — для общедоступного импорта пакета, и <<access>> — для закрытого импорта пакета. При общедоступном (стереотип <<import>>) импорте пакета импортируемые элементы могут быть видимы за пределами импортирующего пакета. Если же импорт пакета является закрытым (стереотип <<access>>), то импортируемые элементы не могут быть видимы за пределами

импортирующего пакета. По умолчанию импорт пакета является общедоступным

Пример импорта пакетов представлен на рис. 32.

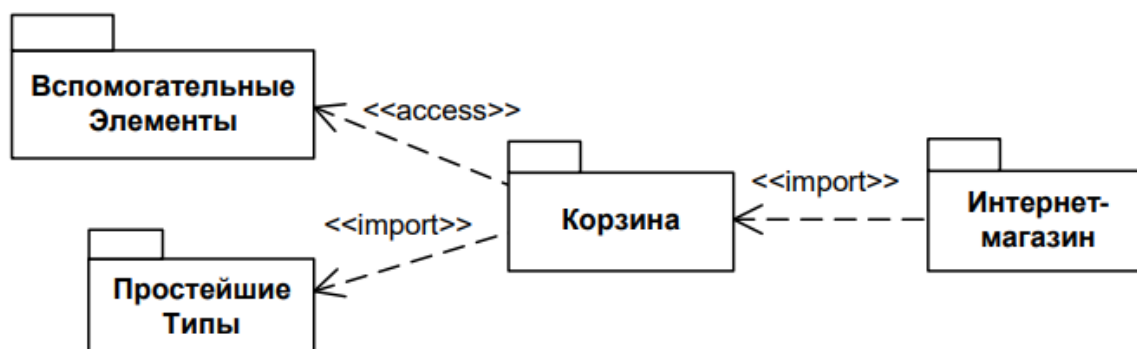


Рис. 32. Примеры общедоступного и закрытого импорта пакетов

В этом случае общедоступные члены пакета Простейшие Типы импортируются в пакет Корзина, после чего они далее, вместе с общедоступными собственными членами пакета Корзина, импортируются в пакет Интернет-магазин. С другой стороны, общедоступные члены пакета Вспомогательные Элементы импортируются только в пакет Корзина. Тем самым на них нельзя ссылаться с использованием неквалифицированных имен из пакета Интернет-магазин.

Также пример импорта и видимости пакетов и их классов представлен на рис. 33.

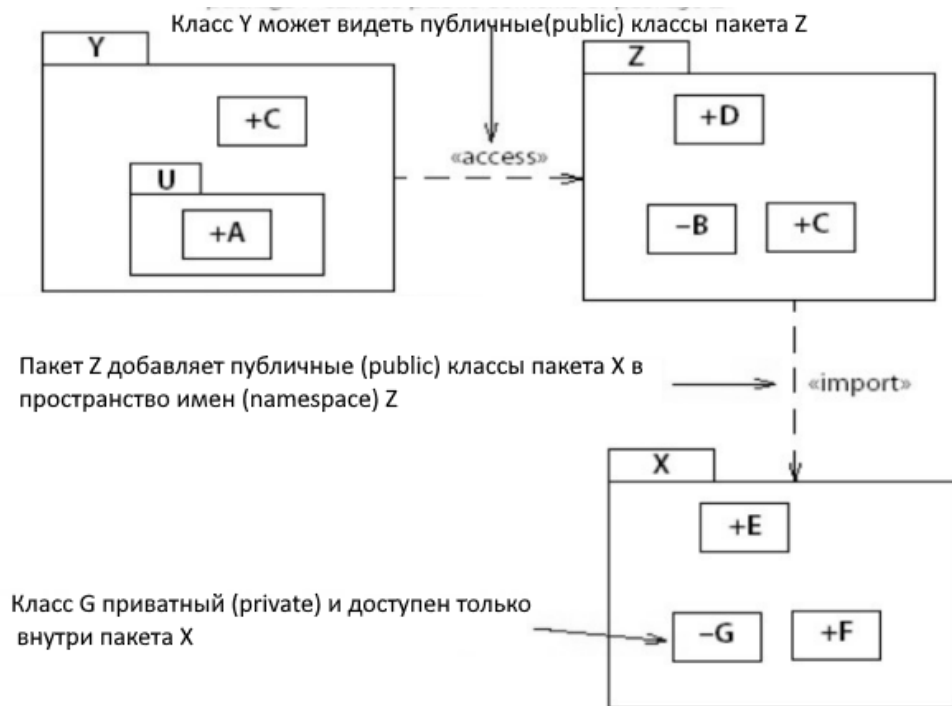


Рис. 33. Пример импорта, видимости пакетов и классов

Слияние пакетов (package merge) — направленное отношение между двумя пакетами, один из которых расширяет свое содержание посредством добавления содержимого другого пакета. Слияние пакетов является специализацией отношения зависимости. При этом зависимый пакет получил название источника (source), а независимый пакет — цели (target). В результате слияния пакетов содержимое пакета источника расширяется за счет добавления к нему содержимого пакета-цели. Слияние пакетов похоже на отношение обобщения в том смысле, что пакет источник концептуально добавляет характеристики пакета цели к своим собственным характеристикам, образуя в результате пакет, который объединяет характеристики обоих пакетов. Концепция слияния пакетов представлена на рис. 34.

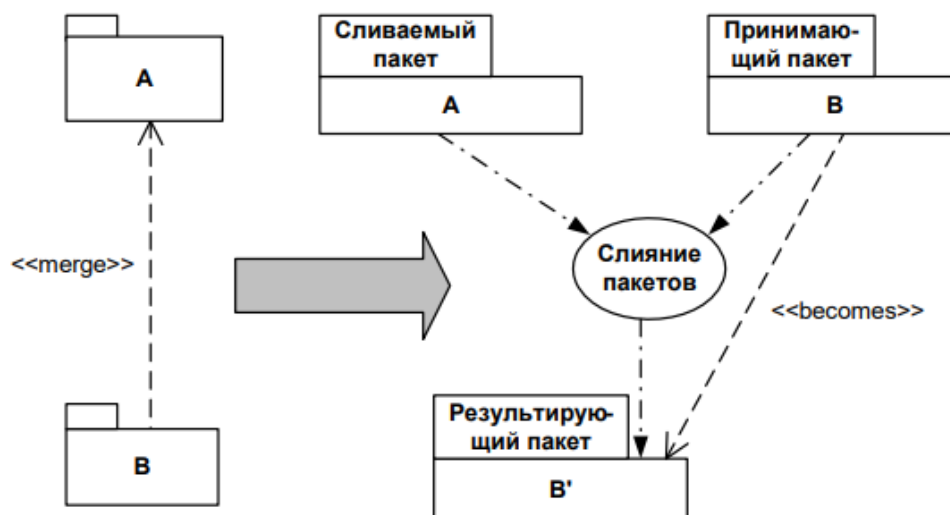


Рис. 34. Концептуальное представление семантики слияния пакетов.

Слияние пакетов изображается пунктирной линией с открытой стрелкой, проведенной от пакета-источника к пакету-цели. Дополнительно рядом с этой пунктирной линией изображается стереотип `<<merge>>`. Некоторые правила и ограничения слияния пакетов приведены ниже.

Ограничения:

- на диаграмме пакетов для отношения слияния не должно быть циклов;
- пакет не может сливаться с пакетом, в котором он содержится;
- пакет не может сливаться с пакетом, который он содержит;
- сливаемый элемент, метатип которого не является видом: Пакет, Класс, Тип данных, Свойство, Ассоциация, Операция, Ограничение, Перечисление или Перечисление литералов, не может иметь в качестве принимающего элемента элемент с тем же именем и метатипом, за исключением, когда принимающий элемент является точной копией сливаемого элемента;
- слияние пакетов является допустимым, если и только если все ограничения, требуемые для выполнения слияния, удовлетворены;

- совпадающие типизированные элементы, например, свойства и параметры, должны иметь согласованные типы.
- для типов, таких как классы или типы данных, согласованный тип является или тем же типом, или общим супертипом. Во всех других случаях соответствие означает, что типы должны быть одними и теми же;
- принимающий элемент не может иметь явных ссылок ни на один из сливаемых элементов.

Преобразования:

- если для сливаемого или принимающего элемента не существует совпадающего элемента, то они полностью копируются в результирующий пакет. Это правило принимается по умолчанию;
- результатом слияния двух элементов с совпадающими именами и метатипами, которые являются точными копиями друг друга, является принимающий элемент;
- совпадающие элементы объединяются согласно правилам преобразования, специфическим для их метатипа, а результаты включаются в результирующий пакет;
- все ссылки на типизированные элементы, которые оканчиваются в результирующем пакете, преобразовываются в ссылки на соответствующие результирующие элементы, но не на соответствующие им приращения;
- для всех совпадающих элементов: если оба совпадающих элемента имеют закрытую видимость, результирующий элемент должен иметь закрытую видимость, в противном случае результирующий элемент должен иметь общедоступную видимость;

- для всех совпадающих элементов классификаторов: если оба совпадающих элемента являются абстрактными, то результирующий элемент является абстрактным, в противном случае результирующий элемент не является абстрактным;
- для всех совпадающих элементов: если оба совпадающих элемента не являются производными, то результирующий элемент также не является производным, в противном случае результирующий элемент является про
- для всех совпадающих элементов кратности: нижняя граница результирующей кратности является меньшей из нижних границ кратности совпадающих элементов;
- для всех совпадающих элементов кратности: верхняя граница результирующей кратности является большей из верхних границ кратности совпадающих элементов;
- любые стереотипы, применяемые к элементу модели или в сливаемом, или в принимающем элементе, являются также применимыми к соответствующему результирующему элементу.

Пример построения диаграммы пакетов приведен на рис.35,36.

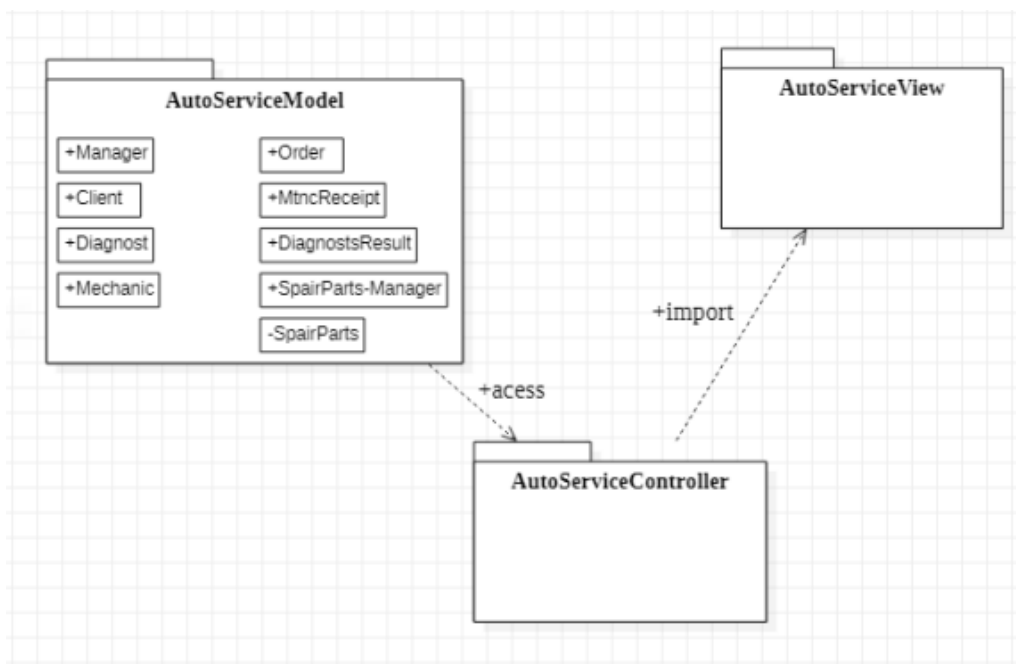


Рис.35. Диаграмма пакетов для ИС «Автомастерская»



Рис. 36 Диаграмма пакетов для ИС «Учет учебной нагрузки преподавателей»

2.5. Диаграмма компонентов

Компонент является физическим фрагментом реализации системы, который включает в себя программный код (исходный, двоичный, исполняемый), сценарные описания или наборы команд (командные файлы). Компонент – физическая и заменяемая часть системы, которая соответствует набору интерфейсов и обеспечивает его реализацию. В языке UML 2.0

компонент изображается в форме прямоугольника классификатора с ключевым словом <<component>>, записанным в форме стереотипа (рис.37). Дополнительно в правом верхнем углу символа классификатора может быть изображена пиктограмма компонента.

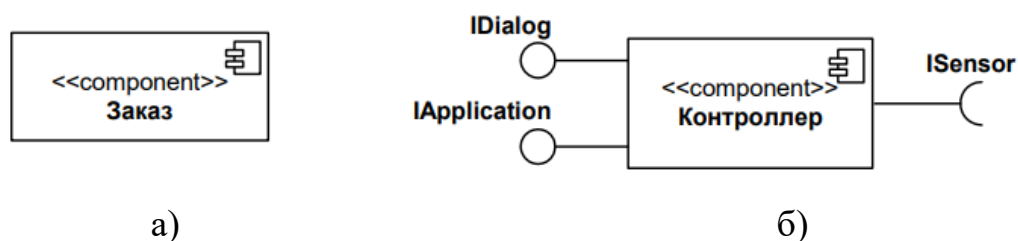


Рис.37. Примеры изображения: а — простого компонента;
б — компонента с интерфейсами

Интерфейс – важная часть понятия «компонент». Он определяет список операций услуг класса или компонента. «Интерфейс – это разъем, который торчит из ящика компонента». Все операции интерфейса открыты и видимы клиенту. Компонент может иметь две разновидности интерфейсов: предоставляемые и требуемые интерфейсы, которые образуют основу для взаимосвязей компонентов между собой с использованием зависимостей и соединителей.

Предоставляемый интерфейс (provided interface) — интерфейс, который компонент предлагает для своего окружения.

Требуемый интерфейс (required interface) — интерфейс, который необходим компоненту от своего окружения для выполнения заявленной функциональности, контракта или поведения.

Наряду с интерфейсами, которые относятся к внешнему представлению, на диаграмме может быть изображено внутреннее представление или представление компонента в форме белого ящика. В этом случае внутри символа компонента изображаются классы или другие классификаторы, которые реализуют поведение этого компонента (рис.38).

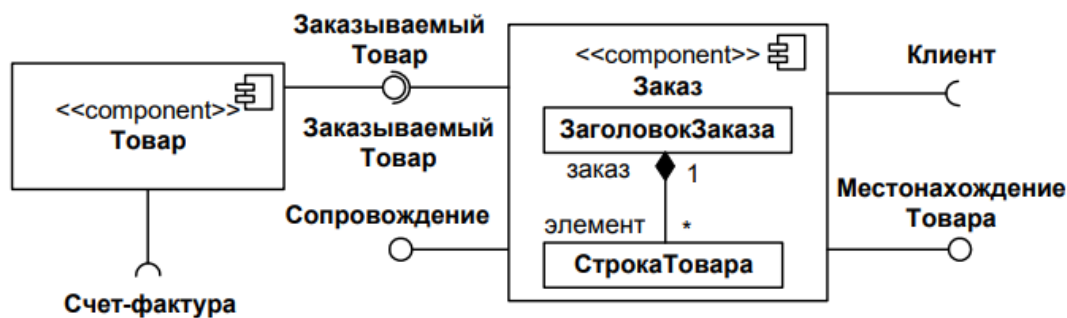


Рис. 38. Пример изображения компонентов с интерфейсами

При проектировании большие системы могут быть разложены на несколько сотен или тысяч компонентов. В текущей модели может быть несколько диаграмм компонентов для отражения пакетов или компонентов некоторого уровня. Компоненты – базисные строительные блоки, из которых может проектироваться и составляться система. Компонент может появляться на различных уровнях иерархии представления сложной системы. Не все стереотипы компонентов, применяемые в программировании, присутствуют в языке UML, но наиболее распространенные имеют соответствующие пиктограммы.

2.6. Диаграмма деятельности (activity diagram)

Центральным аспектом моделирования деятельности в языке UML 2.0 является последовательность действий и условий их выполнения, а также поток объектов, которые являются необходимыми для выполнения отдельных действий или являются результатом их выполнения. Для моделирования деятельности используется механизм, аналогичный сетям Петри. Это позволяет рассматривать не только поток управления, но также потоки объектов и данных.

В общем случае деятельность является самостоятельным элементом поведения, которая, в свою очередь, может включать в себя другие деятельности или отдельные действия. При этом элементарной единицей спе-

цификации поведения является действие. Последовательность выполнения действий в деятельности описывается с помощью потока управления, который лежит в основе понимания модели поведения в представлении диаграммы деятельности.

Деятельность (activity) является спецификацией параметризованного поведения в форме координируемой последовательности подчиненных единиц, индивидуальными элементами которых являются действия.

Элементами, из которых состоят деятельности, являются действия. Действие (action) представляет собой элементарную единицу спецификации поведения, которая не может быть далее декомпозирована в форме деятельности.

2.6.1. Узлы и дуги деятельности

Основными графическими элементами на диаграмме деятельности являются различные узлы и дуги деятельности.

Узел деятельности (activity node) является абстрактным классом для отдельных точек в потоке деятельности, соединенных дугами (рис 39).

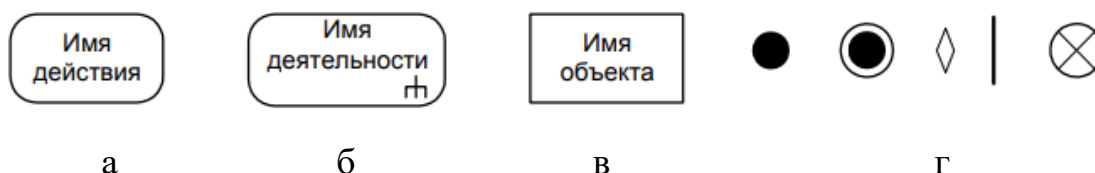


Рис.39. Нотация для различных узлов деятельности: а — узел действия; б — узел вызова деятельности; в — узел объекта; г — узел управления

Дуга деятельности (activity edge) является абстрактным классом для направленных соединений между двумя узлами деятельности (рис. 40).

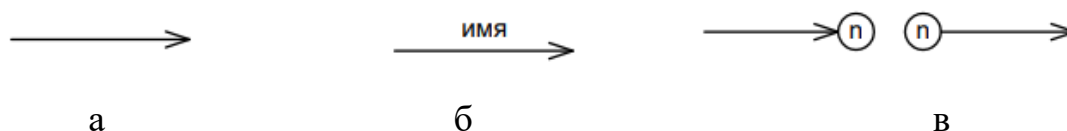


Рис. 40. Нотация для различных дуг деятельности: а — имя дуги отсутствует; б — дуга имеет имя; в — дуга с соединителем

Семантика деятельности в языке UML 2.0 основывается на потоке маркеров. Маркер (token) — элемент модели, предназначенный для представления некоторого объекта, данных или управления и существующий на диаграмме деятельности в отдельном узле. Каждый маркер отличается от любого другого, даже если он содержит, то же значение, что и другой. Маркеры могут протекать вдоль дуги, правила следования которых зависят от вида дуги и характеристик ее источника и цели. Маркер перемещается по дуге, если он одновременно удовлетворяет правилам для узла цели, дуги и узла источника.

Диаграмма активности(деятельности) имеет вид (рис.41):

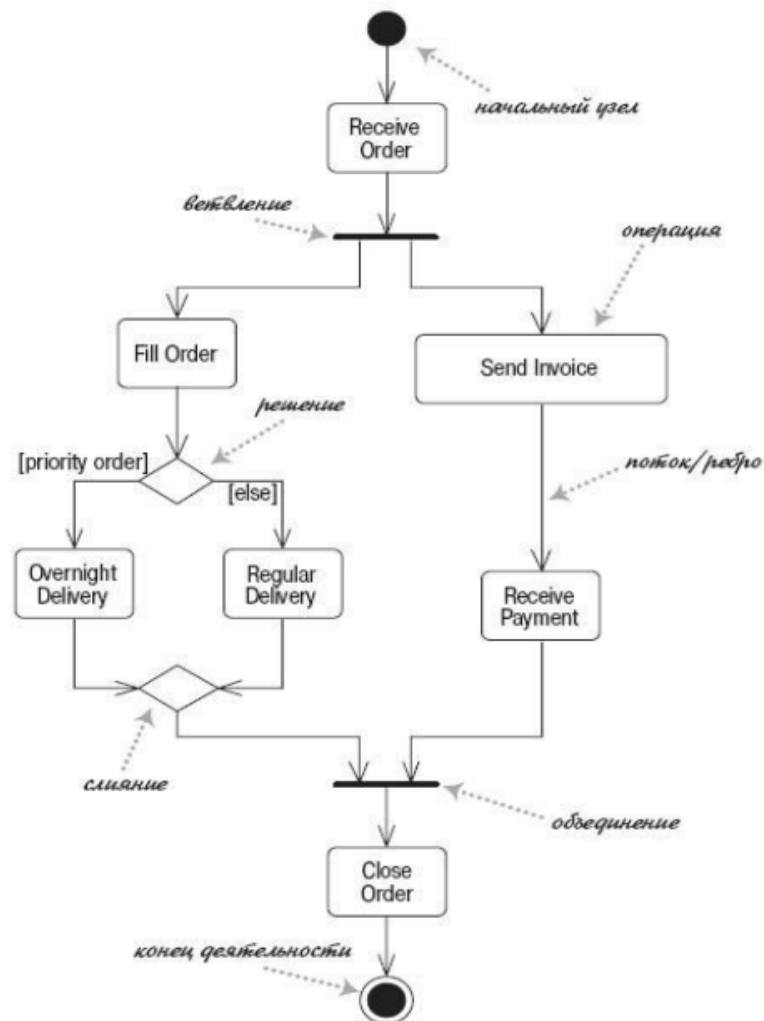


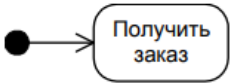
Рис. 40. Пример диаграмма деятельности

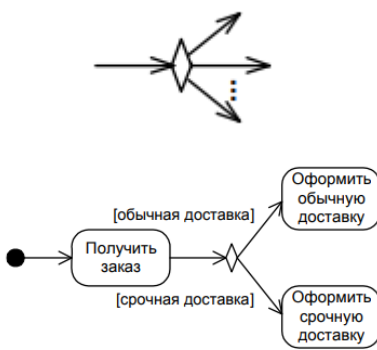
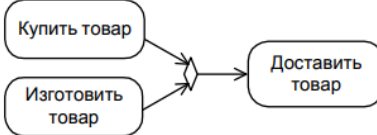
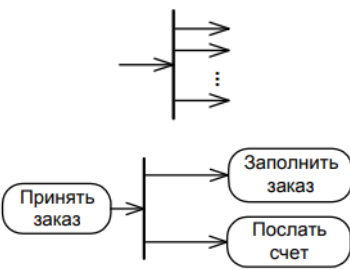
Существуют две разновидности дуг деятельности: дуги потока управления и дуги потока данных. Они обозначаются одинаково — линией с тонкой стрелкой. Поток управления (control flow) представляется в форме дуги деятельности, которая связывает между собой два узла деятельности и по которой передаются только маркеры управления. При этом все маркеры управления, которые предлагаются узлом-источником, предлагаются узлу цели, а вдоль дуги потока управления не могут следовать объекты и данные.

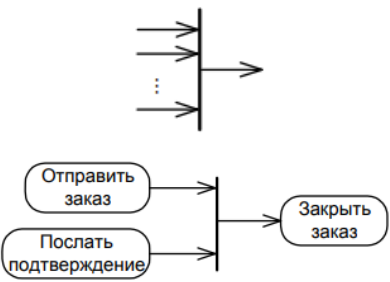
Поток объектов (object flow) представляется в форме дуги деятельности, по которой передаются только маркеры объектов или данных. При этом все маркеры, предлагаемые узлом-источником, предлагаются для узла цели с учетом ограничений, которые могут быть дополнительно специфицированы с помощью веса дуги.

Семантика узлов управления представлена в таблице 10.

Таблица 10 Узлы управления и их описание

Тип графического элемента	Нотация	Описание
Начальный узел (initial node)		Является узлом управления, в котором начинается поток при вызове деятельности. Начальный узел является начальной точкой в выполнении деятельности. Когда деятельность вызывается, то в начальном узле размещается маркер управления, который предлагается всем выходящим из него дугам. Начальный узел не может иметь входящих дуг, а в качестве источника может иметь только дуги управления.
Узел финала деятельности (activity final node)		Является узлом управления, который прекращает или останавливает все потоки в деятельности. Узлы финала деятельности не имеют выходящих дуг, поэтому он только принимает все маркеры, которые предлагаются на входящих дугах. Маркер, дости-

		гающий финального узла деятельности, прекращает эту деятельность.
Узел финала деятельности (activity final node)		Является узлом управления, который прекращает или останавливает все потоки в деятельности. Узлы финала деятельности не имеют выходящих дуг, поэтому он только принимает все маркеры, которые предлагаются на входящих дугах. Маркер, достигающий финального узла деятельности, прекращает эту деятельность.
Узел решения (decision node)		Является узлом управления, который выбирает между выходящими потоками. Узел решения должен иметь отдельную дугу деятельности, входящую в него, и одну или более дуг, выходящих из него. Дуги, входящие в узел решения и выходящие из него, должны быть все или потоками управления, или потоками объектов. Каждый маркер, достигающий узла решения, может перейти только в одну выходящую дугу. При этом маркеры не копируются.
Узел слияния (merge node)		Является узлом управления, который соединяет вместе несколько альтернативных потоков. Узел слияния имеет несколько входящих дуг и одну выходящую дугу. Он не используется для синхронизации параллельных потоков, но производит один поток из нескольких, как правило, альтернативных потоков. Дуги, входящие в узел слияния и выходящие из него, все должны быть или потоками управления, или потоками объектов.
Узел разделения (fork node)		Является узлом управления, который расщепляет поток на несколько параллельных потоков. Узел разделения имеет одну входящую дугу и несколько выходящих дуг. Дуги, входящие в узел разделения и выходящие из него, должны быть все или потоками управления, или потоками объектов. Маркеры, достигающие узла разделения, дублируются во все выходящие дуги. Когда предлагае-

		мый маркер признается на всех выходящих дугах, создаются дубликаты этого маркера, и один экземпляр проходит по каждой дуге
Узел соединения (join node)		Является узлом управления, который синхронизирует несколько потоков. Узел соединения имеет несколько входящих дуг и одну выходящую дугу. Узлы соединения могут иметь дополнительную логическую спецификацию условий, при выполнении которых они должны генерировать маркер на выходе. Если для узла соединения существуют маркеры во всех его входящих дугах, то выходящей дуге предлагаются маркеры согласно следующим правилам: • если все маркеры, предлагаемые на входящих дугах, являются маркерами управления, то выходящей дуге предлагается один маркер управления; • если часть маркеров, предлагаемых на входящих дугах, являются маркерами управления, а другие являются маркерами данных, то выходящей дуге предлагаются только маркеры данных. Они предлагаются выходящей дуге в том же порядке, в каком предлагаются на входе этого узла соединения.

Пример диаграммы деятельности оформление пассажира представлен на рис.42.

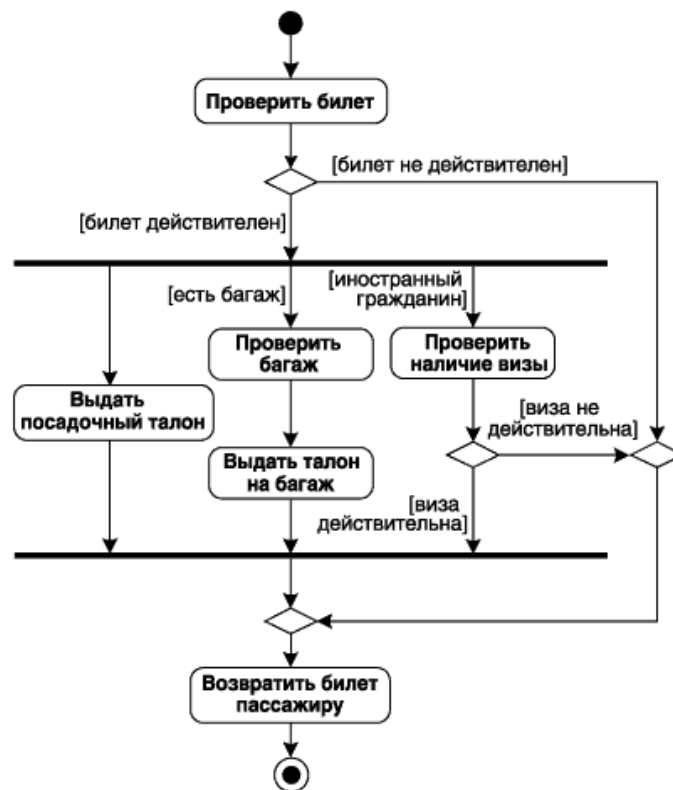


Рис. 42. Диаграмма деятельности оформление пассажира на рейс

2.6.2. Специальные действия: передача сигнала и действие приема событий

Действие передачи сигнала (send signal action) является действием, которое на основе своих входов создает экземпляр сигнала и передает его объекту цели. Результатом передачи сигнала может быть начало выполнения некоторой деятельности или срабатывание перехода некоторого конечного автомата. При этом отправитель сигнала продолжает свое выполнение, не заботясь о результате. Любые ответные сообщения не передаются отправителю или игнорируются им (рис.43).

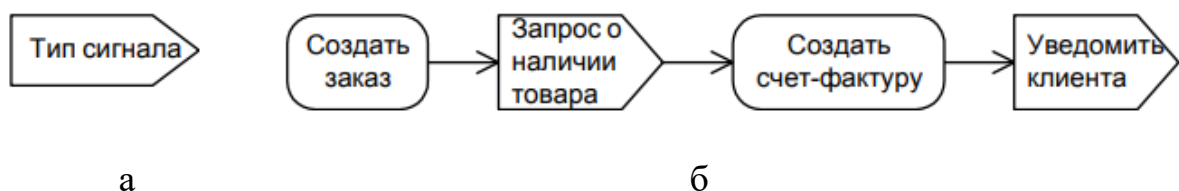


Рис.43. Примеры изображения действий передачи сигнала: а — с помощью выпуклого пятиугольника; б — пример

При выполнении деятельности экземпляр сигнала передается объекту цели только в том случае, если закончено выполнение его входного действия. Объект цели может быть, как локальным, так и удаленным

Действие приема события (accept event action) является действием, которое ожидает наступление некоторого события (рис. 44). Действие приема события предназначено для обработки локальных или глобальных событий в ходе выполнения деятельности. Обработку действия приема события выполняет тот объект, который владеет деятельностью. Наступления событий обнаруживаются объектами независимо от выполняющихся действий, при этом все наступившие события хранятся этим объектом.

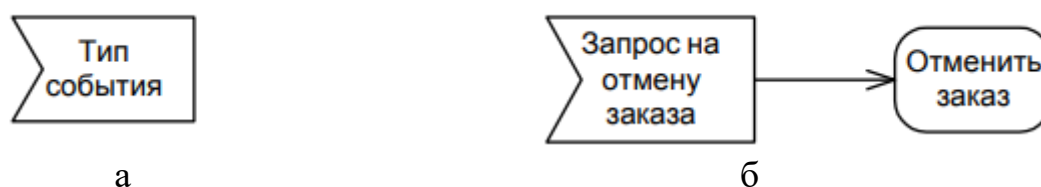


Рис. 44. Примеры изображения действий приема события: а — с помощью вогнутого пятиугольника; б — пример

2.6.3. Специальные регионы: разбиение деятельности

Разбиение деятельности (activity partition) — элемент модели, предназначенный для группировки действий, которые относятся к одной деятельности и имеют некоторую общую характеристику.

На практике при моделировании бизнес-процессов разбиения наиболее часто соответствуют организационным единицам. Разбиения также используются для графического изображения ресурсов совместно с узлами деятельности. В языке UML 2.0 разбиения могут быть иерархическими и многомерными.

Примеры разбиения деятельности приведены на рис. 45.

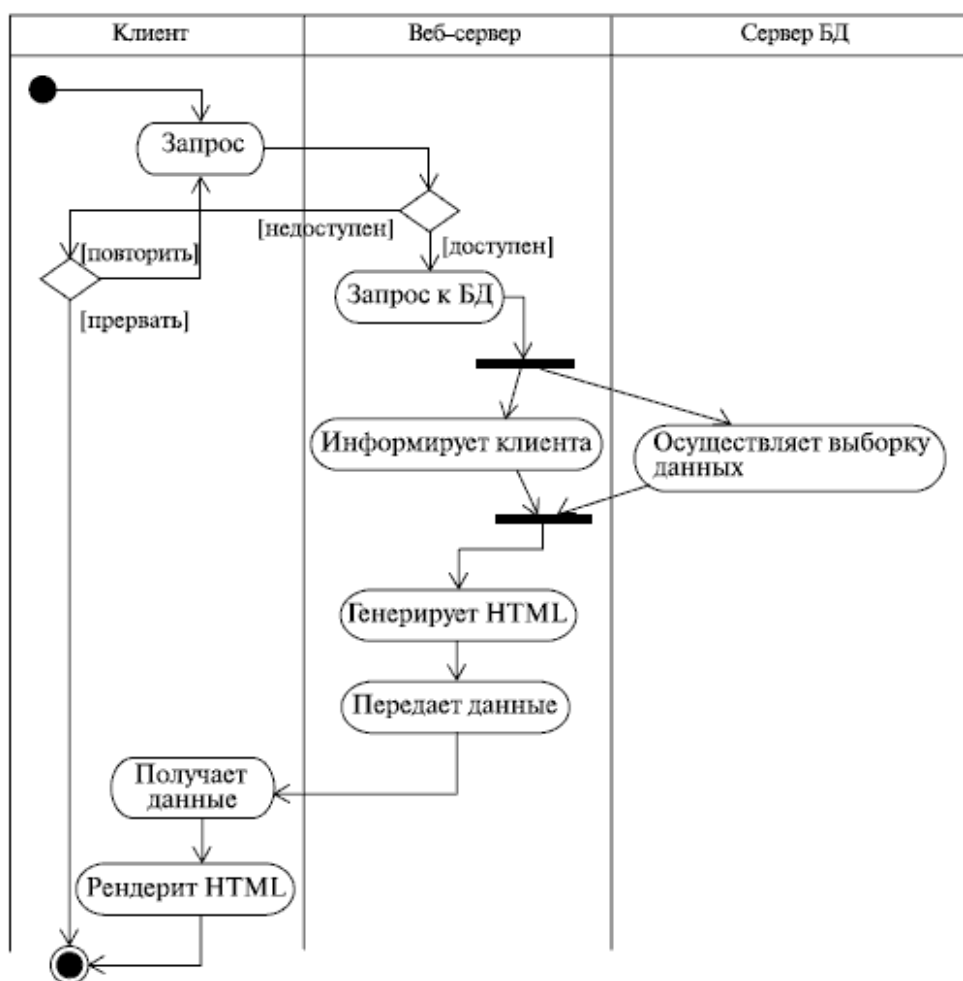


Рис. 45. Пример применения разбиения на диаграмме деятельности

Имя разбиения может означать роль или объект, которому она соответствует.

Пример диаграммы деятельности ИС «Автосервис» (рис.46-48).

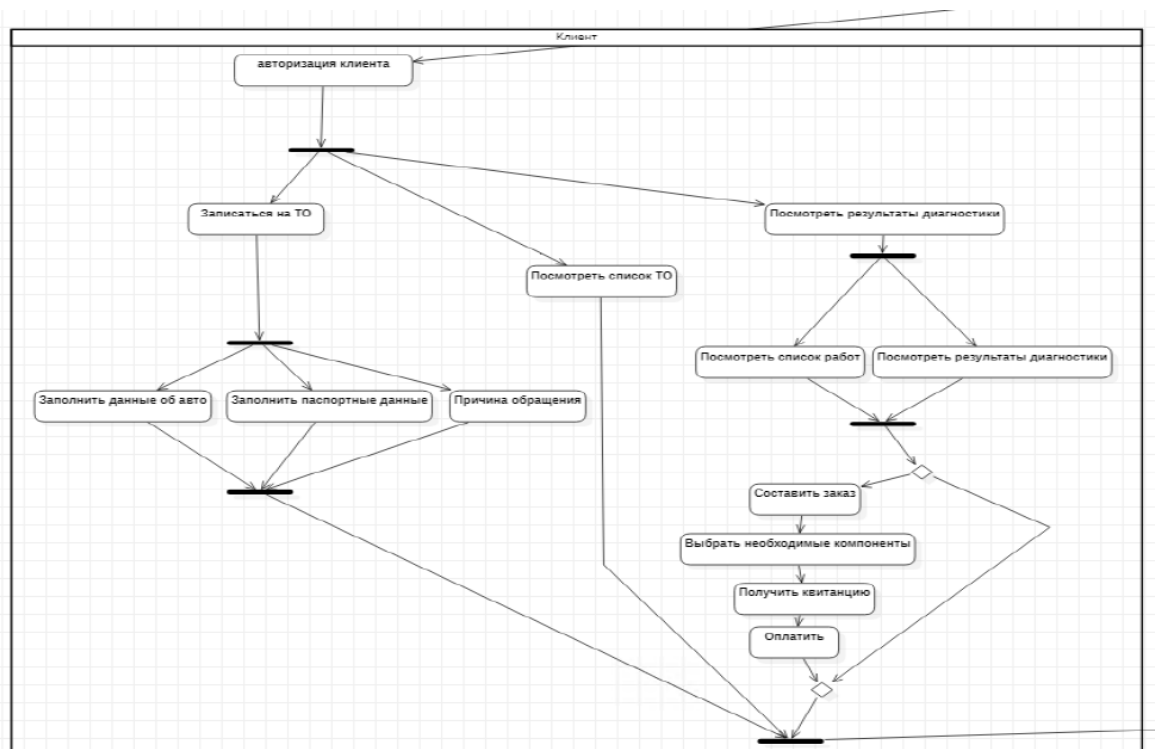


Рис. 46. ИС «Автосервис», разбиение деятельности - Клиент.

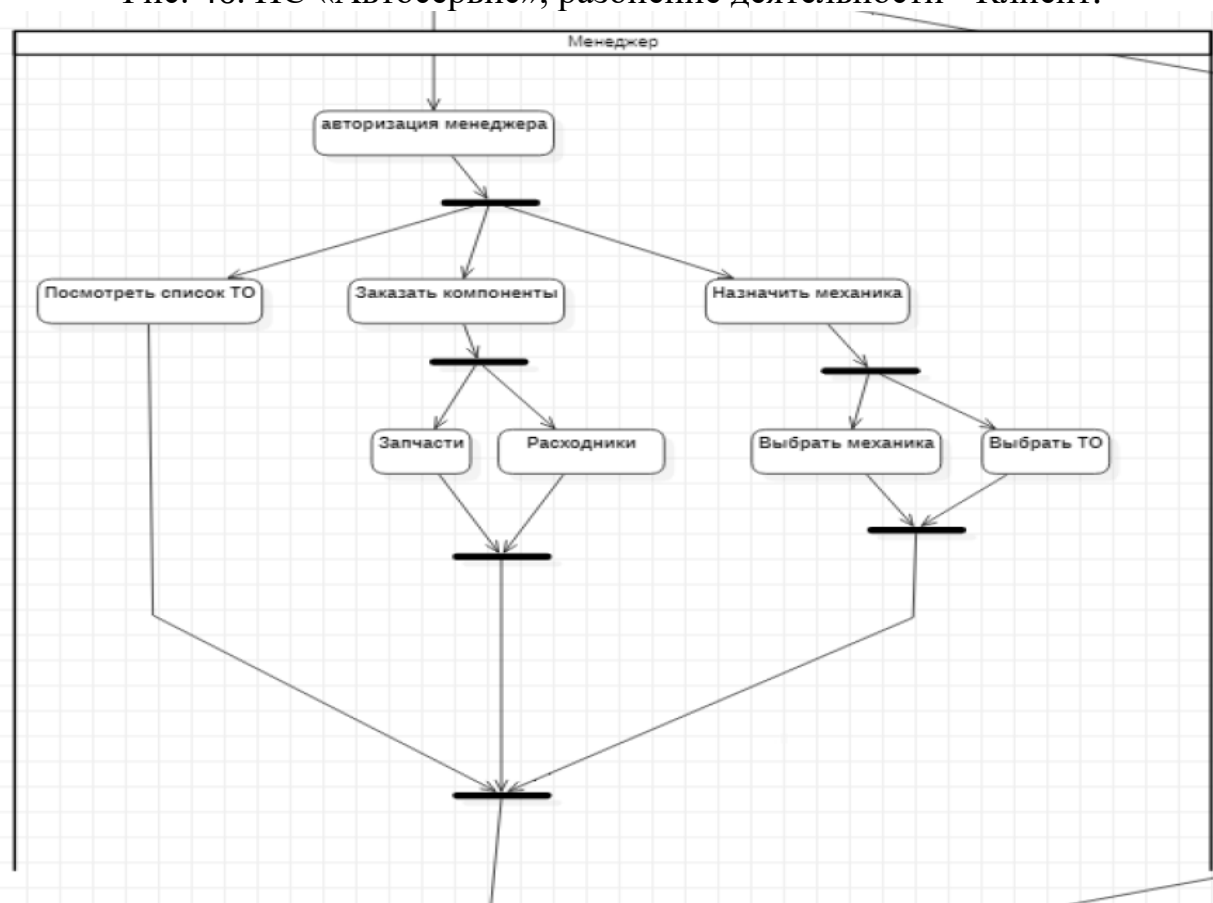


Рис. 47. ИС «Автосервис», разбиение деятельности - Менеджер

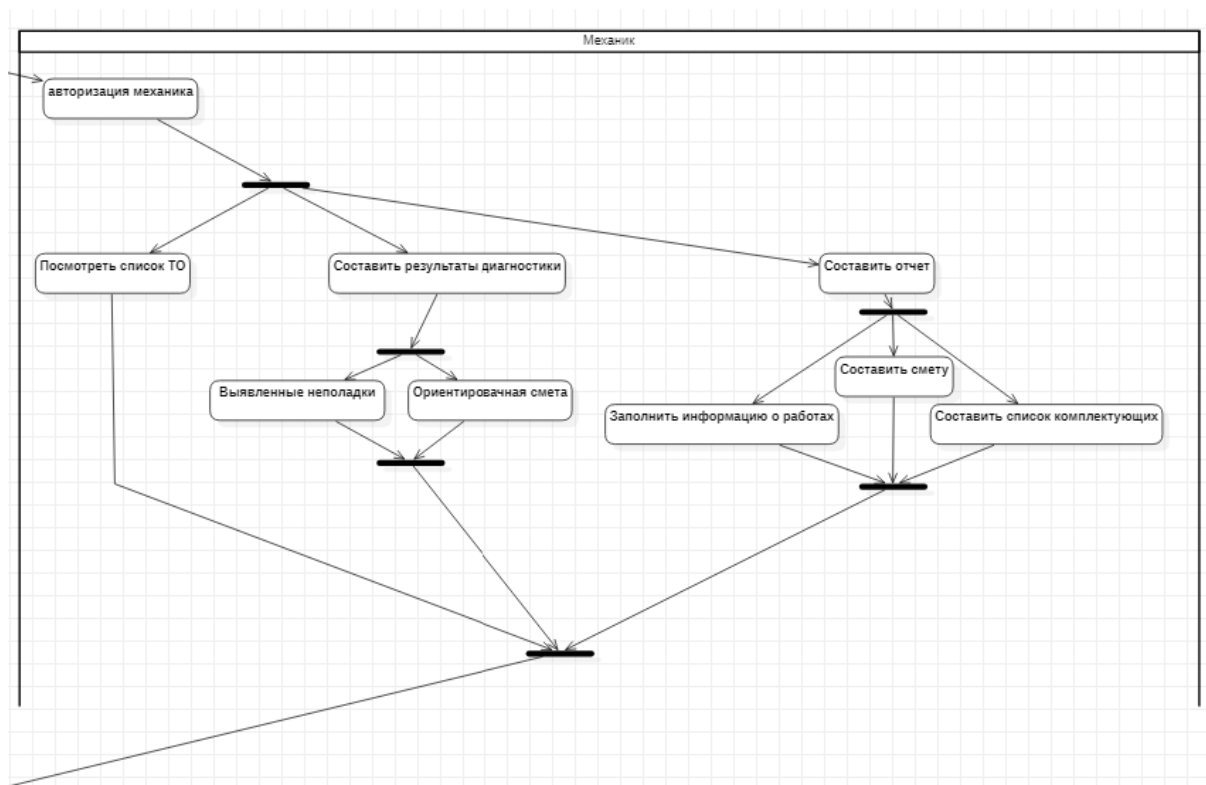


Рис. 48. ИС «Автосервис», разбиение деятельности - Механик

Пример диаграммы деятельности оформление заказа в аптеке (рис.49).

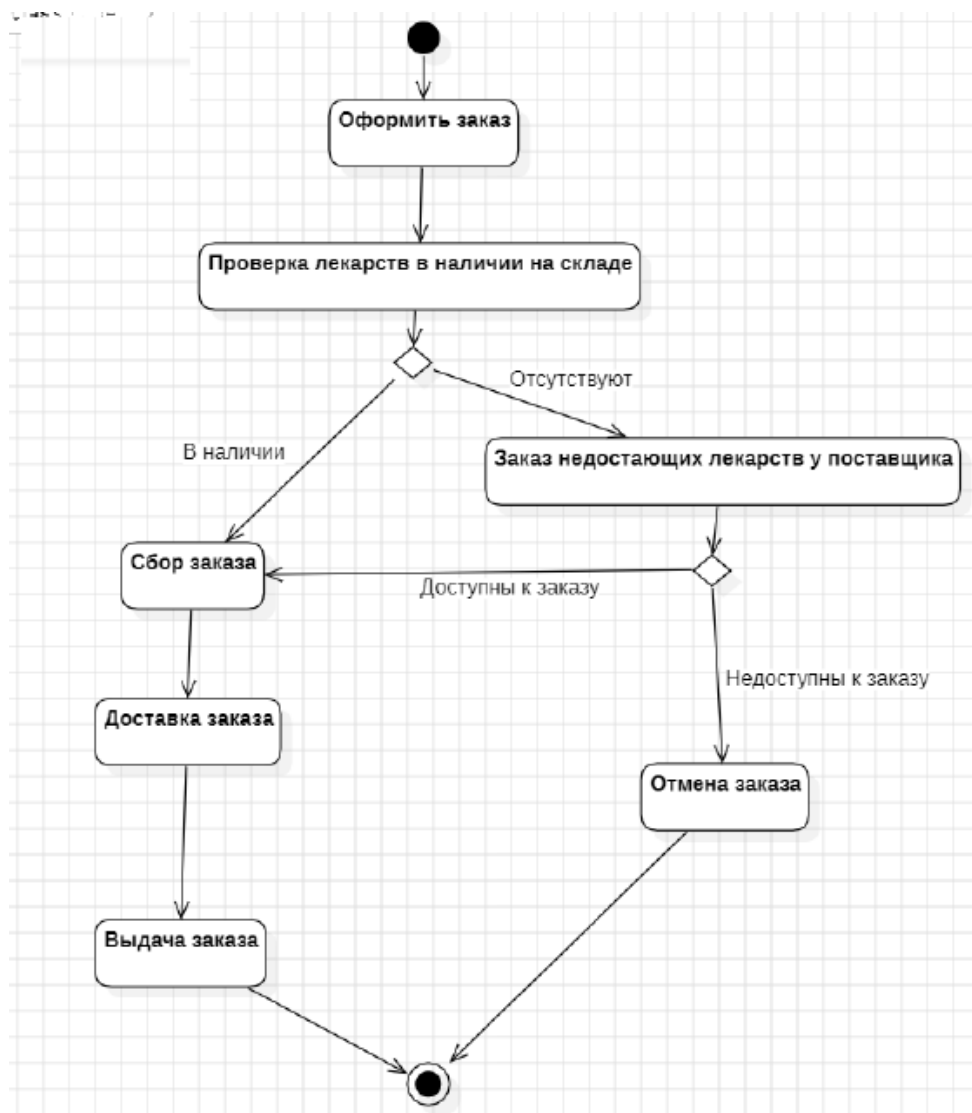


Рис. 49. Диаграмма деятельности оформления заказа в аптеке

2.7. Диаграмма конечного автомата (state machine diagram)

Поведение большинства объектов реальных систем можно представить с точки зрения конечных автоматов, то есть поведение объекта отражается в его состояниях

Диаграмма конечного автомата используется для моделирования дискретных состояний системы. Она является графом, который представляет некоторый конечный автомат

Конечный автомат (state machine) представляет собой некоторый формализм для моделирования поведения отдельных элементов модели или системы в целом.

Автомат описывает поведение в терминах последовательности состояний, через которые проходит объект в течение своей жизни. Автомат задает поведение системы как цельной, единой сущности, моделирует жизненный цикл единого объекта. Автоматный подход применяют для формализации динамики отдельного трудного для понимания блока системы. Концепция конечного автомата определяет множество понятий, которые могут быть использованы для моделирования дискретного поведения в форме графических моделей типа "состояние — переход".

Взаимодействие определяет поведение системы в виде связей между отдельными частями, и система представляется как сообщество совместно работающих объектов.

Поведение (behavior) является спецификацией того, как экземпляр классификатора изменяет значения отдельных характеристик в течение своего времени жизни.

Состояние – это период в жизни объекта, на протяжении которого он удовлетворяет какому-либо условию, выполняет определенную деятельность или ожидает некоторого события. Между состояниями возможны различного рода переходы: переход, вызываемый некоторым событием, переход без событий, условные переходы или охраняемые переходы

Простое состояние — это состояние, не имеющее подструктуры (рис. 50-51). Состояние, которое имеет подсостояния (вложенные состояния), называется составным состоянием.

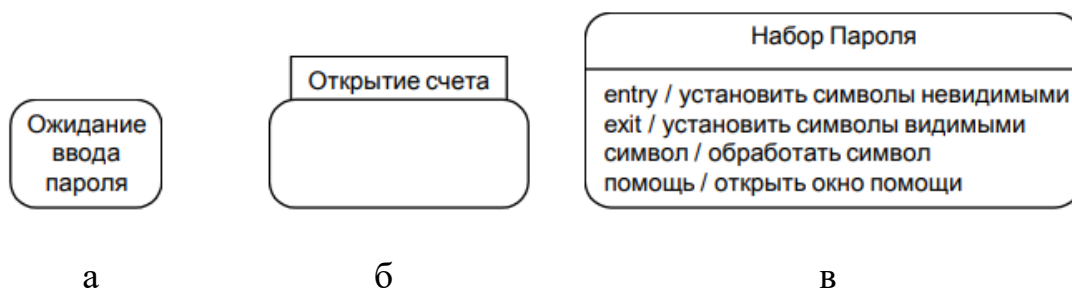


Рис. 50. Варианты изображения простого состояния: а — в виде прямоугольника с закругленными углами; б — в виде прямоугольника со вкладкой, содержащей имя состояния; в — в виде прямоугольника с разделением на несколько секций

Простое состояние с внутренними действиями изображено на рис. 51.

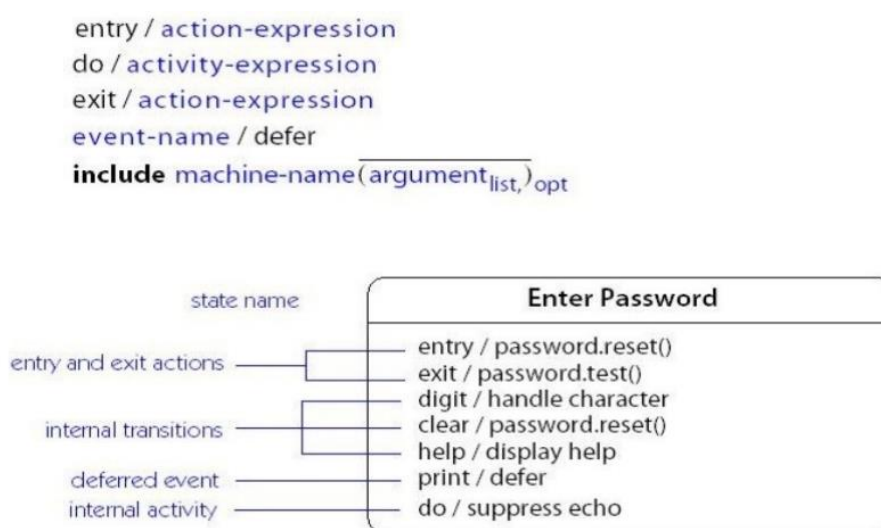


Рис. 51. Простое состояние с внутренними действиями

Подсостояния могут быть вложены на любом уровне. Вложенный конечный автомат может иметь не более одного начального состояния и одного конечного состояния. Подсостояния используются для упрощения сложных плоских автоматов состояний, показывая, что некоторые состояния возможны только в определенном контексте (окружающее состояние).

Узлы на диаграмме конечного автомата имеют ту же семантику, что и узлы на диаграмме активности. На рис. 52 представлен пример использования узла начального псевдосостояния и узла завершения.

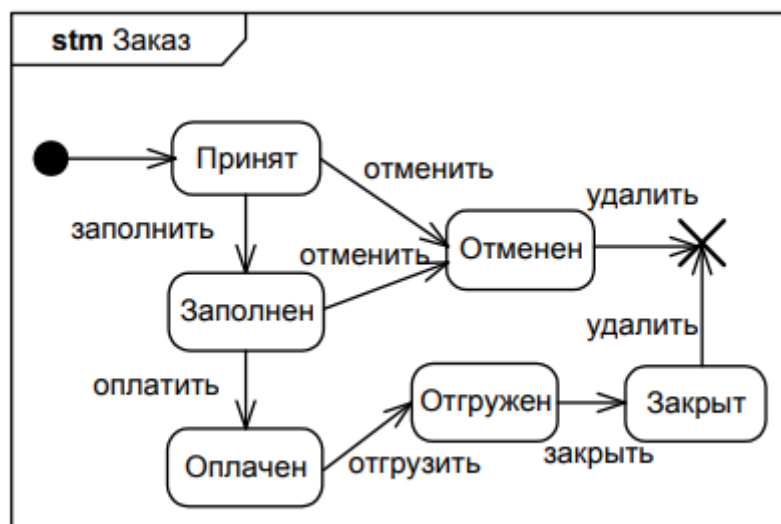


Рис.52. Пример использования начального псевдосостояния и узла завершения

Псевдосостояния соединения и разделение соединяют или расщепляют параллельные потоки входящие или выходящие из данных узлов (рис.53).

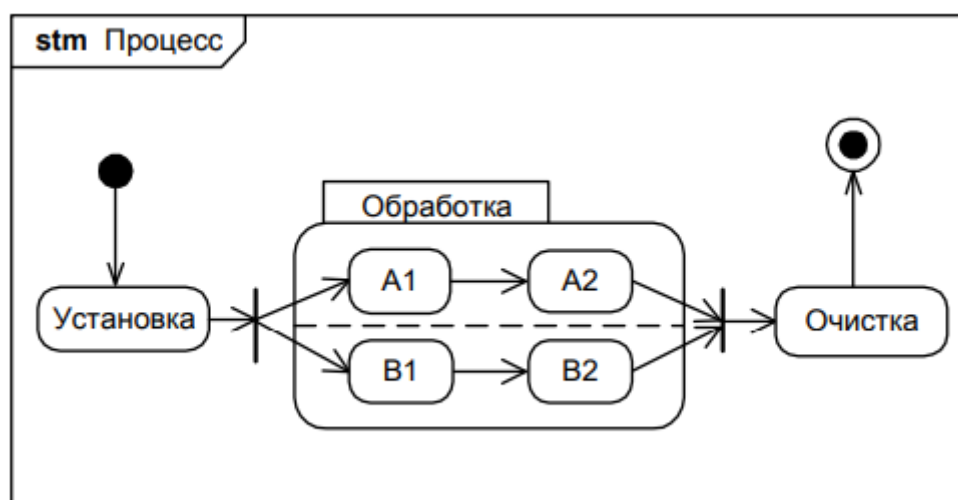


Рис. 53. Пример изображения узлов разделения (слева) и слияния (справа)

2.7.1. Точка входа и выхода, композиционные состояния

Точка входа (entry point) является псевдосостоянием, предназначенным для моделирования входа в некоторый конечный автомат или композиционное состояние. Точка входа используется для соединения некоторого

внешнего перехода, оканчивающегося в этой точке входа с некоторым внутренним переходом, исходящим из этой точки входа.

Точка выхода (exit point) является псевдосостоянием, предназначенным для моделирования выхода из некоторого конечного автомата или композитного состояния. Точка выхода используется для соединения некоторого внутреннего перехода, оканчивающегося в этой точке выхода с некоторым внешним переходом, исходящим из этой точки выхода (рис.54).

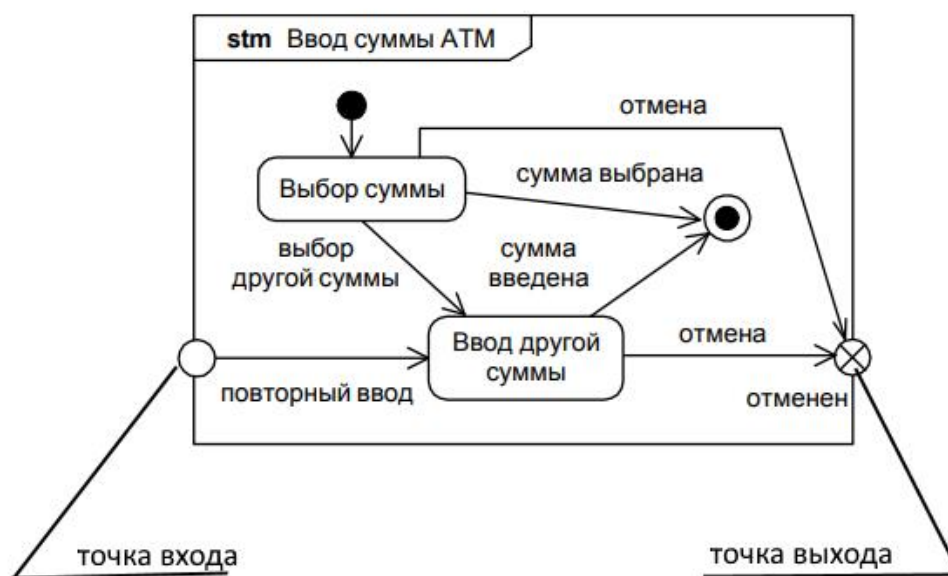


Рис. 54. Пример использования точки входа и точки выхода

Композитное состояние (composite state) — состояние, содержащее в своем составе один регион или несколько ортогональных регионов. Регион (region) — специальный элемент модели, который содержит состояния и переходы и является частью композитного состояния или конечного автомата.

Композитным состоянием является или некоторое простое композитное состояние с всего лишь одним регионом, или некоторое ортогональное композитное состояние с более чем одним регионом. Каждый регион имеет множество взаимно непересекающихся вершин и множество переходов.

Ортогональное (orthogonal) композитное состояние — композитное состояние, содержащее более одного региона, которые в этом случае называются ортогональными регионами (orthogonal regions). Таким образом, любое композитное состояние либо содержит один регион, либо декомпозируется на два или более ортогональных региона.

Любое состояние, заключенное в регион композитного состояния, называется подсостоянием (substate) этого композитного состояния. Оно называется прямым подсостоянием (direct substate), если оно не содержится в никаком другом состоянии; в противном случае оно называется непрямым подсостоянием (indirect substate) (рис.55 -56).

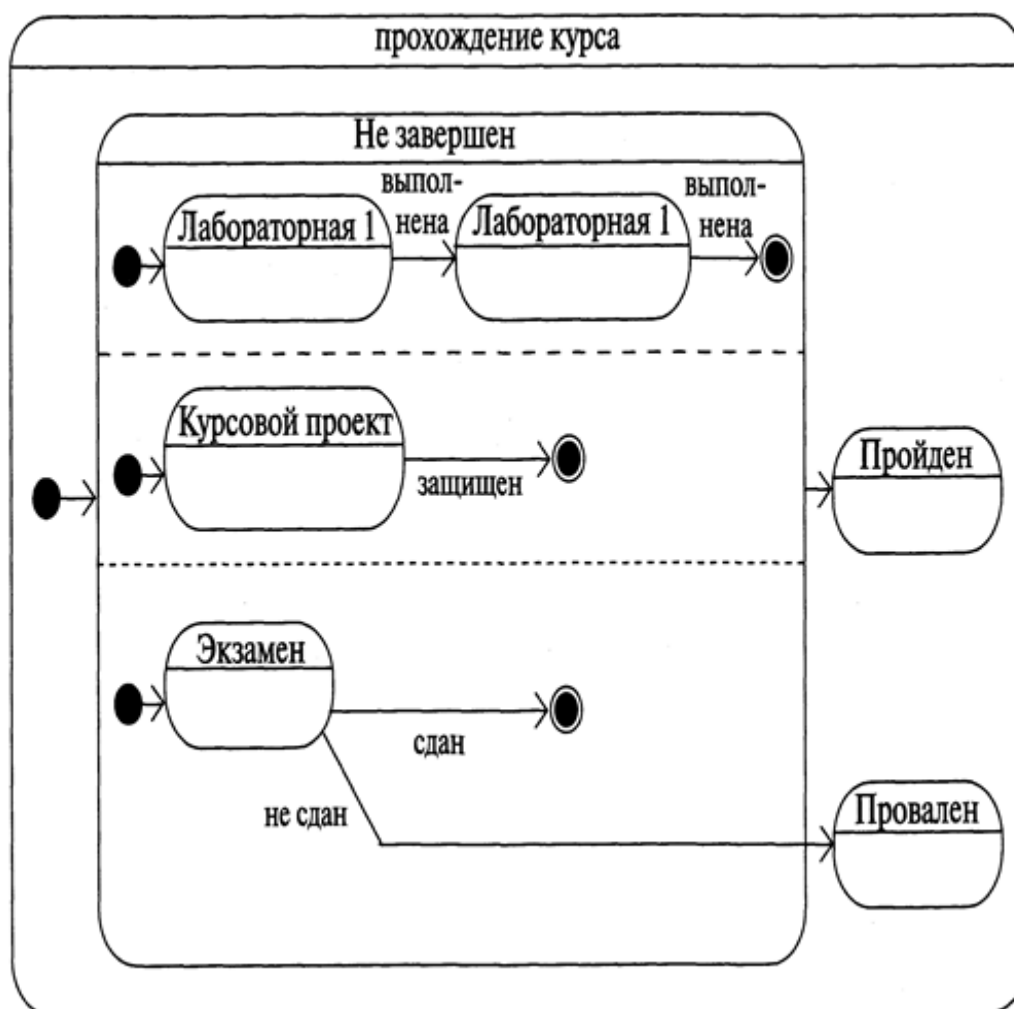


Рис. 55. Пример использования композитного состояния

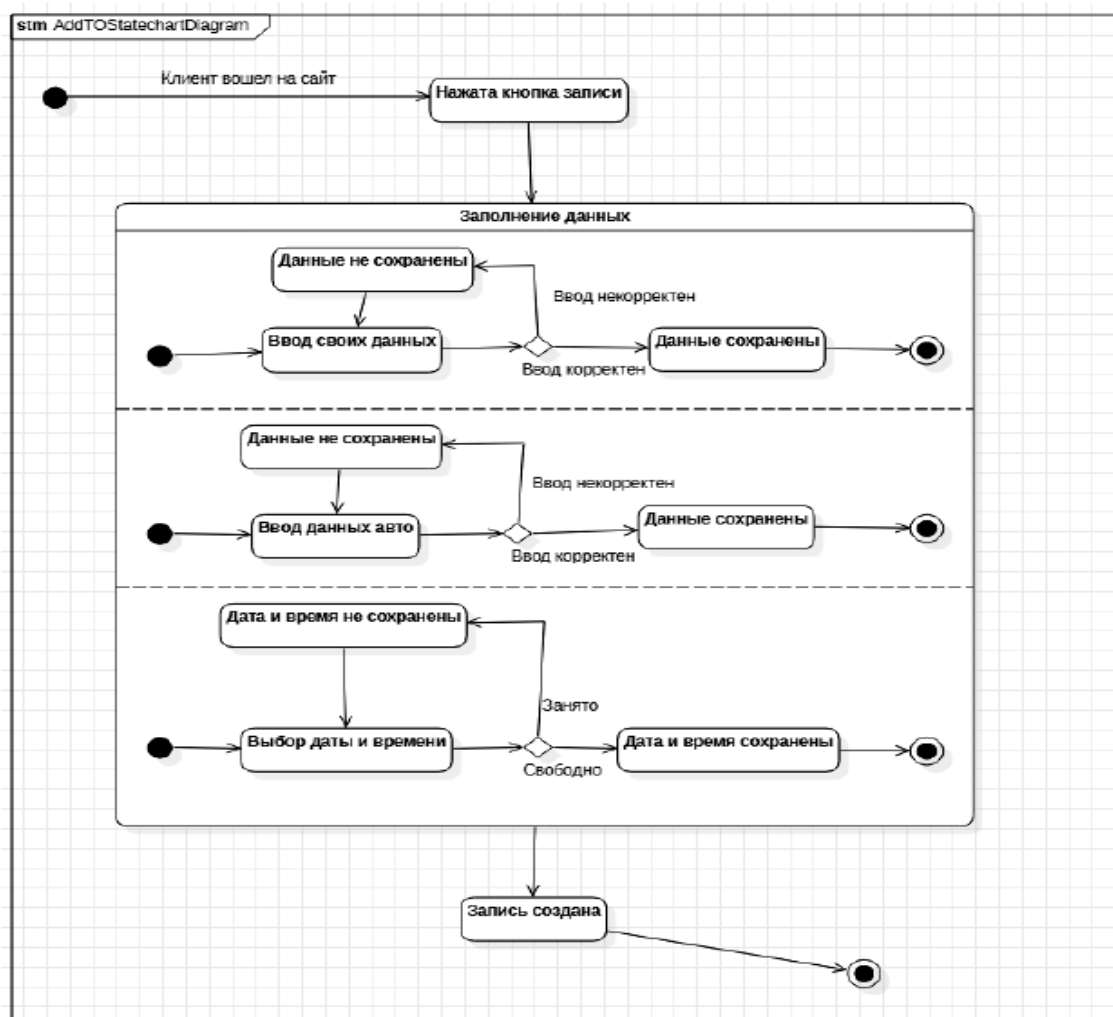


Рис. 56 Пример диаграммы конечного автомата ИС «Автосервис» - процесс запись на ТО

2.8. Диаграмма развертывания (deployment diagram)

Этот тип диаграмм предназначен для описания аппаратной части системы, с помощью этой диаграммы производится анализ необходимой аппаратной конфигурации, на которой будут работать различные процессы системы. Описать взаимодействие их между собой и другими аппаратными устройствами. Для каждой модели такая диаграмма только одна.

При разработке диаграмм развертывания преследуются следующие цели:

- специфицировать физические узлы, необходимые для размещения на них исполнимых компонентов программной системы;
- показать физические связи между узлами реализации системы на этапе ее исполнения;
- выявить узкие места системы и реконфигурировать ее топологию для достижения требуемой производительности.

Узел (node) является элементом модели, который представляет некоторый вычислительный ресурс для развертывания на нем различных артефактов. На диаграммах развертывания узлы обычно служат для представления аппаратных устройств или исполняемых программных средств. С каждым узлом ассоциируется некоторое множество элементов, которые развертываются на нем. Развернутые на узлах элементы представляют собой артефакты.

Среда выполнения (execution environment) представляет собой узел, который обладает функциональностью, необходимой для практической реализации. Среда выполнения моделирует программно-аппаратную платформу, предоставляющую набор стандартных услуг, которые могут использоваться исполнимыми компонентами приложений.

Устройство (device) – данный инструмент позволяет создать на диаграмме объект устройства, не способного выполнять программы. Каждое такое устройство должно относиться к аппаратному обеспечению и должно иметь общее для данного вида имя.

Артефакт (artifact) представляет собой элемент модели, который специфицирует некоторую физически существующую часть информации, используемую или производимую в ходе разработки программного обеспечения или в процессе развертывания и функционирования системы. Размещаемый артефакт (deployed artifact) является абстрактным метаклассом для

представления артефакта или экземпляра артефакта, который должен быть размещен на цели развертывания.

Цель развертывания (deployment target) является абстрактным метаклассом для указания местоположения размещаемого артефакта.

Развертывание (deployment) представляет собой размещение артефакта или экземпляра артефакта на некоторой цели развертывания.

Спецификация развертывания (deployment specification) описывает множество свойств, которые определяют параметры выполнения артефакта компонента, развертываемого на некотором узле.

Наиболее общим отношением на диаграмме развертывания является отношение зависимости. На диаграммах развертывания отношение зависимости используется для представления либо отношения развертывания артефакта или его экземпляра на узле или его экземпляре, либо для отношения между артефактом и элементом модели, который его реализует. Первый случай характерен для отношения развертывания, а второй — для отношения манифестации. В качестве примера первого способа изображения можно привести фрагмент диаграммы развертывания для среды выполнения с именем server1: J2EEСервер, на которой развернуто 7 экземпляров артефактов (рис. 57)

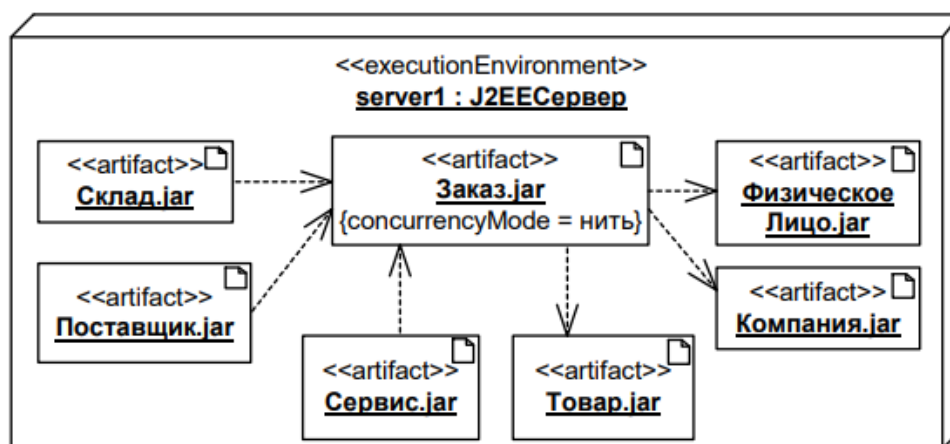


Рис. 57. Первый способ визуального представления множества экземпляров артефактов, развернутых на узле

В качестве примера второго способа изображения можно привести альтернативное представление ранее рассмотренного фрагмента диаграммы развертывания для среды выполнения с именем server1: J2EESервер.

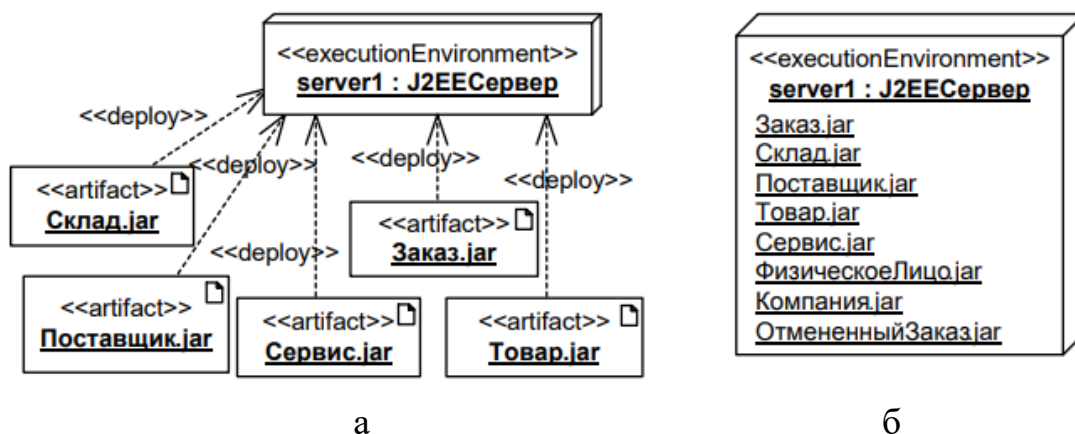


Рис. 58. Второй (а) и третий (б) способы визуального представления множества экземпляров артефактов, развернутых на узле

В качестве примера третьего способа изображения можно привести альтернативное представление ранее рассмотренного фрагмента диаграммы развертывания для среды выполнения с именем server1: J2EESервер.

Следующим отношением, которое используется на диаграммах развертывания, является манифестация.

Манифестация (manifestation) представляет собой отношение для спецификации конкретного физического воплощения одного или нескольких элементов модели посредством артефакта.

Манифестация является специализацией отношения зависимости и находится в собственности артефакта. Артефакт физически реализует один или несколько элементов модели и выступает в роли клиента зависимости. Элементы модели, которыми на диаграммах развертывания являются, как правило, компоненты и классы, выступают в качестве источников зависимости манифестации. Для отношения манифестации могут быть предложены дополнительные текстовые стереотипы с целью указать отдельные фор-

мы манифестации, например, `<tool generated>` и `<custom code>`. Эти стереотипы могут быть двумя манифестациями для различных классов, реализованными в некотором артефакте (рис .59).

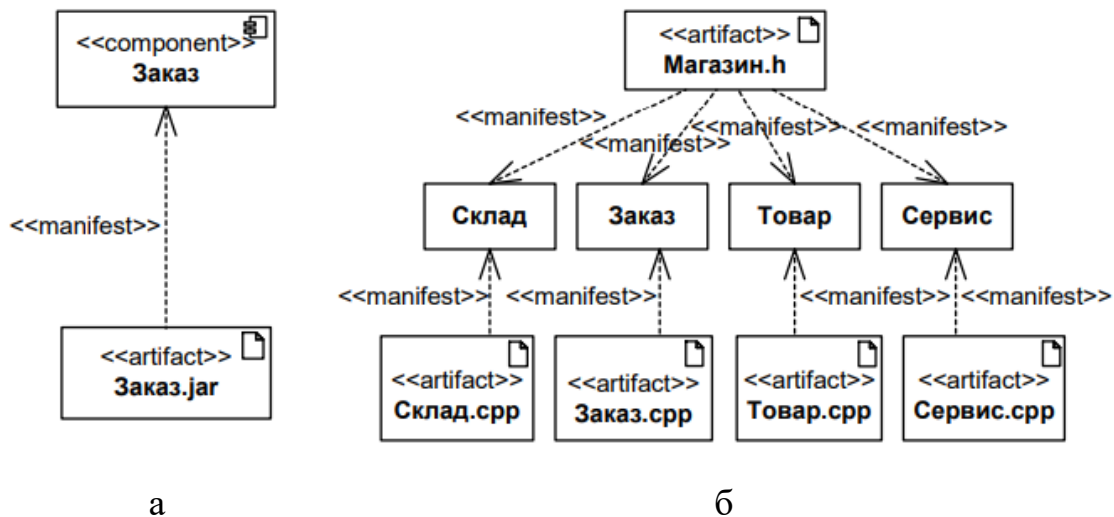


Рис. 59. Примеры изображения отношения манифестации: а — между артефактом и компонентом; б — артефактами, компонентами и классами

3. ПРИМЕРЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНЫХ РАБОТ

Пример 1. ИС «Медицинский кабинет»

В информационной системе присутствует имя, пол, дата рождения и домашний адрес каждого их пациента. Всякий раз, когда врач осматривает больного, явившегося к нему на прием, или сам приходит к нему на дом, он записывает дату и место, где проводится осмотр, симптомы, диагноз и предписания больному, проставляет имя пациента, а также свое имя. Если врач прописывает больному какое-либо лекарство, заносится название лекарства, способ его приема, словесное описание предполагаемого действия и возможных побочных эффектов информационную систему, которая позволяет (рис. 60):

- 1) по заданной дате определить количество вызовов в этот день;
- 2) позволяет определить количество заболевших данной болезнью;
- 3) по заданному лекарству определить его побочный эффект;
- 4) предоставить возможность добавления нового лекарства с описанием его свойств.

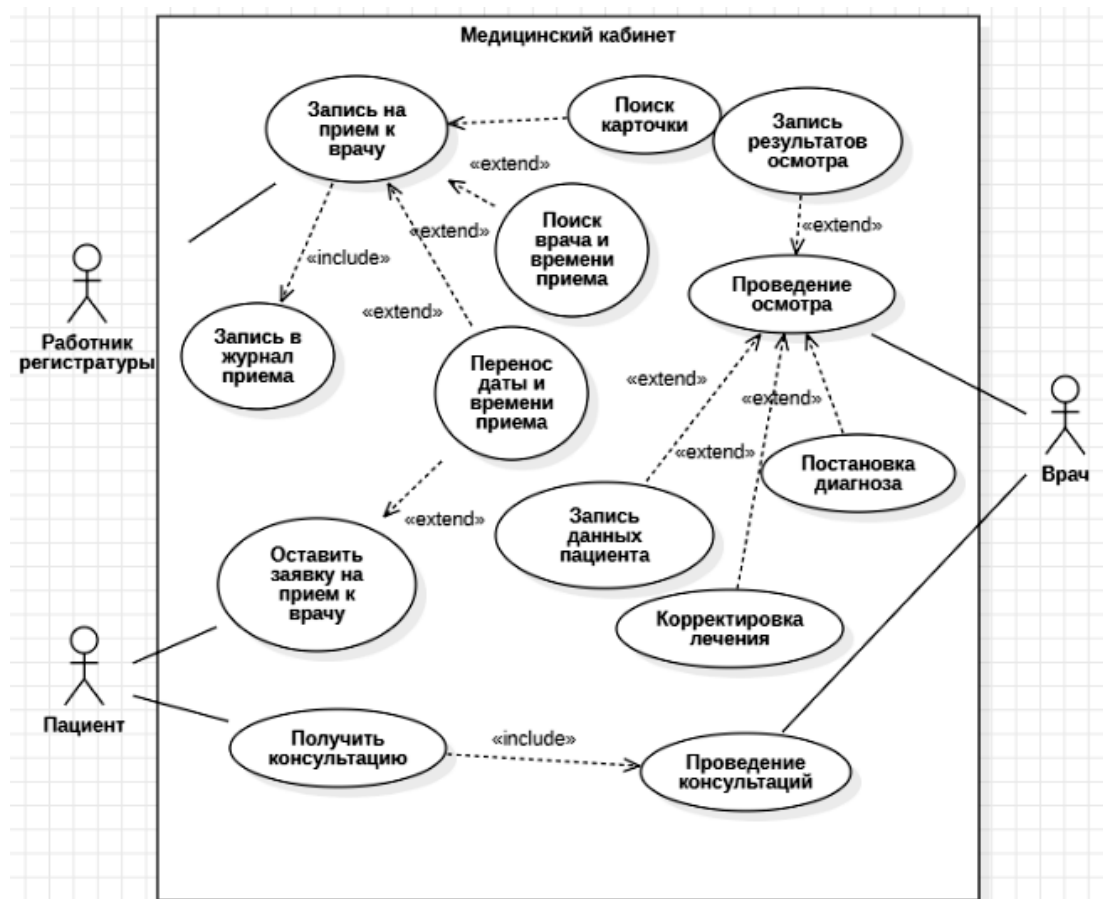


Рис. 60. Диаграмма USE case «Медицинский кабинет»

Запись на прием к врачу

- **Участники:** Пациент, Работник регистратуры, Врач
- **Предусловие:** Пациент должен быть зарегистрирован в системе.
- **Триггер:** Пациент желает записаться на прием.
- **Результат или гарантия:** Пациент записан на прием к врачу на конкретную дату и время.
- **Основной сценарий:**
 1. Пациент обращается в регистратуру или через электронную систему для записи на прием.
 2. Регистратор ищет свободное время у врача.
 3. Регистратор записывает пациента на прием.
- **Альтернативный сценарий:** Если у врача нет свободного времени, регистратор предлагает другие доступные даты или другого специалиста.

- **Расширения:** Пациент может перенести дату и время приема.

2. Поиск карточки

- **Участники:** Работник регистратуры, Пациент
- **Предусловие:** Пациент должен быть предварительно зарегистрирован в системе.
- **Триггер:** Необходимость доступа к медицинской карте пациента.
- **Результат или гарантия:** Найдена и предоставлена медицинская карта пациента.
- **Основной сценарий:**
 1. Регистратор вводит идентификационные данные пациента в систему.
 2. Система предоставляет доступ к электронной медицинской карте пациента.
- **Альтернативный сценарий:** Если карточка не найдена, регистратор может запросить у пациента дополнительные данные или создать новую карточку.
- **Расширения:** Нет.

3. Поиск врача и времени на прием

- **Участники:** Работник регистратуры, Пациент
- **Предусловие:** Пациент должен быть зарегистрирован в системе.
- **Триггер:** Запрос от пациента на запись к врачу.
- **Результат или гарантия:** Найден врач и время приема, соответствующие запросу пациента.
- **Основной сценарий:**
 1. Регистратор получает запрос от пациента.
 2. Регистратор ищет в системе доступных врачей по специализации и доступное время.
 3. Регистратор предлагает пациенту опции.

- **Альтернативный сценарий:** Если подходящие варианты отсутствуют, регистратор информирует пациента и предлагает ожидать доступности или выбрать другую специализацию.
- **Расширения:** Пациент может изменить свои предпочтения или попросить уведомить его о появлении свободного времени.

4. Запись в журнал приема

- **Участники:** Работник регистратуры
- **Предусловие:** Пациент успешно записан на прием.
- **Триггер:** Завершение процедуры записи на прием.
- **Результат или гарантия:** Информация о приеме пациента зафиксирована в журнале.
- **Основной сценарий:**
 1. Регистратор вводит данные о приеме в систему.
 2. Информация сохраняется в журнале приемов.
 - **Альтернативный сценарий:** Нет.
 - **Расширения:** Информация может быть использована для статистики и отчетности.

5. Перенос даты и времени приема

- **Участники:** Работник регистратуры, Пациент
- **Предусловие:** Пациент зарегистрирован на прием.
- **Триггер:** Запрос от пациента о переносе приема.
- **Результат или гарантия:** Прием перенесен на новую дату и время.

Основной сценарий:

1. Пациент запрашивает перенос приема.
2. Регистратор проверяет доступное время и переносит прием.
3. Регистратор уведомляет пациента о новом времени приема.

- **Альтернативный сценарий:** Если новое время недоступно, регистратор предлагает другие варианты.
- **Расширения:** Пациент может отменить прием, если не устраивают новые варианты.

6. Оставить заявку на запись к врачу

- **Участники:** Пациент, Работник регистратуры
 - **Предусловие:** Пациент желает записаться к врачу.
 - **Триггер:** Обращение пациента в регистратуру.
 - **Результат или гарантия:** Заявка на запись к врачу принята.
 - **Основной сценарий:**
 1. Пациент оставляет заявку на прием у регистратора.
 2. Регистратор регистрирует запрос в системе.
 3. Регистратор информирует пациента о статусе заявки.
- **Альтернативный сценарий:** Пациент может оставить заявку через онлайн-форму.
- **Расширения:** Заявка может быть автоматически обработана системой с предложением доступных вариантов.

7. Получить консультацию

- **Участники:** Пациент, Врач
- **Предусловие:** Пациент записан на консультацию.
- **Триггер:** Назначенное время консультации.
- **Результат или гарантия:** Пациент получил медицинскую консультацию.
- **Основной сценарий:**
 1. Врач проводит консультацию, обсуждая с пациентом его состояние и жалобы.

2. Врач предлагает рекомендации или назначает дальнейшие исследования.

- **Альтернативный сценарий:** Если требуется дополнительное мнение, врач может направить пациента к другому специалисту.
- **Расширения:** Консультация может быть проведена удаленно через видеосвязь.

8. Проведение осмотра

- **Участники:** Врач, Пациент
- **Предусловие:** Пациент записан на прием.
- **Триггер:** Назначенное время приема наступило.
- **Результат или гарантия:** Проведен медицинский осмотр, состояние пациента оценено.
- **Основной сценарий:**
 1. Врач проводит осмотр пациента, включая измерение основных показателей (давление, температура, и т.д.).
 2. Врач задает вопросы о симптомах и общем состоянии здоровья.
 3. Врач регистрирует полученные данные в системе.
- **Альтернативный сценарий:** Если врач обнаруживает признаки серьезного заболевания, он может немедленно направить пациента на дополнительные исследования.
- **Расширения:** Осмотр может быть дополнен использованием специализированного диагностического оборудования.

9. Запись данных пациента

- **Участники:** Врач, Пациент
- **Предусловие:** Пациент впервые обращается за помощью или обновляет свои данные.
- **Триггер:** Прием у врача или обновление медицинской информации.

- **Результат или гарантия:** Данные пациента актуализированы и корректно сохранены в медицинской карте.
- **Основной сценарий:**
 1. В ходе приема или консультации пациент предоставляет врачу актуальную информацию о своем здоровье, включая анамнез, текущие лекарства, аллергии и другие важные аспекты.
 2. Врач вводит или обновляет информацию в электронной медицинской карте пациента.
 3. Врач убеждается, что все данные корректно занесены и актуальны.
- **Альтернативный сценарий:** Если врач обнаруживает несоответствия или пропуски в предоставленных данных, он запрашивает у пациента дополнительную информацию или уточнения.
- **Расширения:** в случае изменения здоровья или обстоятельств пациента данные могут быть обновлены при любом последующем посещении.

10. Запись результатов осмотра

- **Участники:** Врач, Пациент
- **Предусловие:** Пациент прошел медицинский осмотр.
- **Триггер:** Завершение осмотра.
- **Результат или гарантия:** Результаты осмотра зафиксированы в медицинской карте пациента.
- **Основной сценарий:**
 1. Врач вводит результаты осмотра в электронную медицинскую карту: описание симптомов, измеренные показатели, предварительные выводы.
 2. Информация становится доступной для дальнейшего анализа и планирования лечения.

- **Альтернативный сценарий:** Если требуется срочное вмешательство, врач немедленно направляет пациента на дополнительное лечение.
- **Расширения:** Врач может добавлять фото, сканы и другие диагностические изображения.

11. Назначение лекарств и процедур

- **Участники:** Врач, Пациент
- **Предусловие:** Диагноз установлен.
- **Триггер:** Необходимость лечения или профилактики заболевания.
- **Результат или гарантия:** Пациент получил назначения лекарств и/или процедур.
- **Основной сценарий:**
 1. Врач определяет подходящее лечение на основе диагноза и состояния пациента.
 2. Врач записывает назначения в медицинскую карту и объясняет пациенту режим приема и возможные побочные действия.
- **Альтернативный сценарий:** Если пациент имеет аллергии или противопоказания, врач выбирает альтернативные средства или методы лечения.
- **Расширения:** Врач может запросить консультацию у других специалистов по поводу лечения.

12. Постановка диагноза

- **Участники:** Врач, Пациент
- **Предусловие:** Проведение осмотра и сбор необходимых медицинских данных.
- **Триггер:** Завершение осмотра пациента.
- **Результат или гарантия:** Пациенту ставится диагноз.
- **Основной сценарий:**

1. Врач анализирует результаты осмотра и проводит необходимые анализы.
 2. Врач сопоставляет симптомы с возможными заболеваниями.
 3. Врач ставит диагноз.
- **Альтернативный сценарий:** Если требуется дополнительная диагностика, врач назначает дополнительные исследования.
 - **Расширения:** Возможность изменения диагноза по результатам дополнительных исследований.

13. Получить консультацию

- **Участники:** Пациент, Врач
- **Предусловие:** Пациент записан на консультацию.
- **Триггер:** Назначенное время консультации.
- **Результат или гарантия:** Пациент получил медицинскую консультацию.
- **Основной сценарий:**
 1. Врач проводит консультацию, обсуждая с пациентом его состояние и жалобы.
 2. Врач предлагает рекомендации или назначает дальнейшие исследования.
- **Альтернативный сценарий:** Если требуется дополнительное мнение, врач может направить пациента к другому специалисту.
- **Расширения:** Консультация может быть проведена удаленно через видеосвязь.

14. Проведение консультации

- **Участники:** Врач, Пациент
- **Предусловие:** Пациент записан на консультацию.
- **Триггер:** Назначенное время консультации наступило.

- **Результат или гарантия:** Пациент проинформирован о своем здоровье и планах лечения.
- **Основной сценарий:**
 1. Врач проводит беседу с пациентом, обсуждает результаты предыдущих исследований, текущее состояние и план лечения.
 2. Врач отвечает на вопросы пациента и дает рекомендации по уходу за собой.
- **Альтернативный сценарий:** Если пациент нуждается в дополнительных услугах, врач может направить его к другим специалистам или на дополнительные процедуры.
- **Расширения:** Консультация может быть проведена в режиме онлайн через видеосвязь, если это удобно и приемлемо для пациента и врача.

Ниже представлены диаграмма классов и последовательности (рис.

61-62)

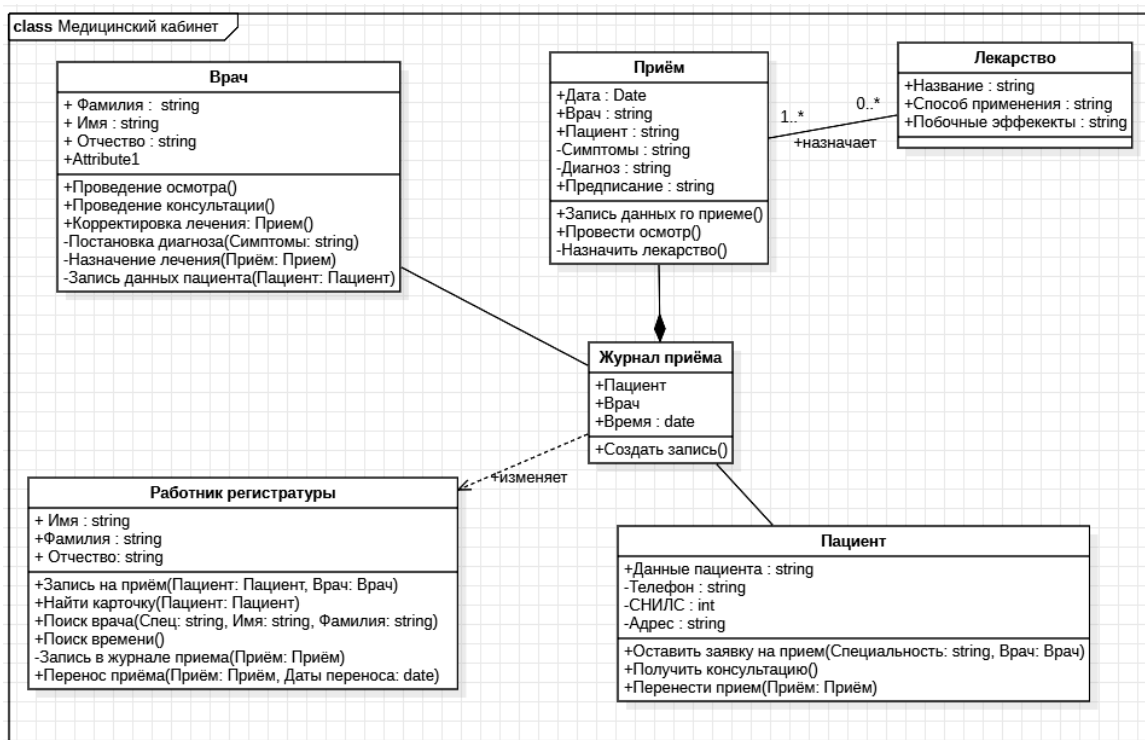


Рис. 61. Диаграмма классов ИС «Медицинский кабинет»

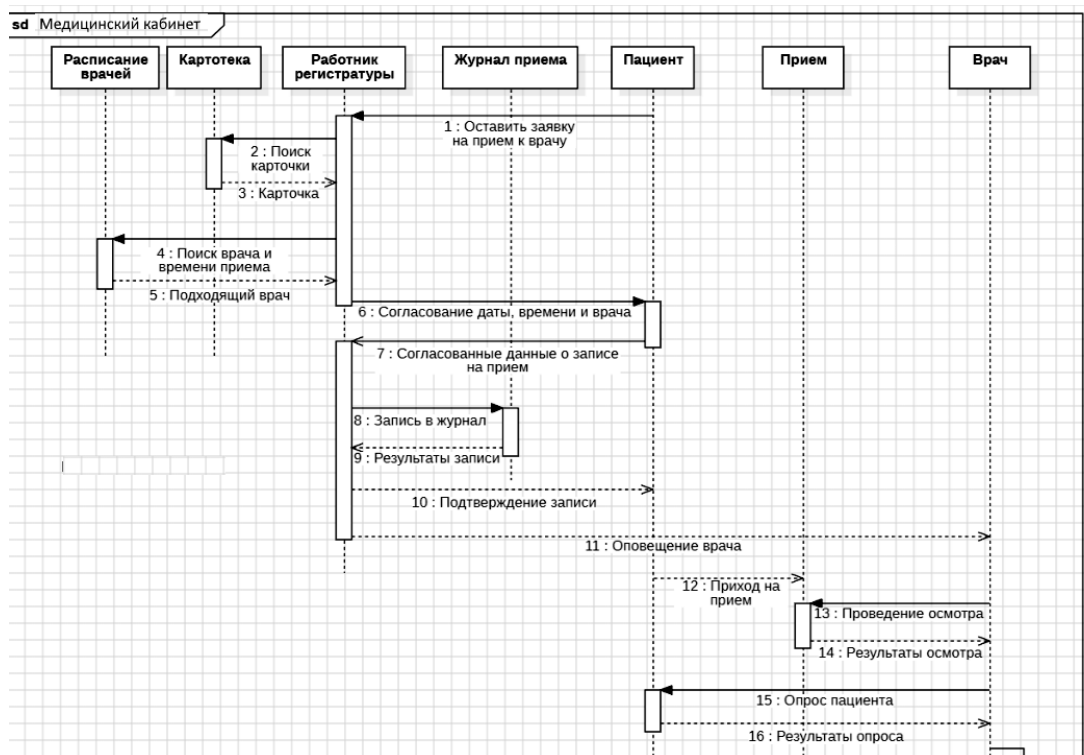
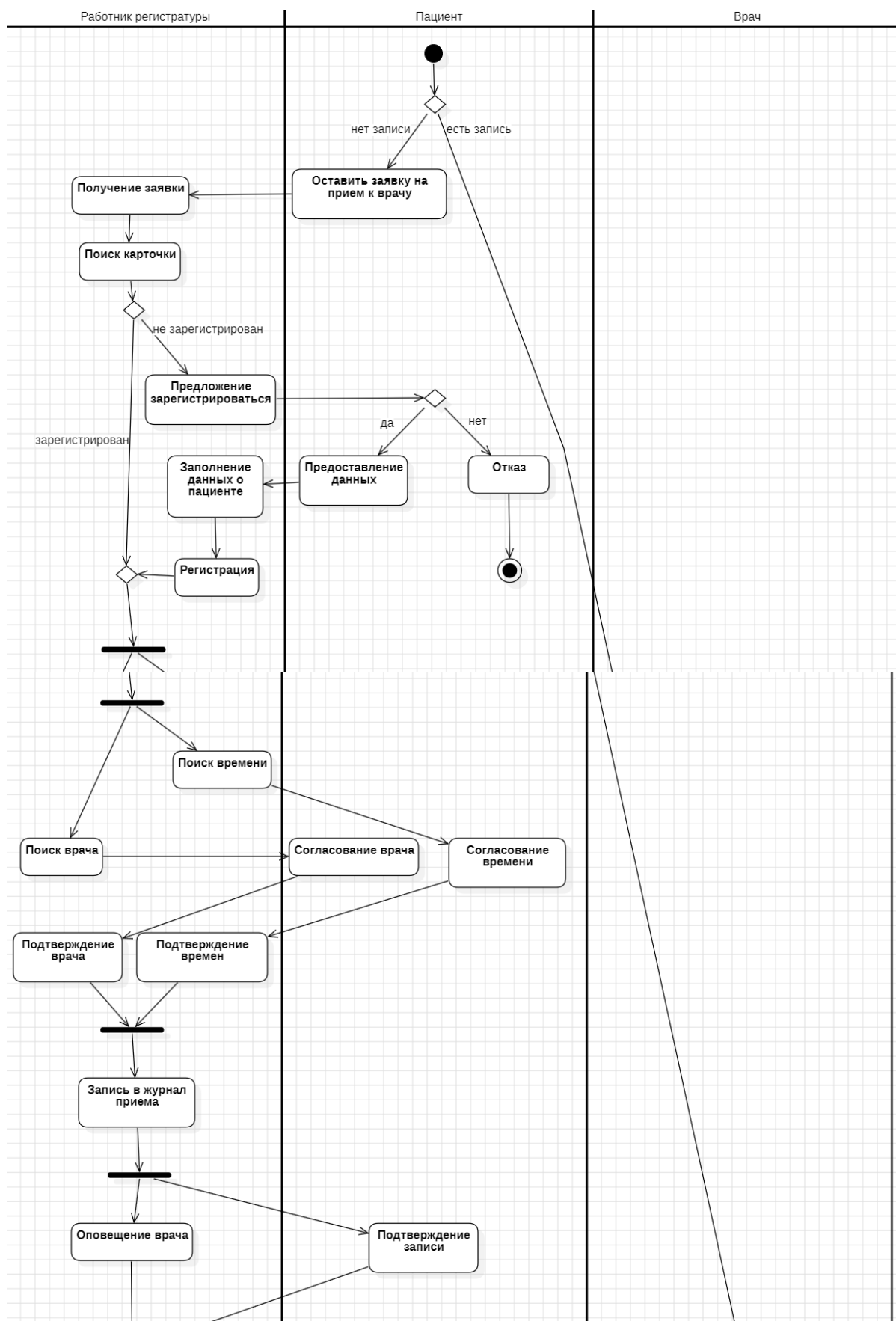
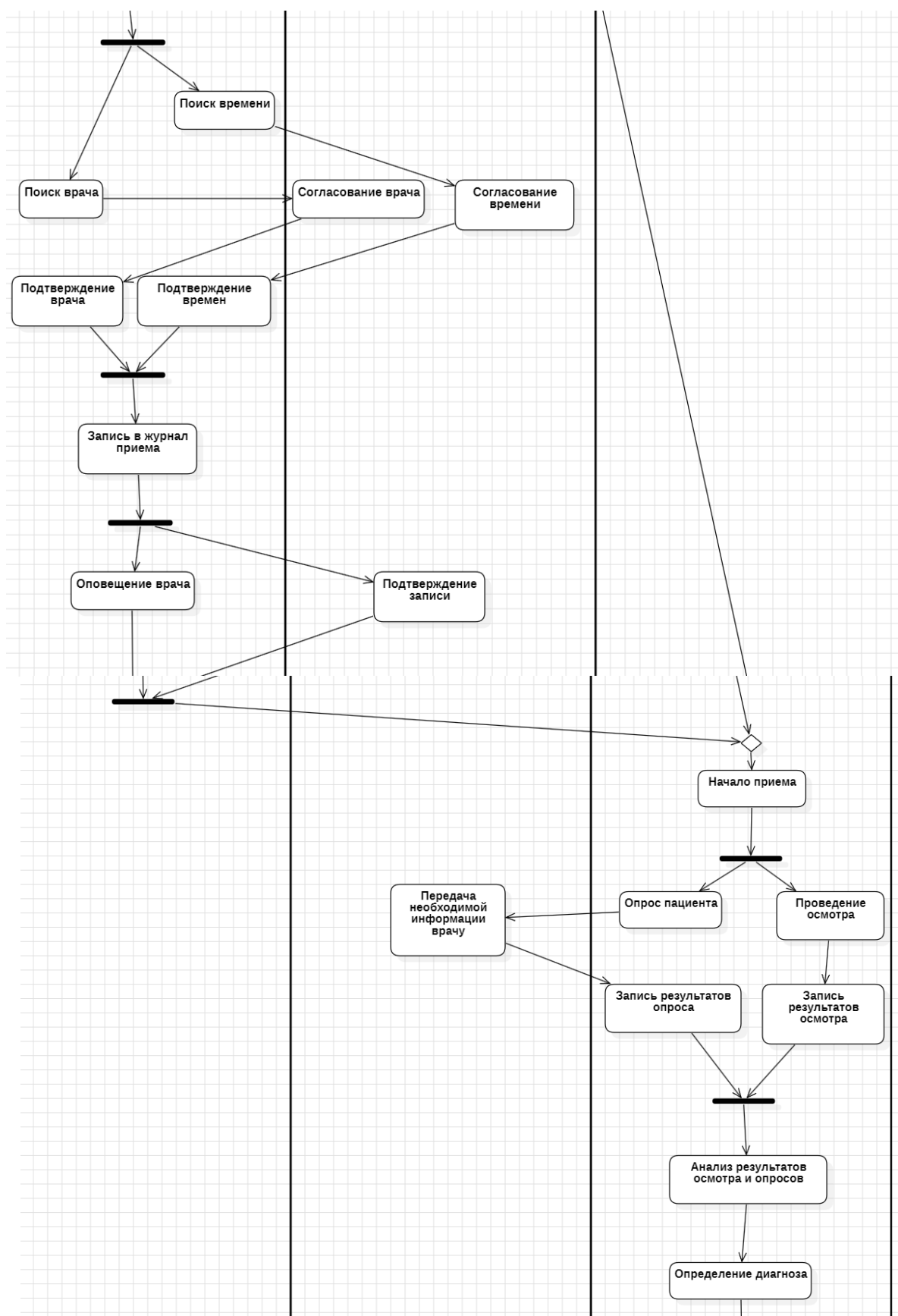


Рис. 62. Диаграмма последовательностей ИС «Медицинский кабинет»





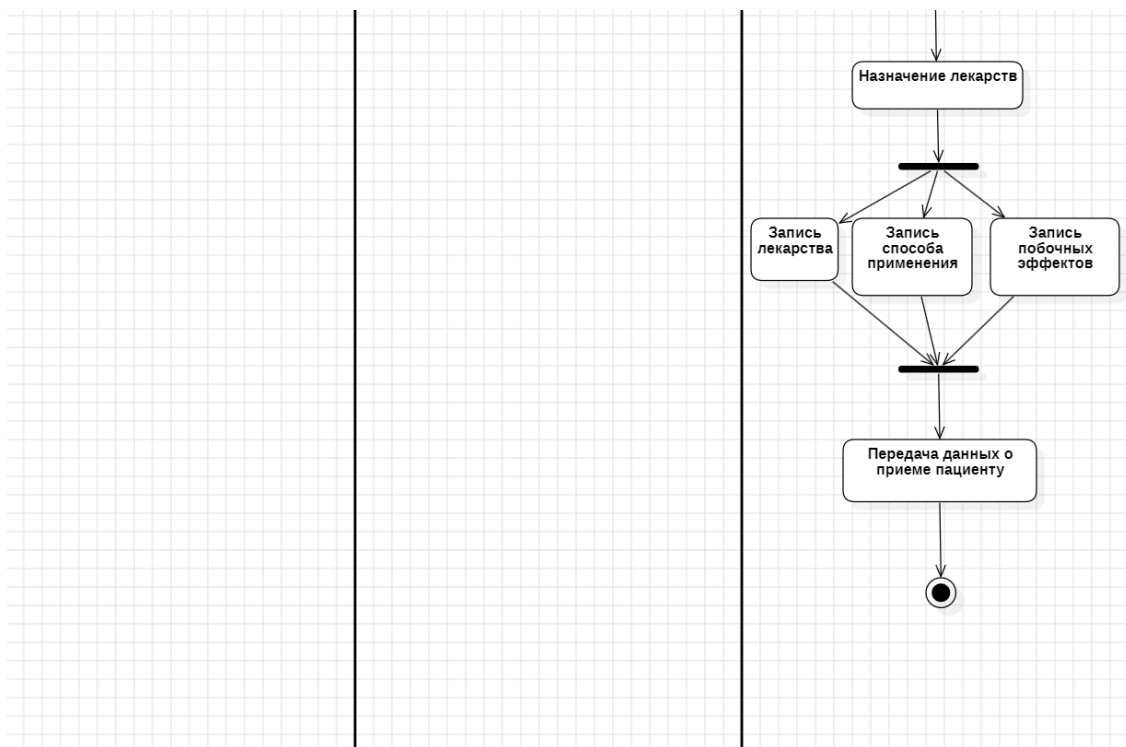


Рисунок Диаграмма деятельности(активности) ИС «Медицинский кабинет»

Также представлена диаграмма пакетов и компонентов (рис.63-64)

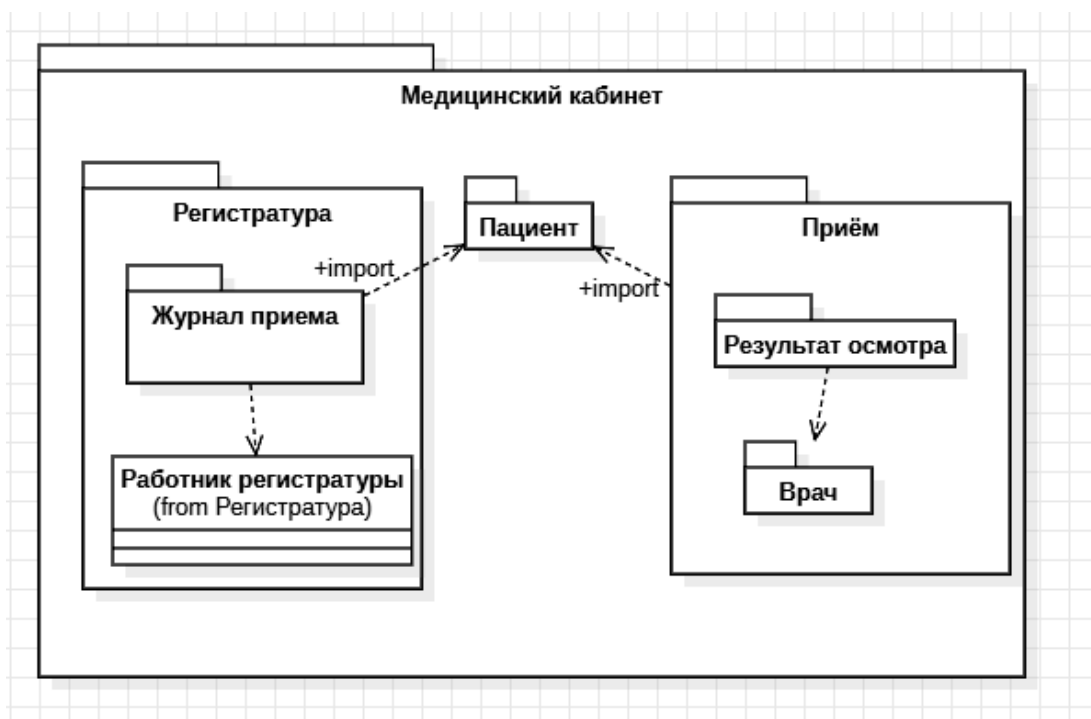


Рис. 63 Диаграмма пакетов ИС «Медицинский кабинет»

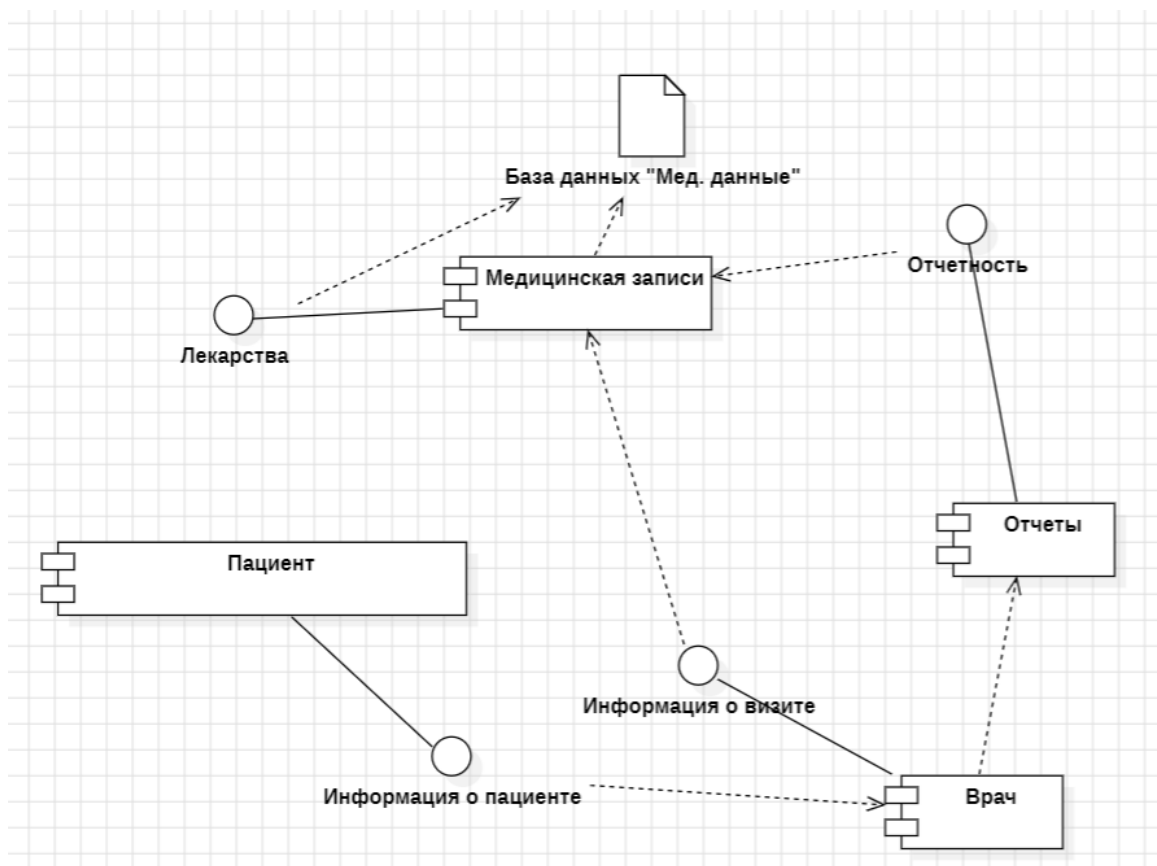


Рис. 64. Диаграмма компонент ИС «Медицинский кабинет»

Задача 4. Музыкальная коллекция.

Спроектировать информационную систему для хранения и обработки информации о музыкальных записях, а также для автоматизации управления магазином «Музыка».

Необходимо хранить информацию о характеристиках альбома: название, дата издания, стиль музыкальных произведений, число дорожек (tracks), общее время звучания в минутах, тип носителя (компакт-диск, магнитофонная лента, грампластинка).

Интерес представляет информация и о песнях, представленных в альбоме (название песни, дата написания, стиль музыки, длительность песни в минутах).

Информация об исполнителях песен должна включать следующие данные: фамилия, имя, дата рождения, место рождения, дата смерти, основной стиль музыки. Кроме того, потребуются данные о песнях исполнителя (является ли исполнитель певцом или певцом и автором песни, дата записи или исполнения).

Надо учесть, что исполнитель может входить в состав одной или нескольких музыкальных групп. В этом случае должна храниться информация о дате, когда исполнитель присоединился к данной группе и о дате, когда исполнитель покинул группу.

Информация о группах должна содержать следующее: название группы, дата основания, дата распада, основной стиль музыки. Наконец, должна хранить информацию о стилях музыки.

Ниже представленна диаграммы ИС «Магазин музыка» (рис. 65-68).

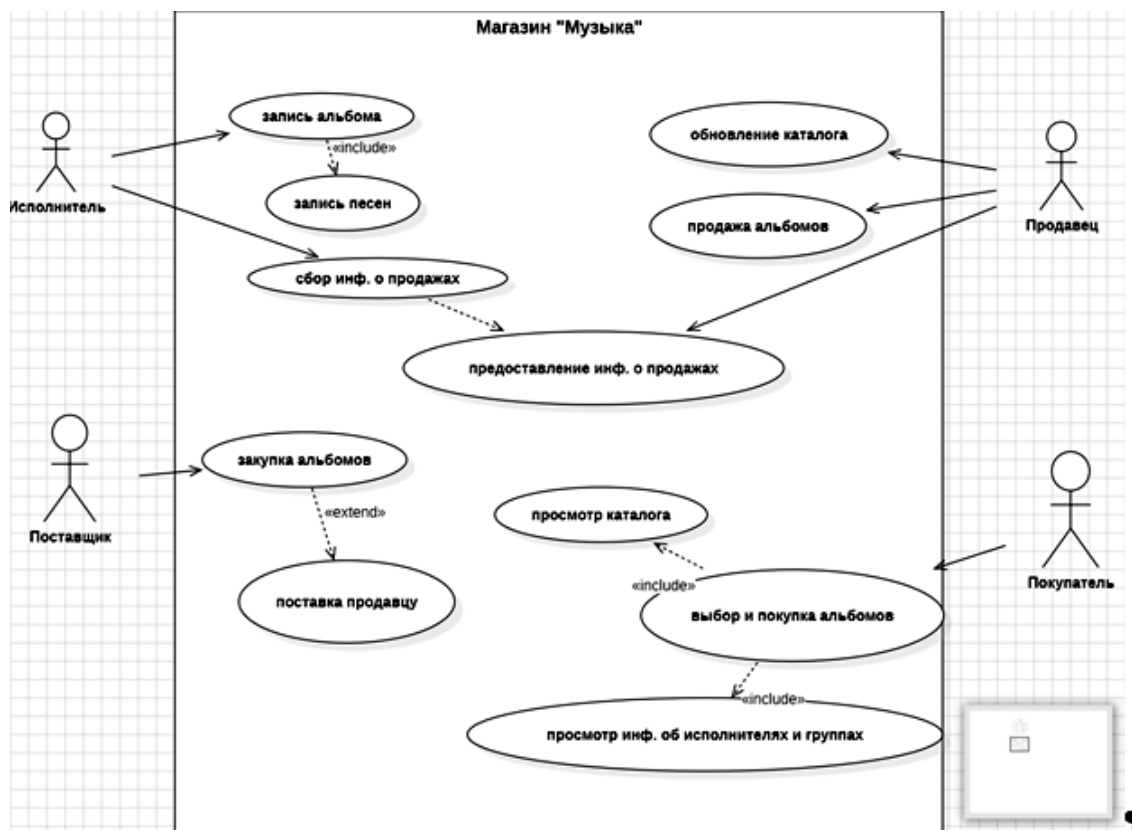


Рис. 65. Диаграмма use case ИС «Магазин музыка»

Описание прецедентов:

Заголовок	Запись альбома
Участники	Исполнитель
Предусловие	Подготовка текста, музыки, выбор продюсера и студии (возможно, некоторые песни были записаны)
Триггер	Музыканты готовы записать альбом
Цель	Записать и выпустить альбом
Основной сценарий	Альбом записан и выпущен
Альтернативный сценарий	Альбом не записан (не хватило финансирования, вдохновения, другие причины)
Расширения (ссылки)	Продать альбомы поставщику и собрать информацию о продажах

Заголовок	Выбор и покупка альбомов
Участники	Покупатель
Предусловие	Покупатель хочет приобрести но-

	вый альбом
Триггер	Вышел новый альбом, и покупатель пришел в магазин
Цель	Купить альбом
Основной сценарий	Покупатель выбрал и купил альбом
Альтернативный сценарий	Покупатель не смог выбрать или купить альбом (товар отсутствовал в магазине, не хватило средств, другие причины)
Расширения (ссылки)	Ждать выхода новых альбомов и готовиться к следующей покупке

Заголовок	Обновление каталога
Участники	Продавец
Предусловие	Были поставлены новые альбомы
Триггер	Каталог устарел и требует обновления
Цель	Обновить каталог
Основной сценарий	Каталог успешно обновлен
Альтернативный сценарий	Каталог не удалось обновить (по техническим причинам)
Расширения (ссылки)	Продать альбомы, предоставить информацию о продажах

Заголовок	Закупка альбомов
Участники	Поставщик
Предусловие	Исполнитель выпустил новый альбом
Триггер	Исполнитель хочет продать альбом
Цель	Закупить альбомы
Основной сценарий	Договор на закупку заключен, альбомы поступили поставщику
Альтернативный сценарий	Догов не был заключен (возможно, возникли разногласия между двумя сторонами договора, или другие причины)
Расширения (ссылки)	Поставка альбомов продавцу

Заголовок	Продажа альбомов
Участники	Продавец
Предусловие	Были закуплены альбомы
Триггер	Новый клиент пришёл в магазин
Цель	Продать альбомы
Основной сценарий	Альбомы успешно проданы
Альтернативный сценарий	Альбомы не проданы (не удалось заинтересовать покупателя или другие причины)
Расширения (ссылки)	Продать другие альбомы, предоставить информацию о продажах, обновить каталог

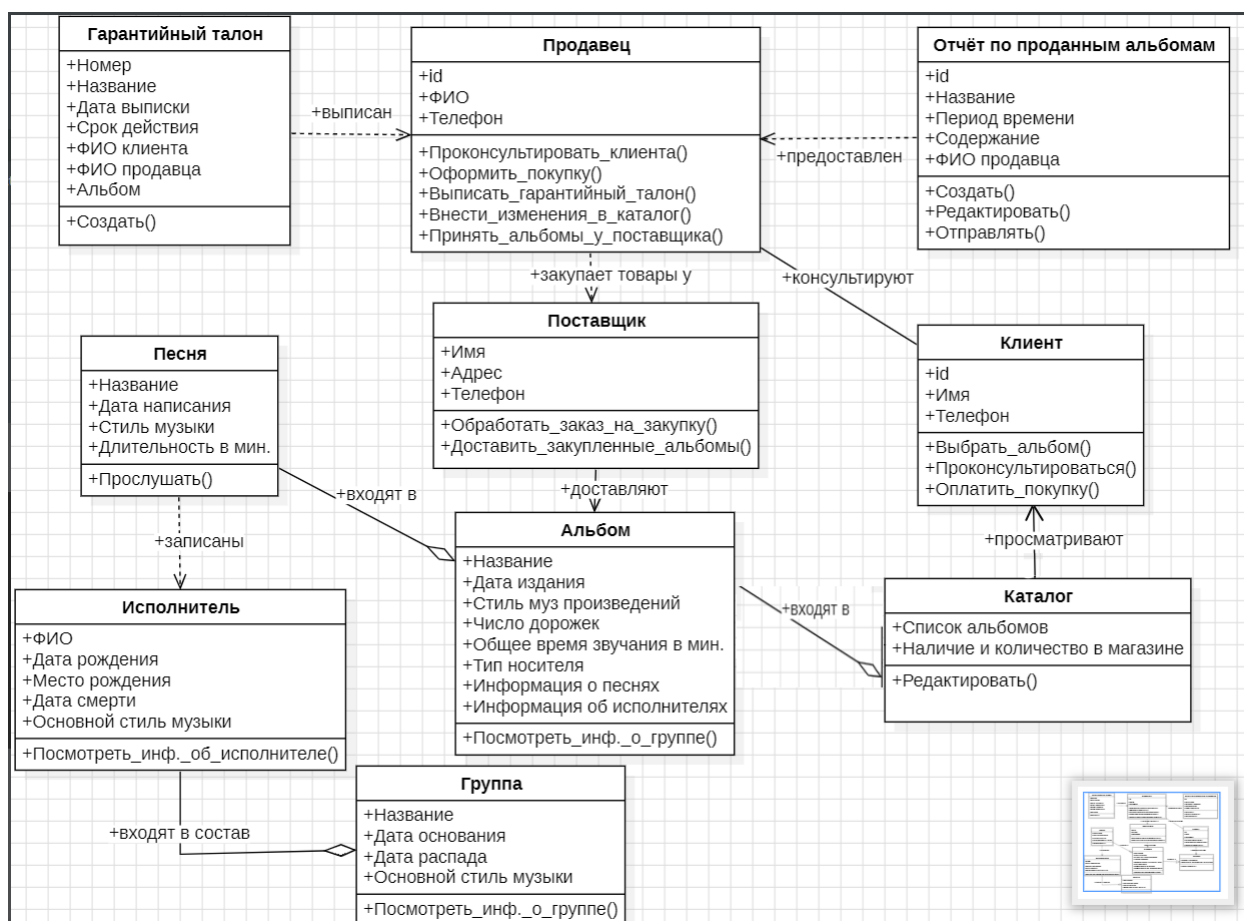


Рис. 66. Диаграмма классов ИС «Магазин музыка»

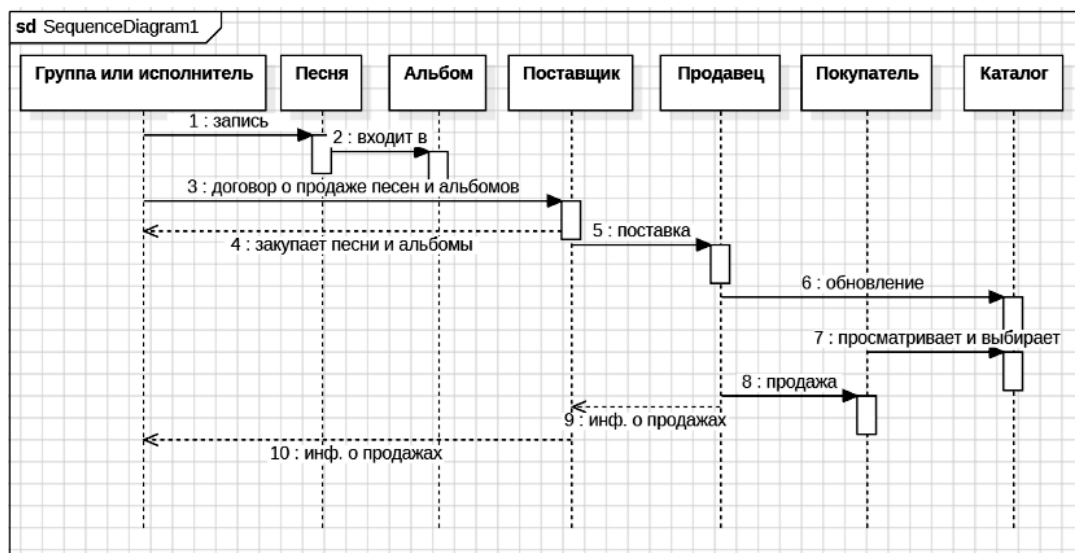


Рис. 67. Диаграмма последовательности ИС «Магазин музыка»

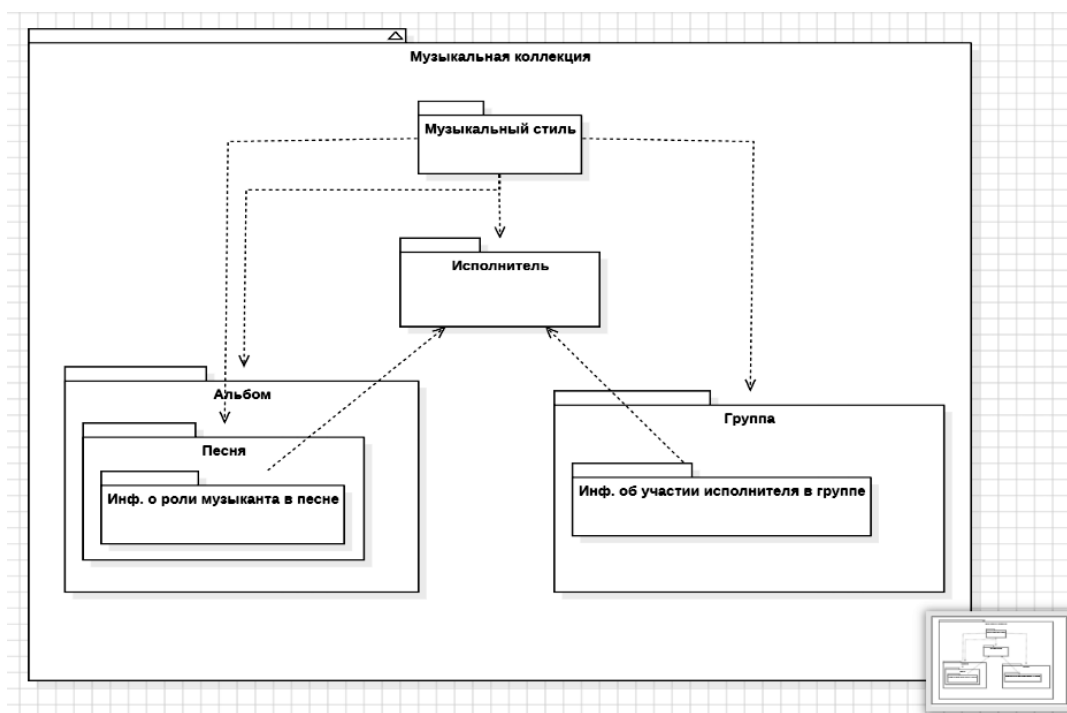


Рис. 68. Диаграмма пакетов ИС «Магазин музыка»

СПИСОК ЛИТЕРАТУРЫ

1. Буч Г. Объектно ориентированный анализ и проектирование с примерами приложений на C++ / Г. Буч. – М.: Бином; СПб: Невский диа лект, 1999. – 560 с.
2. Капустина С.В. Проектирование информационных систем / С.В. Капустин. – Красноярск, 2008. – 252 с.
3. Леоненков А.В. Самоучитель UML 2./А.В. Леоненков — СПб.: БХВ-Петербург, 2007. — 576 с.: ISBN 978-5-94157-878-8
4. Орлов С.А. Технологии разработки программного обеспечения : учеб. / С.А. Орлов. – СПб. : Питер, 2002. – 464 с.
5. Рамбо Дж. UML : спец. справ. / Дж. Рамбо, А. Якобсон, Г. Буч. – СПб . : Питер, 2002. – 656 с.
6. «Введение в модельно-ориентированную системную и про граммную инженерию (MBSSE)" : учебник / В.А. Сухомлин, В.Ю. Романов, Д.А. Гапанович». – Москва : Фонд «Лига интернет-медиа»; МАКС Пресс, 2024. – 672 с. ISBN 978-5-317-07289-6
7. Трофимов С.А. CASE – технологии: практическая работа в Rational Rose / С.А. Трофимов. – М. : Бином-Пресс, 2002. – 288 с.

ПРИЛОЖЕНИЕ

Задачи для выполнения лабораторных работ

Задача 1. *Проектирование информационно-вычислительной системы учебного заведения*

В объектно-ориентированной системе должны присутствовать серверы ИВС (сервер основной, сервер баз данных, сервер приложений, сервер электронной почты, сервер сетевой безопасности). Серверы должны быть описаны по основным характеристикам: назначение, производительность, потребляемая мощность, наработка на отказ и т.д. информационную систему, которая позволит выполнять следующие действия:

1. Для каждой конфигурации показывать суммарную производительность ИВС и предполагаемую наработку на отказ.

1. Предоставить возможность добавления новой техники.
2. Предоставить возможность изменения конфигурации и параметров серверов.
3. Показать список рабочих станций
4. Показать конфигурацию учебных компьютерных классов
5. Показать общее количество учебных компьютеров (компьютерные классы без серверов)
6. Показать количество компьютеров в управленческой сети.

Необходимо разработать:

Class Diagram (диаграмма классов)

Диаграмма классов показывает классы и их отношения, тем самым, представляя логический аспект проекта.

В данной задаче можно выделить 4 класса:

- 1) научно-исследовательская деятельность вуза (научная сеть);
- 2) учебные компьютерные классы (учебная сеть);
- 3) управление вузом (управленческая сеть);
- 4) управление информацией и телекоммуникацией (информационная сеть).

Use Case Diagram (диаграмма сценариев)

Диаграмма сценариев определяет поведение системы относительно пользователя. В диаграмме выделяют актеров и use case элементы – действия.

В данной задаче можно выделить актеров:

- 1) серверы;
- 2) рабочие станции;
- 3) аппаратные средства телекоммуникаций.

Use case элементы:

- 1) место сервера в сети;
- 2) требования к аппаратуре ИВС;
- 3) изменение параметров ИВС;
- 4) показать список серверов, участвующих в решении определенной задачи;
- 5) показать список рабочих станций или пользовательских компьютеров, участвующих в решении определенной задачи.

Statechart Diagram (диаграмма состояний)

Диаграмма состояний показывает пространство состояний данного класса, события, которые влекут переход из одного состояния в другое, действия, которые происходят при изменении состояния.

Диаграмма содержит стартовое состояние – нет информации о серверах, рабочих станциях и требуемых параметрах ИВС. После действий «внесение данных о серверах и рабочих станциях» и «внесение информации о сервере» происходит изменение состояния. Также состояние «Внесены данные о сервере» переходит в состояние «Создана сеть». В состоянии «Создана» можно добавлять новое оборудование в сеть. А в состоянии «Внесены данные о параметрах ИВС» можно изменять данные о требуемых возможностях. Дальше происходит переход в состояние «Спроектирована ИВС» и «Создан список сетей». В состоянии «Создан список сетей» можно выполнять 5 действий. И последнее – конечное состояние.

Activity Diagram (диаграмма активности)

Диаграмма активности – это разновидность диаграммы состояний, отражающая завершение одних и начало других действий.

Диаграмма делится на части – независимые действия.

Диаграмма начинается с внесения данных о требуемых параметрах ИВС и серверах. Эти два действия независимы. Есть возможность осуществлять запрос об изменении параметров ИВС. В этом случае принимается решение изменять или не изменять данные в зависимости от требуемых параметров. Далее регистрируется следующая конфигурация ИВС и создается список всех сетей.

Sequence Diagram (диаграмма последовательностей действий)

Диаграмма последовательности используется, чтобы проследить выполнение сценария, порядок обмена сообщениями между объектами.

Создается 4 объекта. Сначала происходит определение параметров ИВС и серверов. Пока ИВС не спроектирована, можно изменить ее параметры.

Следующее действие во времени – формирование сетей. Последнее действие – определение места сервера в ИВС.

Component Diagram (диаграмма модулей)

Диаграмма модулей показывает распределение классов и объектов по модулям в физическом проектировании системы. Диаграмма состоит из модулей и их зависимостей. Выделяют различные типы модулей. Связи отображают зависимость между соответствующими директивами #include модулей. Связи представляются стрелками, выходящими из зависимого модуля.

Deployment Diagram (диаграмма топологий)

Диаграмма процессов используется, чтобы показать распределение процессов по серверам в физическом проекте системы. Диаграмма показывает физическую совокупность серверов и рабочих станций, обеспечивающих работу системы.

В диаграмме можно выделить один сервер – компьютер, при помощи которого выполняется основная программа. У процессора есть несколько процессов – занесение данных, создание списка сетей и т. д.

Задача 2. Скачки

В информационной системе клуба любителей скачек должна быть представлена информация об участвующих в скачках лошадях (кличка, пол, возраст), их владельцах (имя, адрес, телефон) и жокеях (имя, адрес, возраст, рейтинг). Необходимо иметь данные по каждому состязанию: дата, время и место проведения скачек (ипподром), название состязаний (если таковое имеется), клички, участвующих в заездах лошадей и имена жокеев, занятые ими места и показанное в заезде время.

Спроектировать эту систему и:

- 1) показывать натуральное число, отражающее количество лошадей, имеющих у заданного владельца;
- 2) выводить список кличек лошадей, имеющих у заданного владельца;

3) выводить информацию о владельце (имя, адрес, телефон) по заданной кличке лошади;

4) выводить информацию о забеге (клички участвующих лошадей, имена жокеев, занятые места и показанное в забеге время) по заданной дате забега;

5) показывать лидеров всех забегов;

6) осуществлять ввод данных для нового забега.

Задача 3. Учет успеваемости студентов

В информационной системе должны присутствовать данные о контингенте студентов (фамилия, имя, отчество, год поступления, форма обучения (дневная/вечерняя/заочная), номер или название группы; об учебном плане (название специальности, дисциплина, семестр, количество отводимых на дисциплину часов, форма отчетности (экзамен/зачет)); о журнале успеваемости студентов (год/семестр, студент, дисциплина, оценка). Спроектировать информационную систему, которая позволяет:

1. Для указанной формы обучения посчитать количество студентов этой формы обучения.

2. Для указанной дисциплины получить количество часов и формы отчетности по этой дисциплине.

3. Предоставить возможность добавления и изменения информации о студентах, об учебных планах, о журнале.

Предоставить возможность добавления и изменения информации о журнале успеваемости.

Задача 4. Медицинский кооператив

В информационной системе присутствует имя, пол, дата рождения и домашний адрес каждого их пациента. Всякий раз, когда врач осматривает больного, явившегося к нему на прием, или сам приходит к нему на дом, он записывает дату и место, где проводится осмотр, симптомы, диагноз и предписания больному, проставляет имя пациента, а также свое имя. Если врач прописывает больному какое-либо лекарство, заносится название лекарства, способ его приема, словесное описание предполагаемого действия и возможных побочных эффектов. Спроектировать систему, которая позволяет:

1) по заданной дате определить количество вызовов в этот день;

2) позволяет определить количество заболевших данной болезнью;

3) по заданному лекарству определить его побочный эффект;

4) предоставить возможность добавления нового лекарства с описанием его свойств.

Задача 5. Городская дума

В информационной системе присутствуют имена, адреса, домашние и служебные телефоны всех членов думы. В думе работает порядка сорока комиссий, все участники которых являются членами думы. Каждая комиссия имеет свой профиль, например, вопросы образования, проблемы, связанные с жильем, и так далее. Данные по каждой из комиссий включают ее нынешний состав и председателя, прежних председателей и членов этой комиссии, участвовавших в ее работе за прошедшие 10 лет, даты включения и выхода из состава комиссии, избрания ее председателей. Члены думы могут заседать в нескольких комиссиях. Имеется время и место проведения каждого заседания комиссии с указанием депутатов и служащих думы, которые участвуют в его организации. Спроектировать систему, которая позволяет:

- 1) показать список комиссий, для каждой – ее состав и председателя;
- 2) предоставить возможность добавления нового члена комиссии;
- 3) показать список членов муниципалитета, для каждого из них – список комиссий, в которых он участвовал и/или был председателем;
- 4) предоставить возможность добавления новой комиссии с указанием председателя;
- 5) для указанного интервала дат и комиссии выдать список ее членов с указанием количества пропущенных заседаний;
- 6) предоставить возможность добавления нового заседания с указанием присутствующих;
- 7) по каждой комиссии показать количество проведенных заседаний в указанный период времени.

Задача 6. Рыболовная фирма

Фирме принадлежит небольшая флотилия рыболовных катеров. Каждый катер имеет «паспорт», куда занесены его название, тип, водоизмещение и дата постройки. Фирма регистрирует каждый выход на лов, записывая название катера, имена и адреса членов команды с указанием их должностей (капитан, боцман и т. д.), даты выхода и возвращения, а также вес пойманной рыбы отдельно по сортам (например, трески). За время одного рейса катер может посетить несколько банок. Фиксируется дата прихода на каждую банку и дата отплытия, качество выловленной рыбы (отличное, хорошее, плохое). На борту улов не взвешивается. Спроектировать информационную систему, которая позволяет:

- 1) для каждого катера вывести даты выхода в море с указанием улова;
- 2) предоставить возможность добавления выхода катера в море с указанием команды;
- 3) для указанного интервала дат вывести для каждого сорта рыбы список катеров с наибольшим уловом;
- 4) для указанного интервала дат вывести список банок, с указанием среднего улова за этот период;
- 5) предоставить возможность добавления новой банки с указанием данных о ней;
- 6) для заданной банки вывести список катеров, которые получили улов выше среднего;
- 7) вывести список сортов рыбы и для каждого сорта список рейсов с указанием даты выхода и возвращения, количества улова;
- 8) для выбранного пользователем рейса и банки добавить данные о сорте и количестве пойманной рыбы;
- 9) предоставить возможность пользователю изменять характеристики выбранного катера;
- 10) предоставить возможность добавления нового катера;
- 11) для указанного сорта рыбы и банки вывести список рейсов с указанием количества пойманной рыбы.

Задача 7. Фирма, проводящая аукционы

Фирма занимается продажей с аукциона антикварных изделий и произведений искусства. Владельцы вещей, выставяемых на проводимых фирмой аукционах, юриди-

чески являются продавцами. Лица, приобретающие эти вещи, именуются покупателями. Получив от продавцов партию предметов, фирма решает, на котором из аукционов выгоднее представить конкретный предмет. Перед проведением очередного аукциона каждой из выставляемых на нем вещей присваивается отдельный номер лота, играющий ту же роль, что и введенный ранее шифр товара. Две вещи, продаваемые на различных аукционах, могут иметь одинаковые номера лотов.

В книгах фирмы делается запись о каждом аукционе. Там отмечаются дата, место и время его проведения, а также специфика (например, выставляются картины, написанные маслом и не ранее 1900 г.). Заносятся также сведения о каждом продаваемом предмете: аукцион, на который он заявлен, номер лота, продавец, отправная цена и краткое словесное описание. Продавцу разрешается выставлять любое количество вещей, а покупатель имеет право приобретать любое количество вещей. Одно и то же лицо или фирма может выступать и как продавец, и как покупатель. После аукциона служащие фирмы, проводящей аукционы, записывают фактическую цену, уплаченную за проданный предмет, и фиксируют данные покупателя.

Спроектировать информационную систему, позволяющий осуществить следующие операции:

- 1) для указанного интервала дат вывести список аукционов с указанием наименования, даты и места проведения;
- 2) добавить на указанный пользователем аукцион на продажу предмет искусства с указанием начальной цены;
- 3) вывести список аукционов с указанием суммарного дохода от продажи, отсортированных по доходу;
- 4) для указанного интервала дат, вывести список предметов, которые были проданы на аукционах в этот период времени;
- 5) предоставить возможность добавления факта продажи на указанном аукционе заданного предмета;
- 6) для указанного интервала дат вывести список продавцов с указанием общей суммы, полученной от продажи предметов в этот промежуток времени;
- 7) вывести список покупателей, которые сделали приобретения в указанный интервал дат с указанием количества приобретенных предметов;

8) предоставить возможность добавления записи о проводимом аукционе (место, время);

9) для указанного места, вывести список аукционов;

10) для указанного интервала дат вывести список продавцов, которые принимали участие в аукционах, проводимых в этот период времени;

11) предоставить возможность добавления и изменения информации о продавцах и покупателях.

Задача 8. Библиотека

Разработать информационную систему обслуживания библиотеки, которая содержит следующую информацию: названия книг, ФИО авторов, наименования издательств, год издания, количество страниц, количество иллюстраций, стоимость, название филиала библиотеки или книгохранилища, в которых находится книга, количество имеющихся в библиотеке экземпляров конкретной книги, количество студентов, которым выдавалась конкретная книга, названия факультетов, в учебном процессе которых используется указанная книга. Спроектировать информационную систему, которая позволяет:

1) для указанного филиала посчитать количество экземпляров указанной книги, находящихся в нем;

2) для указанной книги посчитать количество факультетов, на которых она используется в данном филиале, и вывести названия этих факультетов;

3) предоставить возможность добавления и изменения информации о книгах в библиотеке;

4) предоставить возможность добавления и изменения информации о филиалах.

Задача 9 *Учет аудиторного фонда университета*

В объектно ориентированной системе должна присутствовать информация об аудиторном фонде университета: наименование корпуса, в котором расположено помещение, номер комнаты, расположение комнаты в корпусе, ширина и длина комнаты в метрах, назначение и вид помещения, подразделение университета, за которым закреплено помещение. Также должна быть информация о высоте потолков в помещениях в зависимости от места расположения помещений в корпусе. Следует также учитывать, что структура подразделений университета имеет иерархический вид, когда одни подразделения входят в состав других (факультет, кафедра, лаборатория, ...). Спроектировать информационную систему, которая позволяет:

- 1) рассчитать данные о площадях и объемах каждого помещения;
- 2) для указанного корпуса получить количество факультетов, находящихся в этом корпусе, их названия и структуру;
- 3) предоставить возможность добавления и изменения информации о корпусах в университете;
- 4) предоставить возможность добавления и изменения информации о комнатах в корпусах университета.

Задача 10. *Регистрация происшествий*

В информационной системе должны присутствовать данные, необходимые для регистрации сообщений о происшествиях (регистрационный номер сообщения, дата регистрации, краткая фабула (тип происшествия)); информация о принятом по происшествию решении (отказано в возбуждении дел, удовлетворено ходатайство о возбуждении уголовного дела с указанием регистрационный номера заведенного дела, отправлено по территориальному признаку); информация о лицах, виновных или подозреваемых в совершении происшествия (регистрационный номер лица, фамилия, имя, отчество, адрес, количество судимостей), отношение конкретных лиц к конкретным происшествиям (виновник, потерпевший, подозреваемый, свидетель, ...).

Спроектировать эту систему, которая позволяет:

- 1) рассчитать количество происшествий в определенный параметром промежуток времени;
- 2) для указанного лица определить количество происшествий, в которых он зарегистрирован;

3) предоставить возможность добавления и изменения информации о происшествии;

4) предоставить возможность добавления и изменения информации о лицах, зарегистрированных в участии в происшествиях.

Задача 11. Обслуживание работы конференции

Информационная система должна содержать справочник персоналий участников конференции (фамилия, имя, отчество, ученая степень, ученое звание, научное направление, место работы, кафедра (отдел), должность, страна, город, почтовый индекс, адрес, рабочий телефон, домашний телефон, e-mail) и информацию, связанную с участием в конференции (докладчик или участник, дата рассылки 1-го приглашения, дата поступления заявки, тема доклада, отметка о поступлении тезисов, дата рассылки 2-го приглашения, дата поступления оргвзноса, размер поступившего оргвзноса, дата приезда, дата отъезда, потребность в гостинице).

Спроектировать эту систему, которая позволяет:

1) для указанной даты 1-й рассылки вывести список приглашенных и посчитать их количество;

2) предоставить возможность добавления приглашенных на конференцию с указанием оргвзноса и даты его уплаты;

3) вывести список приглашенных с указанием даты об уплате оргвзноса;

4) для указанного интервала дат вывести список участников, уплативших оргвзнос в этом диапазоне;

5) для указанного города вывести название тезисов докладов, поступивших из этого города;

6) для указанного города вывести список нуждающихся в гостинице.

Задача 12. Обслуживание склада

Информационная система должны содержать следующие данные:

- информацию об «единицах хранения» – номер ордера, дата, код поставщика, балансный счет, код сопроводительного документа по справочнику документов, номер сопроводительного документа, код материала по справочнику материалов, счет материала, код единицы измерения, количество пришедшего материала, цена единицы измерения);

- информацию о хранящихся на складе материалах (справочник материалов – код класса материала, код группы материала, наименование материала);

- информацию о единицах измерения конкретных видов материалов – код материала, единица измерения (метры, килограммы, литры и т. д.);

- информацию о поставщиках материалов – код поставщика, его наименование, ИНН, юридический адрес (индекс, город, улица, дом), адрес банка (индекс, город, улица, дом), номер банковского счета.

Спроектировать информационную систему, которая позволяет:

- 1) посчитать количество поставщиков данного материала;
- 2) предоставить возможность добавления единицы хранения с указанием всех реквизитов;
- 3) вывести список поставщиков с указанием всех реквизитов данного материала на склад;
- 4) для указанного адреса банка посчитать количество поставщиков склада, пользующихся услугами этого банка.

Задача 13. Фирма

Фирма отказалась от приобретения некоторых товаров у своих поставщиков, решив самостоятельно наладить их производство. С этой целью она организовала сеть специализированных цехов, каждый из которых принимает определенное участие в технологическом процессе.

Каждому виду выпускаемой продукции присваивается, как обычно, свой шифр товара, под которым он значится в файле товарных запасов. Этот же номер служит и шифром продукта. Для шифра указывается, когда была изготовлена последняя партия этого продукта, какова ее стоимость, сколько операций потребовалось.

Операцией считается законченная часть процесса производства, которая целиком выполняется силами одного цеха в соответствии с техническими требованиями, перечисленными на отдельном чертеже. Для каждого продукта и для каждой операции есть описание операции, ее средняя продолжительность и номер, по которому можно отыскать требуемый чертеж. Кроме того, указывается номер цеха, обычно производящего данную операцию. Для конкретной операции есть потребные количества расходуемых материалов, а также присвоенные им шифры товара. Расходуемыми называют такие материалы, как, например, электрический кабель, который нельзя использовать повторно. Когда, готовясь к выполнению операции, расходуемый материал забирают со склада, регистрируется фактически выданное количество, соответствующий шифр товара, номер служащего, ответственного за выдачу, дата и время выдачи, номер операции и номер наряда на проведение работ, который будет обсуждаться ниже.

Реально затраченное количество материала может не совпадать с расчетным, например, из-за того, что часть изготовленной продукции бракуется. Каждый из цехов располагает многочисленными инструментами и приспособлениями. При выполнении некоторых операций их все же не хватает, и цех вынужден обращаться в центральную инструментальную за недостающими. Каждый тип инструмента снабжен отдельным номером, и на него заведена запись со словесным описанием. Кроме того, там отмечено, какое количество инструментов этого типа выделено цехам и какое осталось в инструментальной. Экземпляры инструмента конкретного типа, например, гаечные ключи одного размера, различаются по своим индивидуальным номерам. На фирме для каждого типа инструмента имеется запись, содержащая перечень всех индивидуальных номеров. Кроме того, указаны даты их поступления на склад.

Когда, готовясь к выполнению операции, расходуемый материал забирают со склада, регистрируется фактически выданное количество, соответствующий шифр товара, номер служащего, ответственного за выдачу, дата и время выдачи, номер операции и номер наряда на проведение работ, который будет обсуждаться ниже. Реально затраченное количество материала может не совпадать с расчетным, например, из-за того, что часть изготовленной продукции бракуется. Каждый из цехов располагает многочисленными инструментами и приспособлениями. При выполнении некоторых операций их все же не хватает, и цех вынужден обращаться в центральную инструментальную за недостающими. Каждый тип инструмента снабжен отдельным номером, и на него заведена запись со словесным описанием. Кроме того, там отмечено, какое количество инструментов этого типа выделено цехам и какое осталось в инструментальной. Экземпляры инструмента конкретного типа, например, гаечные ключи одного размера, различаются по своим индивидуальным номерам. На фирме для каждого типа инструмента имеется запись, содержащая перечень всех индивидуальных номеров. Кроме того, указаны даты их поступления на склад.

По каждой операции в фирме отмечают типы и количество инструментов этих типов, которые должны использоваться при ее выполнении. Когда инструменты действительно берутся со склада, фиксируется индивидуальный номер каждого экземпляра, указываются номер заказавшего их цеха и номер наряда на проведение работ. И в этом случае затребованное количество не всегда совпадает с заказанным.

Наряд на проведение работ по форме напоминает заказ на приобретение товаров, но, в отличие от последнего, он направляется не поставщику, а в один из цехов. Оформляется этот наряд после того, как руководство фирмы сочтет необходимым выпустить партию некоторого продукта. В наряд заносятся шифр продукта, дата оформления наряда, срок, к которому должен быть выполнен заказ, а также требуемое количество продукта.

Спроектировать информационную систему, которая позволяет:

- 1) для выбранного цеха выдать список операций, выполняемых им;
- 2) для каждой операции список расходуемых материалов, с указанием количества;
- 3) показать список инструментов и предоставить возможность добавления нового;

- 4) выдать список используемых инструментов;
- 5) для указанного интервала дат, вывести список нарядов;
- 6) показать список операций и предоставить возможность добавления новой операции;
- 7) выдать список расходуемых материалов, используемых в различных нарядах;
- 8) выдать список товаров с указанием используемых инструментов;
- 9) показать список нарядов и предоставить возможность добавления нового;
- 10) выдать отчет о производстве товаров различными цехами, указав наименование цеха, название товара и его количество.

Задач 14. Музыкальный магазин

Информационная система должна содержать следующие данные: информацию о музыкантах, музыкальных произведениях и обстоятельствах их исполнения. Нескольких музыкантов, образующих единый коллектив, называются ансамблем. Это может быть классический оркестр, джазовая группа, квартет, квинтет и т. д. К музыкантам причисляют исполнителей (играющих на одном или нескольких инструментах), композиторов, дирижеров и руководителей ансамблей.

Кроме того, имеется информация о пластинках, которыми магазин торгует. Каждая пластинка, а точнее, ее наклейка, идентифицируется отдельным номером, так что всем копиям, отпечатанным с матрицы в разное время, присвоены одинаковые номера. На пластинке может быть записано несколько исполнений одного и того же произведения – для каждого из них в базе заведена отдельная запись. Когда выходит новая пластинка, регистрируется название выпускавшей ее компании (например, EM1), а также адрес оптовой фирмы, у которой магазин может приобрести эту пластинку. Не исключено, что компания-производитель занимается и оптовой продажей своих пластинок. Магазин фиксирует текущие оптовые и розничные цены на каждую пластинку, дату ее выпуска, количество экземпляров, проданных за прошлый год и в нынешнем году, а также число еще не распроданных пластинок. Спроектировать информационную систему, которая позволяет:

- 1) посчитать количество музыкальных произведений заданного ансамбля.
- 2) выводит название всех компакт-дисков заданного ансамбля.
- 3) показать лидеров продаж текущего года, то есть названия компакт-дисков, которые чаще всего покупали в текущем году.

4) предусмотреть изменения данных о компакт-дисках и ввод новых данных.

5) предусмотреть ввод новых данных об ансамблях.

Задача 15. Поваренная книга. Спроектировать информационную систему для хранения кулинарных рецептов или для ведения дел в ресторане.

Прежде всего, в ИС должны храниться информация о блюдах: полное описание блюда, категория блюда (завтрак, ленч, обед, мясо, домашняя птица, рыбное блюдо, салат, закуска и пр.), цена блюда, если оно подается в ресторане.

Рецепт приготовления блюда должен содержать следующую информацию:

ингредиенты – количество ингредиента для данного рецепта, единица измерения (столовая ложка, чайная ложка, грамм, чашка и т.д.);

описание рецепта, категорию приготавливаемого блюда, картинку готового блюда;

этапы приготовления - номер этапа, описание того, что нужно выполнить.

Необходимо также хранить информацию об ингредиентах (описание ингредиента, наличие ингредиента, единица измерения, поставщик ингредиента) и их поставщиках (имя поставщика, почтовый индекс, город, адрес, номер телефона, факс, сумма текущей задолженности поставщику, дата следующей оплаты, сумма следующего платежа).

Задача 16. Аукцион. Фирма "Уиллоби" занимается продажей с аукциона антикварных изделий и произведений искусства. Владельцы вещей, выставяемых на аукционах "Уиллоби", юридически являются продавцами. Лица, приобретающие зги вещи, именуются покупателями. Получив от продавцов партию предметов, фирма решает, на котором из аукционов выгоднее представить конкретный предмет. Перед проведением очередного аукциона каждой из выставяемых на нем вещей присваивается отдельный номер лота, играющий ту же роль, что и введенный ранее шифр товара. Две вещи, продаваемые на различных аукционах, могут иметь одинаковые номера лотов.

В книгах "Уиллоби" делается запись о каждом аукционе. Там отмечаются дата, место и время его проведения, а также специфика (например, выставяются картины, написанные маслом и не ранее 1900 г.). Заносятся также сведения о каждом продаваемом предмете: аукцион, на который он заявлен, номер лота, продавец, отправная цена и краткое словесное описание. Продавцу разрешается выставять любое количество вещей, а покупатель имеет право приобретать сколько ему угодно. Одно и то же лицо или фирма может выступать и как продавец, и как покупатель. После аукциона служащие

"Уиллоби" записывают фактическую цену, уплаченную за проданный предмет и фиксируют данные покупателя.

Задача 17. Остров в океане [5]. Примерно посередине воображаемого великого океана лежит воображаемый остров. Вот уже триста лет ведется подробная летопись острова. В летопись заносятся и данные обо всех людях, которые хоть какое-то время жили на острове. Записываются их имена, пол, даты рождения и смерти. Хранятся там и имена их родителей, если известно, кто они. У некоторых отсутствуют сведения об отце, у некоторых - о матери, а часть людей, судя по записям - круглые сироты. Из летописи можно узнать, когда был построен каждый дом, стоящий на острове, а если сейчас его уже нет, то, когда он был снесен, точный адрес и подробный план этого дома, кто и когда в нем жил.

Точно так же, как и столетия назад, на острове действуют предприниматели, занимающиеся, в частности, ловлей рыбы, заготовкой сахарного тростника и табака. Большинство из них делают все сами, а некоторые нанимают работников, заключая с ними контракты разной продолжительности. Имеются записи и о том, кто кого нанимал, на какую работу, когда начался и закончился контракт. Собственно, круг занятий жителей острова крайне невелик и не меняется веками. Неудивительно поэтому, что в летописи подробно описывается каждое дело, будь то рыбная ловля или выпечка хлеба. Все предприниматели — уроженцы острова. Некоторые объединяются в кооперативы, и по записям можно установить, кто участвовал в деле, когда вступил и когда вышел из него, каким паем владел. Имеются краткие описания деятельности каждого частного предпринимателя или кооператива, сообщающие, в том числе, когда было начато дело, когда и почему прекращено.

Задача 18. Фирма «Тик так» [5]. Фирма "Тик так" решила самостоятельно наладить производство некоторых товаров. С этой целью она организовала сеть специализированных цехов, каждый из которых принимает определенное участие в технологическом процессе. Каждому виду продукции, выпускаемой "Тик так", присваивается, как - обычно, свой шифр товара, под которым он значится в таблице товарных запасов. Этот же номер служит и шифром продукта. В записи с этим шифром указывается, когда была изготовлена последняя партия этого продукта, какова ее стоимость, сколько операций потребовалось.

Операцией считается законченная часть процесса производства, которая целиком выполняется силами одного цеха в соответствии с техническими требованиями, перечисленными на отдельном чертеже. Для каждого продукта и для каждой операции в "Тик так" заведена запись, содержащая описание операции, ее среднюю продолжительность и номер чертежа, по которому можно отыскать требуемый чертеж. Кроме того, указывается номер цеха, обычно производящего данную операцию.

В запись, связанную с конкретной операцией, заносятся потребные количества расходуемых материалов, а также присвоенные им шифры товара. Расходуемыми называют такие материалы, как, например, электрический кабель, который нельзя использовать повторно. Когда, готовясь к выполнению операции, расходуемый материал забирают со склада, регистрируется фактически выданное количество, соответствующий шифр товара, номер служащего, ответственного за выдачу, дата и время выдачи, номер операции и номер наряда на проведение работ, который будет обсуждаться ниже. Реально затраченное количество материала может не совпадать с требуемым, из-за того, например, что часть изготовленной продукции бракуется.

Каждый из цехов располагает многочисленными инструментами и приспособлениями. При выполнении некоторых операций их все же не хватает, и цех вынужден обращаться в центральную инструментальную за недостающими. В "Тик так" каждый тип инструмента снабжен отдельным номером и на него заведена запись со словесным описанием. Кроме того, там отмечено, какое количество инструментов этого типа выделено цехам и какое осталось в инструментальной. Экземпляры инструмента конкретного типа, например, гаечные ключи одного размера, различаются по своим индивидуальным номерам. На фирме для каждого типа инструмента имеется запись, содержащая перечень всех индивидуальных номеров. Кроме того, указаны даты их поступления на склад.

По каждой операции в "Тик так" отмечают типы и количества инструментов этих типов, которые должны использоваться при ее выполнении. Когда инструменты действительно берутся со склада, фиксируется индивидуальный номер каждого экземпляра, указываются номер заказавшего их цеха и номер наряда на проведение работ. И в этом случае потребное количество не всегда совпадает с заказанным.

Наряд на проведение работ направляется в один из цехов. Оформляется этот наряд после того, как руководство "Тик так" сочтет необходимым выпустить партию не-

которого продукта. В наряд заносятся шифр продукта, дата оформления наряда, срок, к которому должен быть выполнен заказ, а также требуемое количество продукта.

Подберите имена полей и таблиц, в которых могла бы разместиться вся эта информация. Разработанная система таблиц должна обеспечивать возможность получения разнообразных справок в ответ на запросы, например, следующего характера.

а) Каковы количества и шифры всех расходуемых материалов, фактически потраченных при выполнении работ по наряду номер 6531?

б) Располагает ли фирма всеми инструментами, необходимыми для выпуска партии продукта номер 421729?

в) Где находится (на складе или в цехе с таким-то номером) каждый инструмент типа 321? В каких операциях, и при какой продукции он применяется?

Задание 19*. Это командное задание было заданием предварительного тура на VIII Всероссийской олимпиаде по аналитике и проектированию информационных систем «Аналитик 2024»¹.

Компания организовала платную дорогу. Длительность рассматриваемого участка – 400 км. На дороге есть 15 съездов (заездов) и 4 официальных пункта пропуска (оплаты), расположенных в специальных точках. Для необорудованных пунктами пропуска заездов и участков планируется организовать онлайн отслеживание загрузки и возможность дальнейшей оплаты. Уже есть система, позволяющая осуществлять оплату (наличными, транспондером сторонней компании, через фиксацию автомобилей без транспондеров на сайте компании и оплату картой). На трассе уже смонтированы камеры с датчиками, которые позволяют передавать изображения и дополнительные параметры автомобилей.

Необходимо разработать программный модуль, который позволит контролировать и собирать статистику по количеству машин, проезжающих через разные участки. Оператор может контролировать поток, делать записи и фото, следить за зонами аварийной остановки и сигнализировать соответствующим службам. Система умеет определять по набору параметров тип автомобилей (легковой, грузовой до 12 тонн, грузовой свыше 12 тонн). Работа ведется круглосуточно.

Оператор имеет возможность выгрузки разного вида отчетов: по загрузке дороги, по количеству автомобилей с транспондером и без, по типам и количеству автомобилей

¹ <https://analyst.tilda.ws/>

в разное время суток на разных участках, по системе контроля за работоспособностью уже существующих датчиков.

Для описанного проекта нужно выполнить два задания. Задание 1

Продумать возможный функционал системы для выполнения пожеланий бизнеса. Представить описание функционала в виде диаграммы Use Cases (UML).

Критерии проверки результата задания 1

1. Полнота покрытия потребностей заказчика.
2. Необходимость и достаточность предлагаемого функционала.
3. Правильность наименования Use Cases.

Задание 2. Для выполнения задания необходимо вспомнить дисциплину «Базы данных», раздел «Проектирование баз данных».

Разработать модель данных (ER модель) для системы, предполагая, что все данные об объектах и субъектах процессов компании будут храниться в ней. Диаграмма должна быть выполнена в электронном виде в любой известной участникам Олимпиады нотации для моделирования подобных моделей (например, нотации Чена, Баркера, IDEF1X, Class Diagram(UML) и т.д.).

Результат выполнения задания должен содержать рисунок ER-модели и краткое пояснение в текстовом виде. В нем должно содержаться краткое описание модели и, если необходимо, приведены сделанные командой предположения и ограничения, в рамках которых была описана данная модель.

Критерии проверки результата задания 2

- Полнота указания основных сущностей.
- Правильность указания связей между сущностями.
- Полнота определения атрибутов для каждой из сущностей.
- Правильность наименования сущностей и связей.

Задание 20* . Это командное задание было заданием финального тура на VIII Всероссийской олимпиаде по аналитике и проектированию информационных систем «Аналитик 2024»².

Задание моделирует реальный этап в разработке информационных систем: разработка коммерческого предложения на систему. На этом этапе Заказчик озвучивает свои

² <https://analyst.tilda.ws/>

потребности в будущей системе и выбирает из предложенных решений различных исполнителей лучший вариант.

Каждая команда представляет команду исполнителей, которые должны предложить Заказчику свое видение будущей системы. В качестве входных данных для команды выступает Запрос на разработку от Заказчика. Он может быть написан в достаточно свободной форме и не является строгим техническим заданием.

Каждая команда разрабатывает концепцию будущей системы, из которой будет понятно, какие проблемы и потребности закрывает данная система, какую функциональность она реализует, какие технические решения содержит и как она будет выглядеть для пользователя. Можно дополнить диаграммы необходимыми материалами и пояснениями. Диаграммы и пояснения оформляются в презентацию.

Запрос на разработку от Заказчика (сохранены стиль и оформление Заказчика).

Наша компания хочет запустить проект по перевозке грузов автомобилями-беспилотниками.

Для пилотного запуска проекта планируется использовать платную трассу без пунктов пропуска. Сейчас уже есть необходимый набор грузовиков и терминалов на обоих концах трассы.

Для контроля движения грузовиков необходимо разработать комплексную систему, которая позволит выстроить процесс беспилотной транспортировки грузов, включая контроль движения и сбор статистики. Предложения по автоматизации должны дать нам конкурентное преимущество в сравнении с подобными проектами.

Ниже описаны некоторые идеи, которые возникли для минимизации издержек, на них стоит обратить внимание при создании системы, но мы готовы рассмотреть любые ваши предложения.

Сейчас все беспилотные автомобили контролируют реальные водители. Нужно сделать систему, которая бы позволила контролировать процесс доставки без присутствия человека в автомобиле, например, дистанционный контроль, при этом с не меньшей надежностью.

У существующих беспилотников имеются различные технические проблемы с навигацией на дороге, оборудованием, адаптацией к изменяющимся условиям на дороге и т.д. Хочется увидеть в предлагаемом решении, каким образом предлагаемая система решает хотя бы некоторые из них.

Необходимо учесть создание собственной цифровой модели дороги, адаптацию нейросети под эту модель и отслеживание актуальной информации о дорожном движении в интеграции с разными сервисами.

Для нашей компании это новая область деятельности, мы пока не очень четко представляем все возможности. Поэтому от вендоров ожидаем не только технического решения, но и продумывания основных рисков и предложения возможностей, закрывающих наиболее чувствительные проблемы.

Ожидается, что из подготовленной командами презентации предлагаемого решения будет понятно, какие проблемы будет решать будущая система, каким образом она будет это делать, какой будет ее функционал, с какими сервисами будет интегрироваться, из каких компонентов будет состоять (верхнеуровневая архитектура).

Учебное издание

Астахова Ирина Федоровна,
Трофименко Елена Владимировна,
Хицкова Юлия Владимировна

**ПРОЕКТИРОВАНИЕ
ИНФОРМАЦИОННЫХ СИСТЕМ.
ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ
ПРОЕКТИРОВАНИЕ**

Учебное пособие

Издано в авторской редакции

Подписано в печать 27.02.2025. Формат 60×84/16.

Усл. п. л. 7,7. Тираж 50 экз. Заказ 60

Издательский дом ВГУ
394018 Воронеж, пл. Ленина, 10
Отпечатано с готового оригинал-макета
в типографии Издательского дома ВГУ
394018 Воронеж, ул. Пушкинская, 3